

2025 EDITION

FREE

AI AGENTS

THE ILLUSTRATED GUIDEBOOK



**Daily Dose of
Data Science**

Avi Chawla & Akshay Pachaar
DailyDoseofDS.com

How to make the most out of this book and your time?

The reading time of this book is about 8 hours. But not all chapters will be of relevance to you. This 2-minute assessment will test your current expertise and recommend chapters that will be most useful to you.

Do you have the skills to build production-ready Agents?

Answer 10 yes/no questions and we'll email you the list of chapters you must read to level up your AI Agent skills

Take the Assessment Now!



Start The Assessment

Scan the QR code below or open this link to start the assessment. It will only take 2 minutes to complete.



<https://bit.ly/agents-assessment>

Table of contents

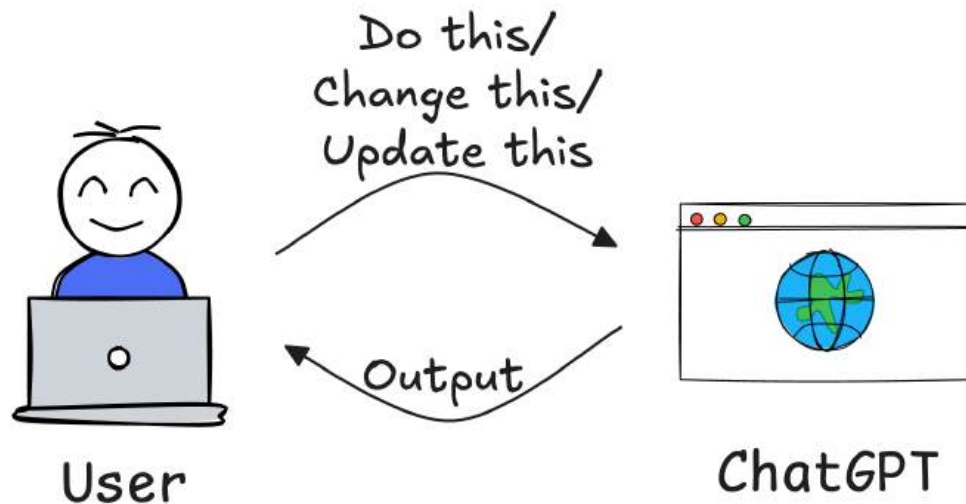
AI Agents.....	4
What is an AI Agent?.....	5
Agent vs LLM vs RAG.....	8
LLM (Large Language Model).....	8
RAG (Retrieval-Augmented Generation).....	9
Agent.....	9
Building blocks of AI Agents.....	10
1) Role-playing.....	10
2) Focus/Tasks.....	11
3) Tools.....	11
#3.1) Custom tools.....	12
#3.2) Custom tools via MCP.....	17
4) Cooperation.....	21
5) Guardrails.....	22
6) Memory.....	22
5 Agentic AI Design Patterns.....	24
#1) Reflection pattern.....	25
#2) Tool use pattern.....	25
#3) ReAct (Reason and Act) pattern.....	26
#4) Planning pattern.....	28
#5) Multi-Agent pattern.....	29
5 Levels of Agentic AI Systems.....	30
#1) Basic responder.....	31
#2) Router pattern.....	31
#3) Tool calling.....	32
#4) Multi-agent pattern.....	32
#5) Autonomous pattern.....	33

AI Agents Projects.....	34
#1) Agentic RAG.....	35
#2) Voice RAG Agent.....	41
#3) Multi-agent Flight finder.....	46
#4) Financial Analyst.....	54
#5) Brand Monitoring System.....	59
#6) Multi-agent Hotel Finder.....	67
#7) Multi-agent Deep Researcher.....	75
#8) Human-like Memory for Agents.....	82
#9) Multi-agent Book Writer.....	89
#10) Multi-agent Content Creation System.....	98
#11) Documentation Writer Flow.....	106
#12) News Generator.....	112

AI Agents

What is an AI Agent?

Imagine you want to generate a report on the latest trends in AI research. If you use a standard LLM, you might:

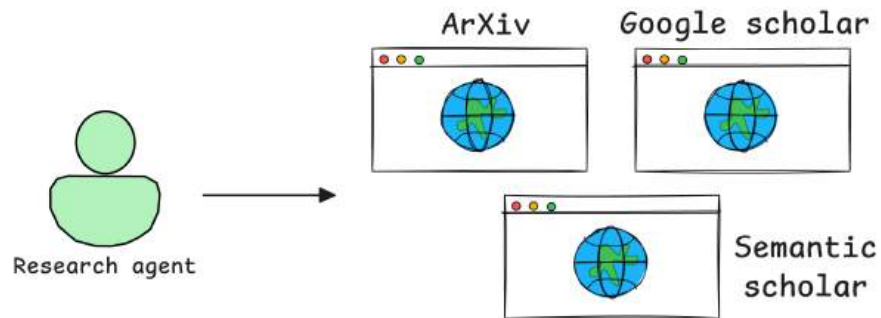


1. Ask for a summary of recent AI research papers.
2. Review the response and realize you need sources.
3. Obtain a list of papers along with citations.
4. Find that some sources are outdated, so you refine your query.
5. Finally, after multiple iterations, you get a useful output.

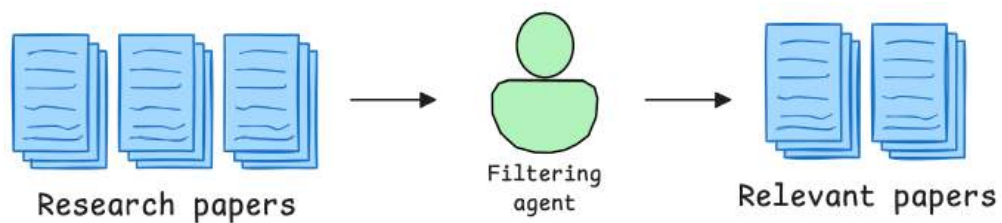
This iterative process takes time and effort, requiring you to act as the decision-maker at every step.

Now, let's see how AI agents handle this differently:

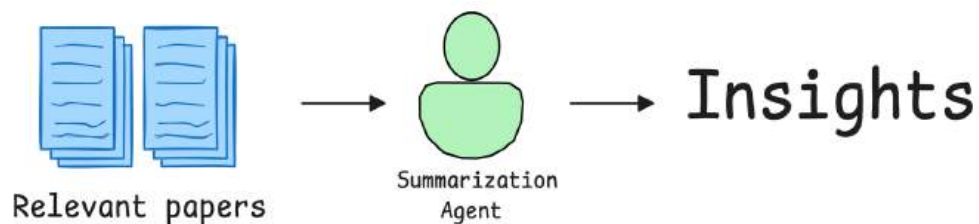
A Research Agent autonomously searches and retrieves relevant AI research papers from arXiv, Semantic Scholar, or Google Scholar.



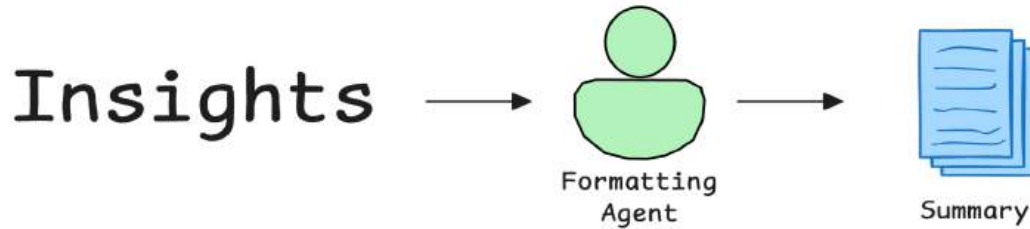
- A Filtering Agent scans the retrieved papers, identifying the most relevant ones based on citation count, publication date, and keywords.



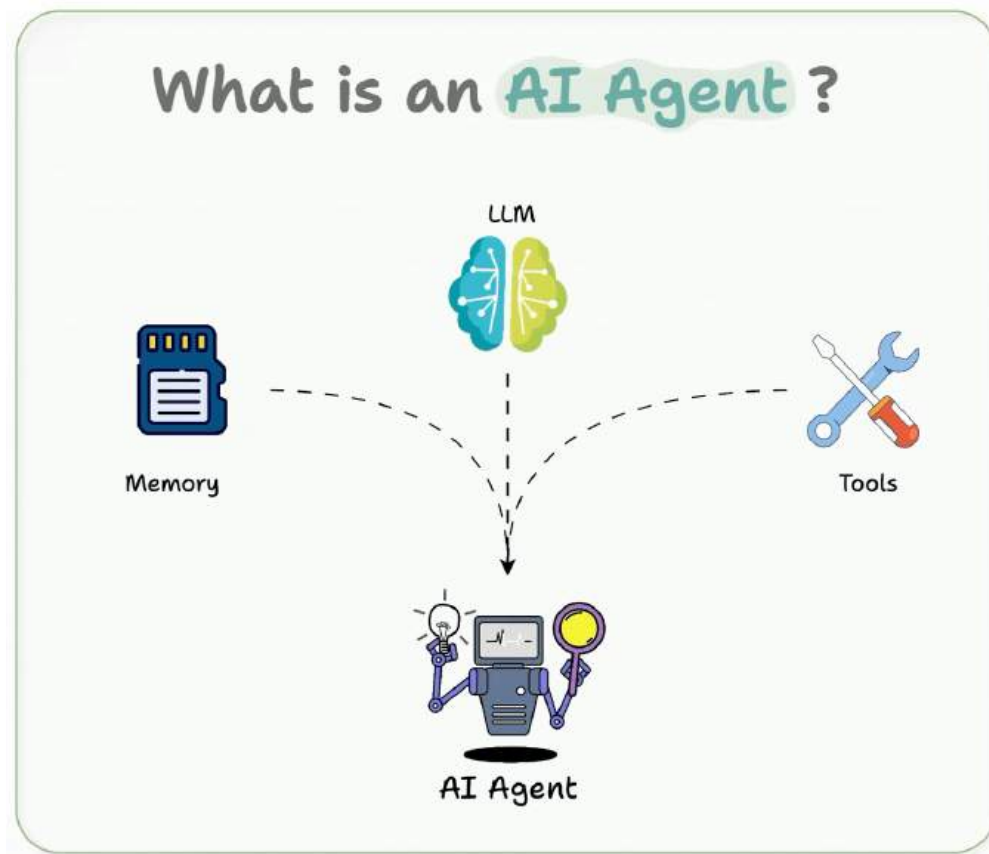
- A Summarization Agent extracts key insights and condenses them into an easy-to-read report.



- A Formatting Agent structures the final report, ensuring it follows a clear, professional layout.



Here, the AI agents not only execute the research process end-to-end but also self-refine their outputs, ensuring the final report is comprehensive, up-to-date, and well-structured - all without requiring human intervention at every step.



To formalize AI Agents are autonomous systems that can reason, think, plan, figure out the relevant sources and extract information from them when needed, take actions, and even correct themselves if something goes wrong.

Agent vs LLM vs RAG



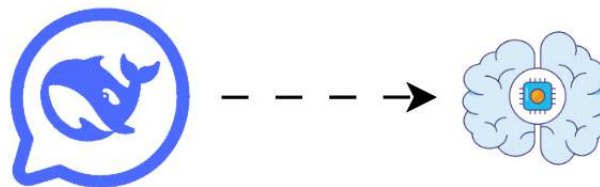
Let's break it down with a simple analogy:

- LLM is the brain.
- RAG is feeding that brain with fresh information.
- An agent is the decision-maker that plans and acts using the brain and the tools.

LLM (Large Language Model)

An LLM like GPT-4 is trained on massive text data.

It can reason, generate, summarize but only using what it already knows (i.e., its training data).



LLM is smart but static

It's smart, but static. It can't access the web, call APIs, or fetch new facts on its own.

RAG (Retrieval-Augmented Generation)

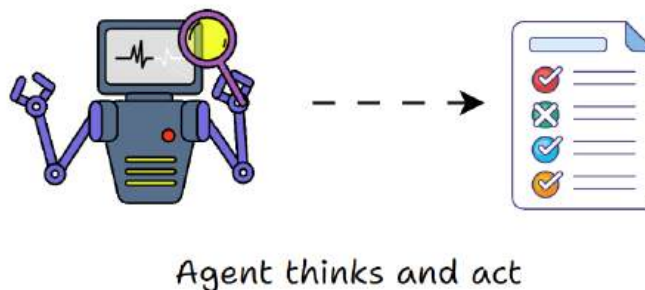
RAG enhances an LLM by retrieving external documents (from a vector DB, search engine, etc.) and feeding them into the LLM as context before generating a response.



RAG makes the LLM aware of updated, relevant info without retraining.

Agent

An Agent adds autonomy to the mix.



It doesn't just answer a question—it decides what steps to take:

Should it call a tool? Search the web? Summarize? Store info?

An Agent uses an LLM, calls tools, makes decisions, and orchestrates workflows just like a real assistant.

Building blocks of AI Agents

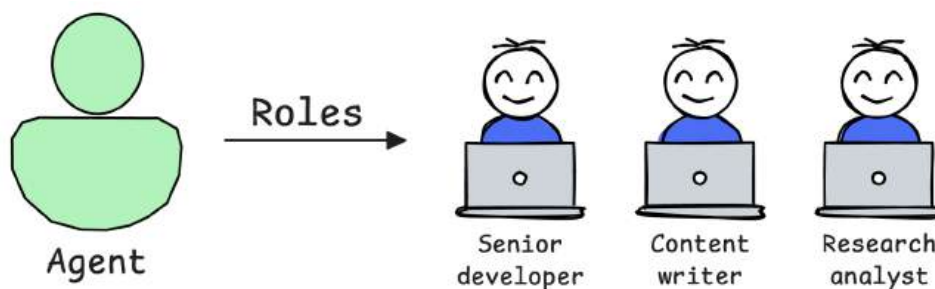
AI agents are designed to reason, plan, and take action autonomously. However, to be effective, they must be built with certain key principles in mind. There are six essential building blocks that make AI agents more reliable, intelligent, and useful in real-world applications:

1. Role-playing
2. Focus
3. Tools
4. Cooperation
5. Guardrails
6. Memory

Let's explore each of these concepts and understand why they are fundamental to building great AI agents.

1) Role-playing

One of the simplest ways to boost an agent's performance is by giving it a clear, specific role.



A generic AI assistant may give vague answers. But define it as a “Senior contract lawyer,” and it responds with legal precision and context.

Why?

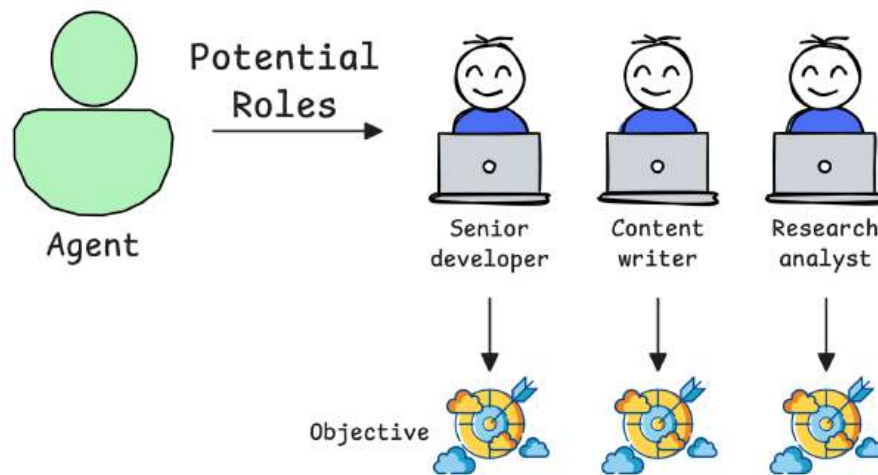
Because role assignment shapes the agent's reasoning and retrieval process. The

more specific the role, the sharper and more relevant the output.

2) Focus/Tasks

Focus is key to reducing hallucinations and improving accuracy.

Giving an agent too many tasks or too much data doesn't help - it hurts.



Overloading leads to confusion, inconsistency, and poor results.

For example, a marketing agent should stick to messaging, tone, and audience not pricing or market analysis.

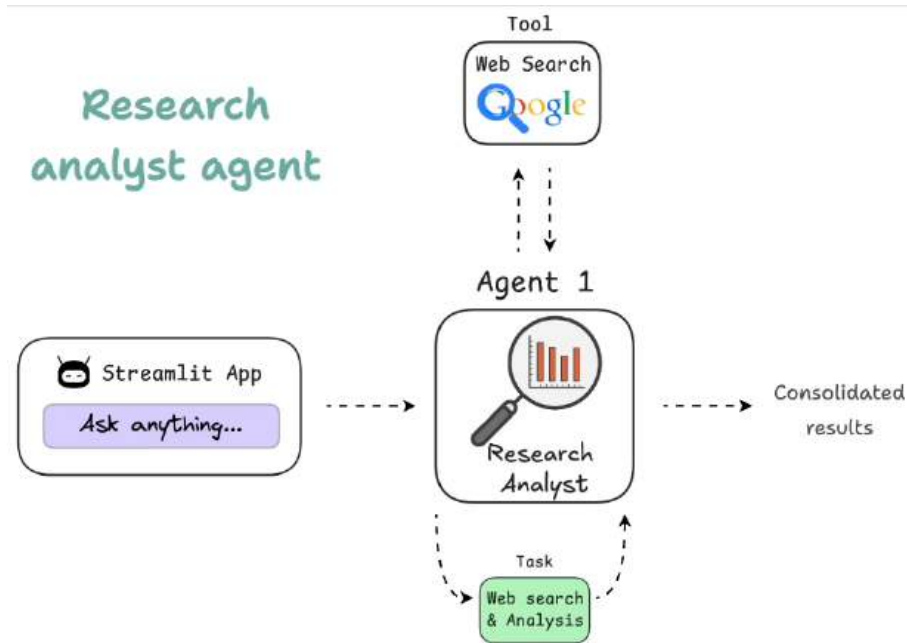
Instead of trying to make one agent do everything, a better approach is to use multiple agents, each with a specific and narrow focus.

Specialized agents perform better - every time.

3) Tools

Agents get smarter when they can use the right tools.

But more tools $\neq$ better results.



For example, an AI research agent could benefit from:

- A web search tool for retrieving recent publications.
- A summarization model for condensing long research papers.
- A citation manager to properly format references.

But if you add unnecessary tools—like a speech-to-text module or a code execution environment—it could confuse the agent and reduce efficiency.

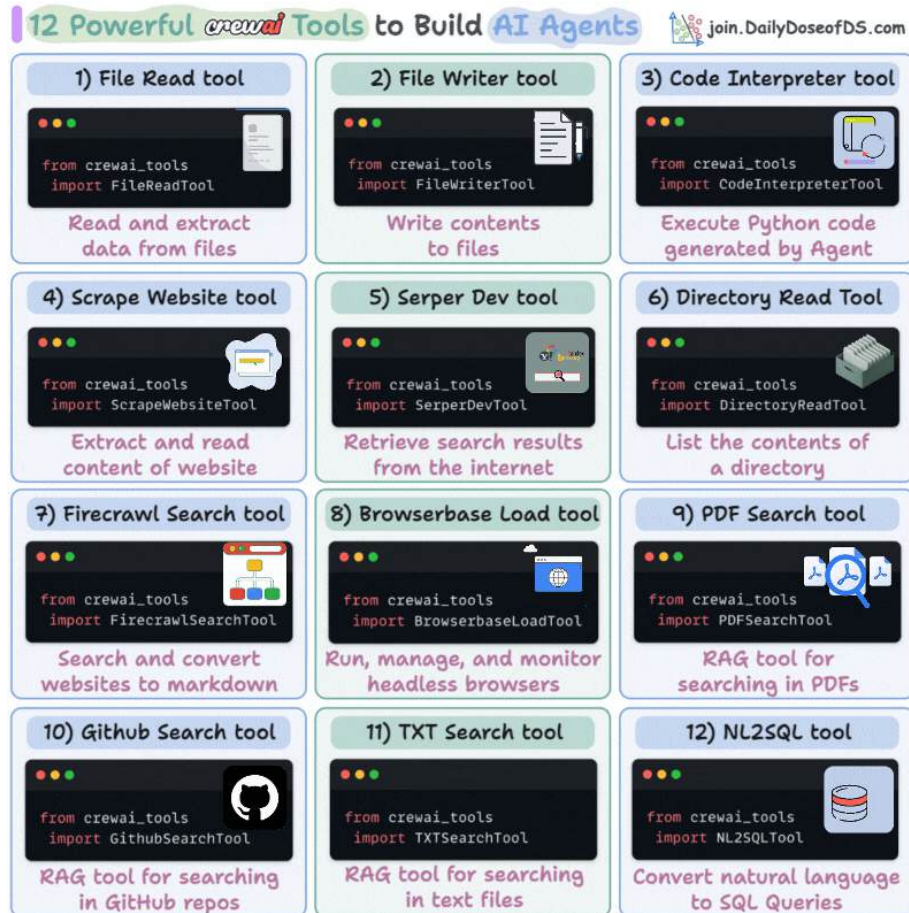
#3.1) Custom tools

While LLM-powered agents are great at reasoning and generating responses, they lack direct access to real-time information, external systems, and specialized computations.

Tools allow the Agent to:

- Search the web for real-time data.
- Retrieve structured information from APIs and databases.
- Execute code to perform calculations or data transformations.
- Analyze images, PDFs, and documents beyond just text inputs.

CrewAI supports several tools that you can integrate with Agents, as depicted below:

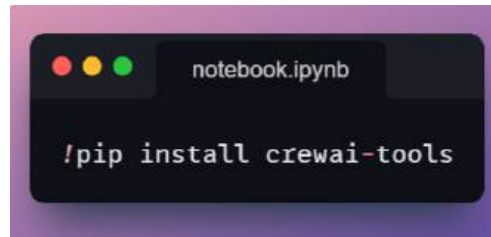


However, you may need to build custom tools at times.

In this example, we're building a real-time currency conversion tool inside CrewAI. Instead of making an LLM guess exchange rates, we integrate a custom tool that fetches live exchange rates from an external API and provides some insights.

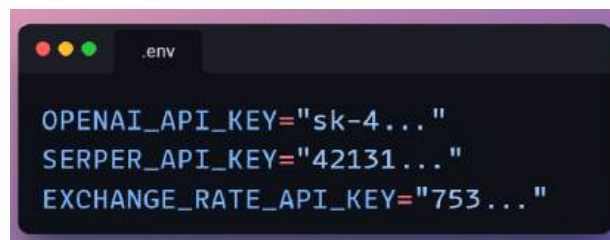
Below, let's look at how you can build one for your custom needs in the CrewAI framework.

Firstly, make sure the tools package is installed:



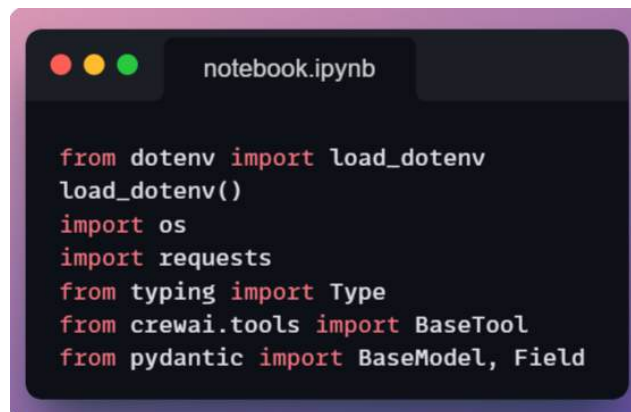
```
notebook.ipynb
!pip install crewai-tools
```

You would also need an API key from here: <https://www.exchangerate-api.com/> (it's free). Specify it in the .env file as shown below:



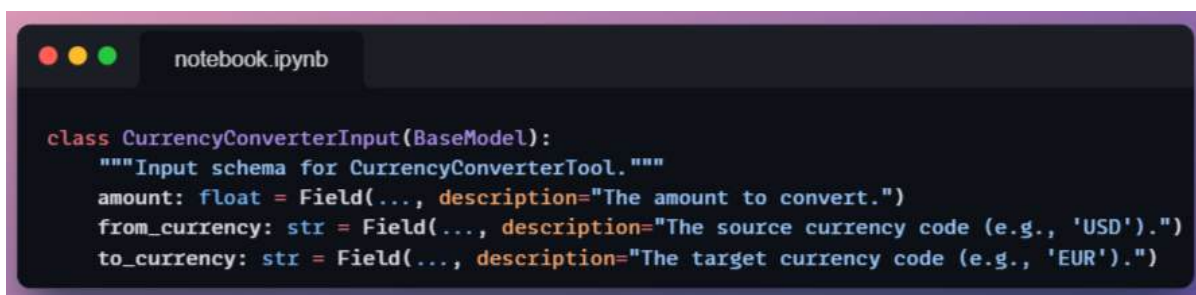
```
.env
OPENAI_API_KEY="sk-4..."
SERPER_API_KEY="42131..."
EXCHANGE_RATE_API_KEY="753..."
```

Once that's done, we start with some standard import statements:



```
notebook.ipynb
from dotenv import load_dotenv
load_dotenv()
import os
import requests
from typing import Type
from crewai.tools import BaseTool
from pydantic import BaseModel, Field
```

Next, we define the input fields the tool expects using Pydantic.



```
notebook.ipynb
class CurrencyConverterInput(BaseModel):
    """Input schema for CurrencyConverterTool."""
    amount: float = Field(..., description="The amount to convert.")
    from_currency: str = Field(..., description="The source currency code (e.g., 'USD').")
    to_currency: str = Field(..., description="The target currency code (e.g., 'EUR').")
```


Now, we define the CurrencyConverterTool by inheriting from *BaseTool*:

```
class CurrencyConverterTool(BaseTool):
    name: str = "Currency Converter Tool"
    description: str = "Converts an amount from one currency to another."
    args_schema: Type[BaseModel] = CurrencyConverterInput
    api_key: str = os.getenv("EXCHANGE_RATE_API_KEY")
```

Every tool class should have the `_run` method which we will execute whenever the Agents wants to make use of it.

For our use case, we implement it as follows:

```
def _run(self, amount: float, from_currency: str, to_currency: str) -> str:
    url = f"https://v6.exchangerate-api.com/v6/{self.api_key}/latest/{from_currency}"
    response = requests.get(url)

    if response.status_code != 200:
        return "Failed to fetch exchange rates."

    data = response.json()
    if "conversion_rates" not in data or to_currency not in data["conversion_rates"]:
        return f"Invalid currency code: {to_currency}"

    rate = data["conversion_rates"][to_currency]
    converted_amount = amount * rate
    return f"{amount} {from_currency} is equivalent to {converted_amount:.2f} {to_currency}."
```

In the above code, we fetch live exchange rates using an API request. We also handle errors if the request fails or the currency code is invalid.

Now, we define an agent that uses the tool for real-time currency analysis and attach our CurrencyConverterTool, allowing the agent to call it directly if needed:

```
from crewai import Agent

currency_analyst = Agent(
    role="Currency Analyst",
    goal="Provide real-time currency conversions and financial insights.",
    backstory=(
        "You are a finance expert with deep knowledge of global exchange rates."
        "You help users with currency conversion and financial decision-making."
    ),
    tools=[CurrencyConverterTool()], # Attach our custom tool
    verbose=True
)
```


We assign a task to the currency_analyst agent.

```
notebook.ipynb

from crewai import Task

currency_conversion_task = Task(
    description=(
        "Convert {amount} {from_currency} to {to_currency} "
        "using real-time exchange rates."
        "Provide the equivalent amount and "
        "explain any relevant financial context."
    ),
    expected_output=(
        "A detailed response including the "
        "converted amount and financial insights."
    ),
    agent=currency_analyst
)
```

Finally, we create a Crew, assign the agent to the task, and execute it.

```
notebook.ipynb

from crewai import Crew, Process

crew = Crew(
    agents=[currency_analyst],
    tasks=[currency_conversion_task],
    process=Process.sequential
)

response = crew.kickoff(inputs={
    "amount": 100,
    "from_currency": "USD",
    "to_currency": "EUR"
})
```

Printing the response, we get the following output:

```
from IPython.display import Markdown
Markdown(response.raw)
```

✓ 0.0s Python

Converting 100 USD to EUR using real-time exchange rates results in approximately 95.40 EUR.

In the financial context, it's worth noting that exchange rates can fluctuate due to various factors like economic indicators, interest rates, and geopolitical events. As of now, the conversion reflects current market conditions, which are influenced by the latest economic data releases and monetary policies in both the United States and the Eurozone. Given the recent trends, if you're planning a trip to Europe or making an investment, these rates may change, so it's beneficial to monitor them regularly.

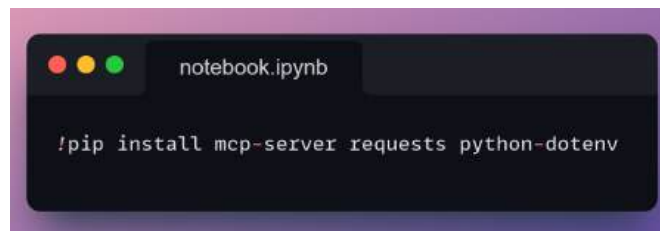
Works as expected!

#3.2) Custom tools via MCP

Now, let's take it a step further.

Instead of embedding the tool directly in every Crew, we'll expose it as a reusable MCP tool—making it accessible across multiple agents and flows via a simple server.

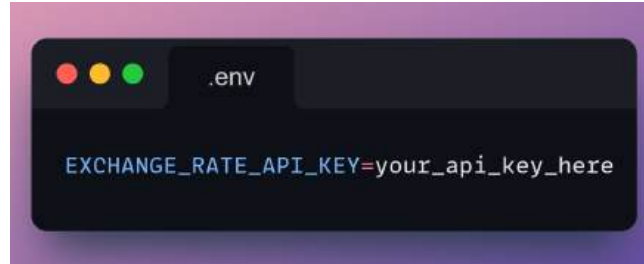
First, install the required packages:



A terminal window titled 'notebook.ipynb' with a dark background. It contains the command to install the MCP server and its dependencies using pip.

```
!pip install mcp-server requests python-dotenv
```

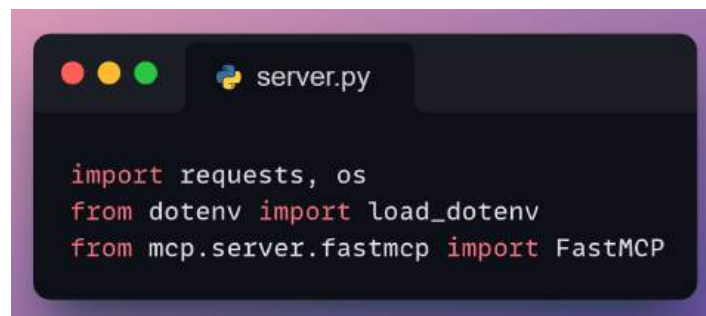
We'll continue using ExchangeRate-API in our .env file:



A terminal window titled '.env' with a dark background. It shows the configuration for the ExchangeRate-API key in the environment file.

```
EXCHANGE_RATE_API_KEY=your_api_key_here
```

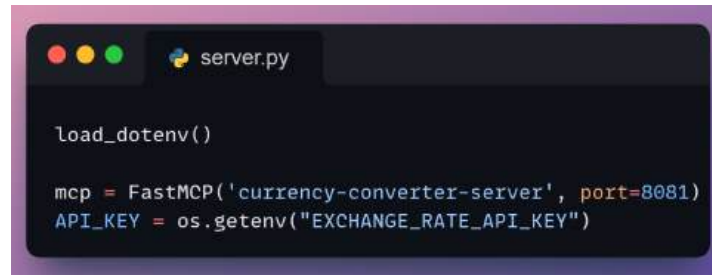
We'll now write a lightweight server.py script that exposes the currency converter tool. We start with the standard imports:



A terminal window titled 'server.py' with a dark background. It shows the Python imports for the server script, including requests, os, dotenv, and the MCP server components.

```
import requests, os
from dotenv import load_dotenv
from mcp.server.fastmcp import FastMCP
```

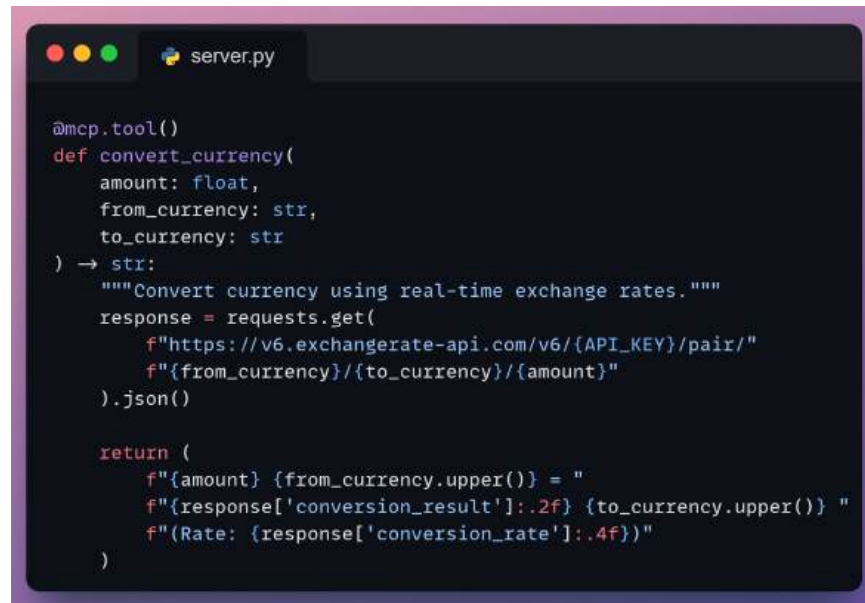
Now, we load environment variables and initialize the server:



```
load_dotenv()

mcp = FastMCP('currency-converter-server', port=8081)
API_KEY = os.getenv("EXCHANGE_RATE_API_KEY")
```

Next, we define the tool logic with `@mcp.tool()`:



```
@mcp.tool()
def convert_currency(
    amount: float,
    from_currency: str,
    to_currency: str
) → str:
    """Convert currency using real-time exchange rates."""
    response = requests.get(
        f"https://v6.exchangerate-api.com/v6/{API_KEY}/pair/"
        f"{from_currency}/{to_currency}/{amount}"
    ).json()

    return (
        f"{amount} {from_currency.upper()} = "
        f"{response['conversion_result']:.2f} {to_currency.upper()} "
        f"(Rate: {response['conversion_rate']:.4f})"
    )
```

This function takes three inputs—amount, source currency, and target currency—and returns the converted result using the real-time exchange rate API.

To make the tool accessible, we need to run the MCP server. Add this at the end of your script:

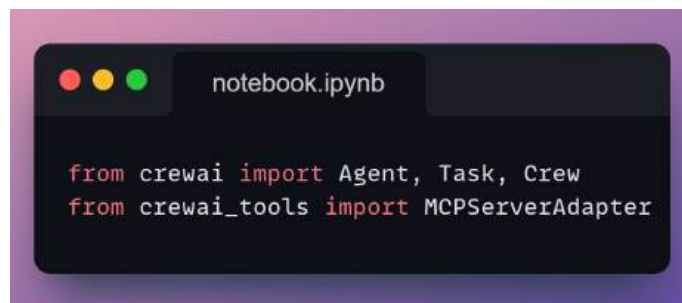


```
if __name__ == "__main__":
    mcp.run(transport="sse")
```

This starts the server and exposes your `convert_currency` tool at:
`http://localhost:8081/sse`.

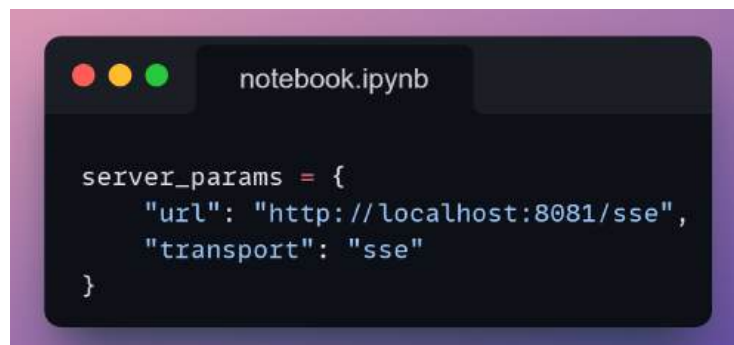
Now any CrewAI agent can connect to it using `MCPServerAdapter`. Let's now consume this tool from within a CrewAI agent.

First, we import the required CrewAI classes. We'll use `Agent`, `Task`, and `Crew` from `CrewAI`, and `MCPServerAdapter` to connect to our tool server.



```
from crewai import Agent, Task, Crew
from crewai_tools import MCPServerAdapter
```

Next, we connect to the MCP tool server. Define the server parameters to connect to your running tool (from `server.py`).



```
server_params = {
    "url": "http://localhost:8081/sse",
    "transport": "sse"
}
```

Now, we use the discovered MCP tool in an agent:

This agent is assigned the `convert_currency` tool from the remote server. It can now call the tool just like a locally defined one.

```
notebook.ipynb

currency_agent = Agent(
    role="Currency Analyst",
    goal="Convert currency using real-time exchange rates.",
    backstory=(
        "You help users convert between currencies "
        "using up-to-date market data."
    ),
    allow_delegation=False,
    tools=[mcp_tools["convert_currency"]],
)
```

We give the agent a task description:

```
notebook.ipynb

conversion_task = Task(
    description=(
        "Convert {amount} {from_currency} to {to_currency} "
        "using real-time exchange rates."
    ),
    agent=currency_agent,
    expected_output="A formatted result with exchange rate.",
)
```

Finally, we create the Crew, pass in the inputs and run it:

```
notebook.ipynb

crew = Crew(
    agents=[currency_agent], tasks=[conversion_task], verbose=True
)

result = crew.kickoff(inputs={
    "amount": 100, "from_currency": "USD", "to_currency": "INR"
})

print(result)
```

Printing the result, we get the following output:

```
print(result)

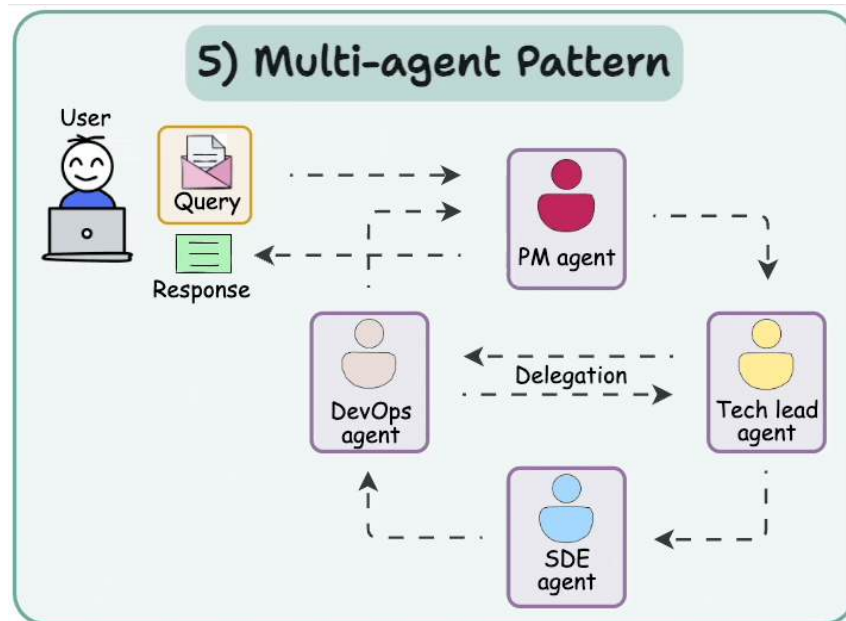
[1] ✓ 2.8s

... 100 USD = 8734.13 INR (Rate: 87.3413)
```

4) Cooperation

Multi-agent systems work best when agents collaborate and exchange feedback.

Instead of one agent doing everything, a team of specialized agents can split tasks and improve each other's outputs.



Consider an AI-powered financial analysis system:

- One agent gathers data
- another assesses risk,
- a third builds strategy,
- and a fourth writes the report

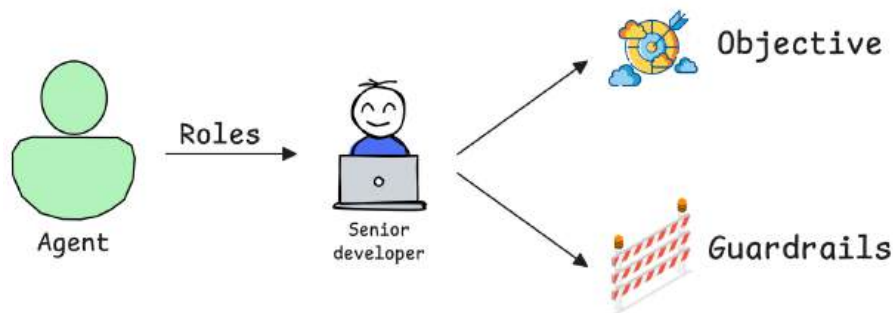
Collaboration leads to smarter, more accurate results.

The best practice is to enable agent collaboration by designing workflows where agents can exchange insights and refine their responses together.

5) Guardrails

Agents are powerful but without constraints, they can go off track. They might hallucinate, loop endlessly, or make bad calls.

Guardrails ensure that agents stay on track and maintain quality standards.



Examples of useful guardrails include:

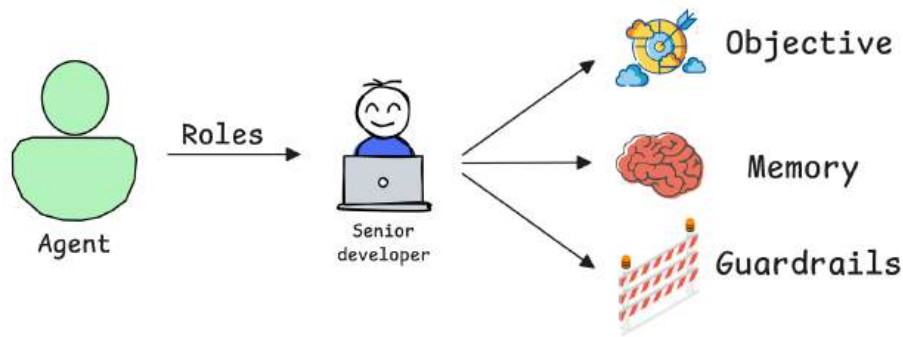
- Limiting tool usage: Prevent an agent from overusing APIs or generating irrelevant queries.
- Setting validation checkpoints: Ensure outputs meet predefined criteria before moving to the next step.
- Establishing fallback mechanisms: If an agent fails to complete a task, another agent or human reviewer can intervene.

For example, an AI-powered legal assistant should avoid outdated laws or false claims - guardrails ensure that.

6) Memory

Finally, we have memory, which is one of the most critical components of AI agents.

Without memory, an agent would start fresh every time, losing all context from previous interactions. With memory, agents can improve over time, remember past actions, and create more cohesive responses.



Different types of memory in AI agents include:

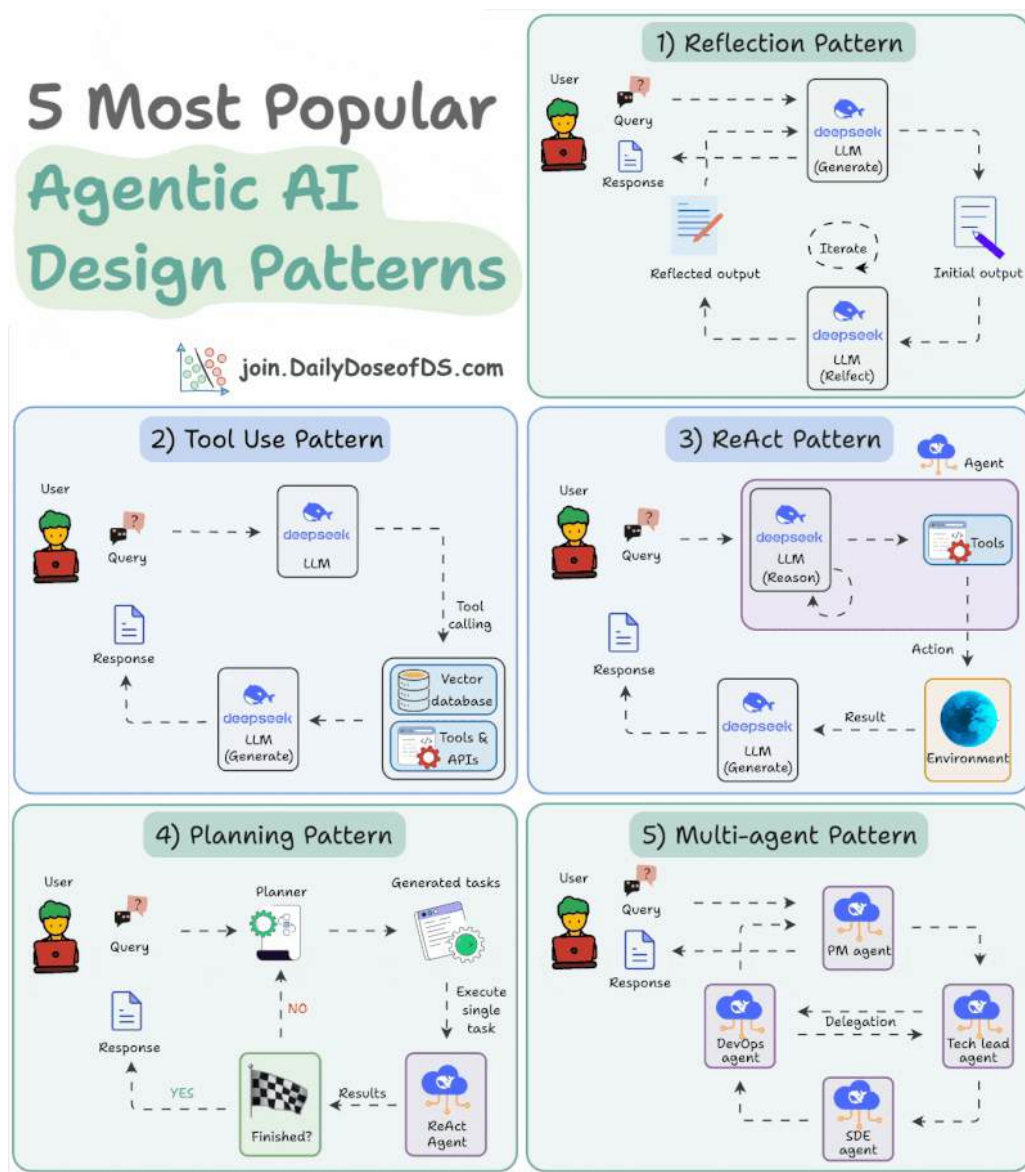
- Short-term memory – Exists only during execution (e.g., recalling recent conversation history).
- Long-term memory – Persists after execution (e.g., remembering user preferences over multiple interactions).
- Entity memory – Stores information about key subjects discussed (e.g., tracking customer details in a CRM agent).

For example, in an AI-powered tutoring system, memory allows the agent to recall past lessons, tailor feedback, and avoid repetition.

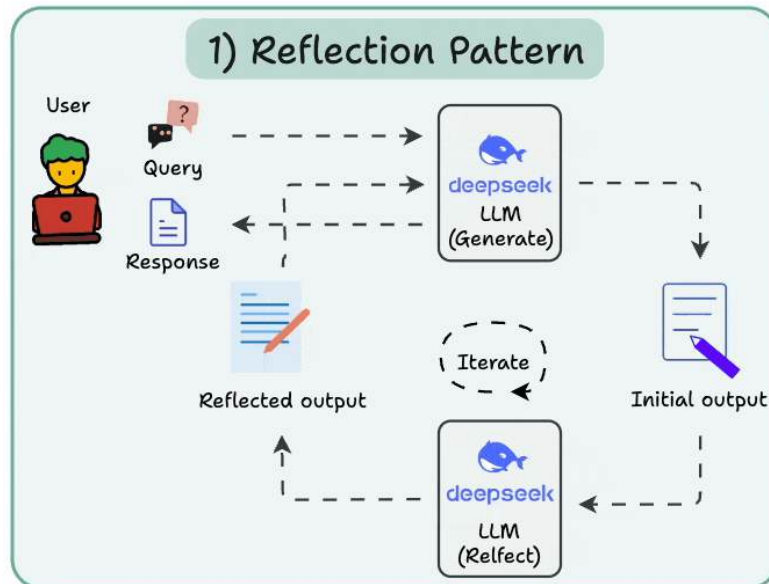
5 Agentic AI Design Patterns

Agentic behaviors allow LLMs to refine their output by incorporating self-evaluation, planning, and collaboration!

The following visual depicts the 5 most popular design patterns employed in building AI agents.

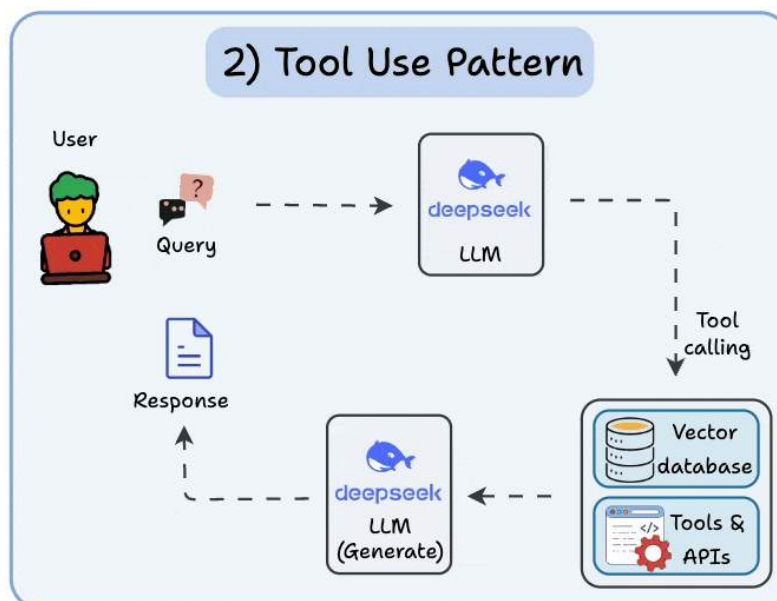


#1) Reflection pattern



The AI reviews its own work to spot mistakes and iterate until it produces the final response.

#2) Tool use pattern

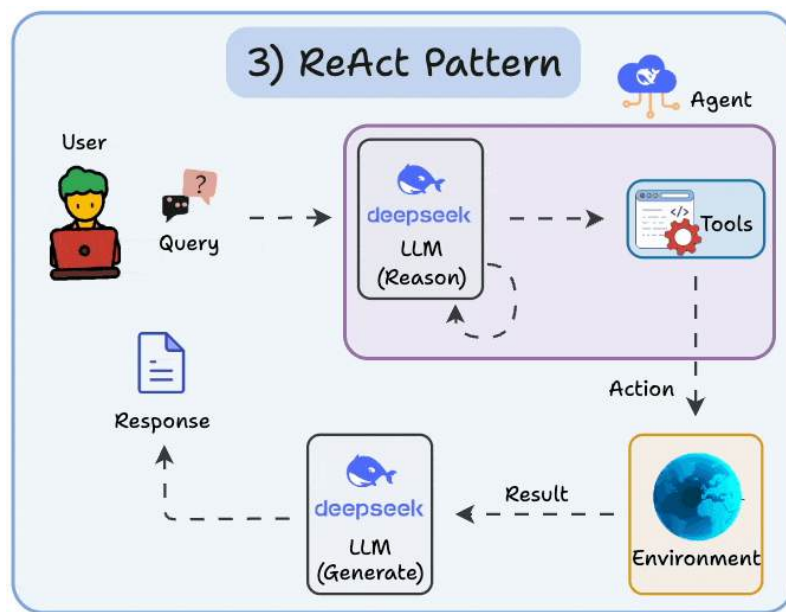


Tools allow LLMs to gather more information by:

- Querying a vector database
- Executing Python scripts
- Invoking APIs, etc.

This is helpful since the LLM is not solely reliant on its internal knowledge.

#3) ReAct (Reason and Act) pattern

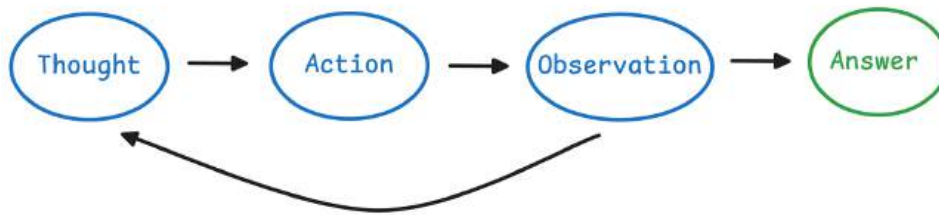


ReAct combines the above two patterns:

- The Agent reflects on the generated outputs.
- It interacts with the world using tools.

A ReAct agent operates in a loop of Thought → Action → Observation, repeating until it reaches a solution or a final answer. This is analogous to how humans solve problems:

ReAct Pattern



Note: Frameworks like CrewAI primarily use this by default.

To understand this, consider the output of a multi-agent system below:

```
# Agent: News Collector
## Task: Search for the latest news on Agent2Agent Protocol

# Agent: News Collector
## Final Answer:
1. ==Agent2Agent Protocol Expands Blockchain Communication" - November 10, 2023==
The Agent2Agent Protocol has announced its latest advancements in facilitating seamless communication between decentralized agents on various blockchains.

2. ==Agent2Agent Protocol Announces Strategic Partnership" - November 7, 2023==
A significant partnership has been formed between Agent2Agent Protocol and a leading technology firm to broaden their capabilities in artificial intelligence and machine learning.

# Agent: News Reporter
## Task: Create a news headline on Agent2Agent Protocol from the News Collector. Also validate the news is relevant to Agent2Agent Protocol using the internet.

# Agent: News Reporter
## Thought: I need to validate the relevance of the news related to the Agent2Agent Protocol by searching the internet for recent developments.
## Using tool: Search the internet
## Tool Input:
{"search_query": "\"Agent2Agent Protocol news November 2023\""}
## Tool Output:
[{"title": "Agent2Agent Protocol Announces Strategic Partnership", "link": "https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/", "snippet": "Today, we're launching a new, open protocol called Agent2Agent (A2A), with support and contributions from more than 50 technology partners."}]

# Agent: News Reporter
## Thought: The search results indicate that the Agent2Agent Protocol is currently being discussed in various contexts, especially in relation to blockchain and artificial intelligence.
## Using tool: Extract the internet
## Tool Input:
{"search_query": "\"Agent2Agent Protocol latest news\""}
## Tool Output:
[{"title": "Agent2Agent Protocol Announces Strategic Partnership", "link": "https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/", "snippet": "Today, we're launching a new, open protocol called Agent2Agent (A2A), with support and contributions from more than 50 technology partners."}]

# Agent: News Reporter
## Thought: The Agent2Agent Protocol has made significant strides recently with new announcements and partnerships involving AI and blockchain technology.
## Using tool: Search the internet
## Tool Input:
{"search_query": "\"Agent2Agent Protocol news updates\""}
## Tool Output:
[{"title": "Agent2Agent Protocol Announces Strategic Partnership", "link": "https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/", "snippet": "Today, we're launching a new, open protocol called Agent2Agent (A2A), with support and contributions from more than 50 technology partners."}]
```

As shown above, the Agent is going through a series of thought activities before producing a response.

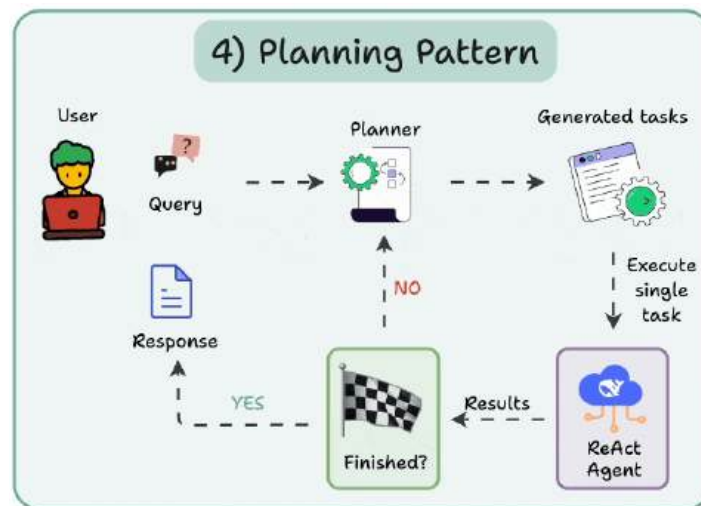
This is the ReAct pattern in action!

More specifically, under the hood, many such frameworks use the ReAct (Reasoning and Acting) pattern to let LLM think through problems and use tools to act on the world.

For example, an agent in CrewAI typically alternates between reasoning about a task and acting (using a tool) to gather information or execute steps, following the ReAct paradigm.

This enhances an LLM agent's ability to handle complex tasks and decisions by combining chain-of-thought reasoning with external tool use like in this [ReAct implementation from scratch](#).

#4) Planning pattern



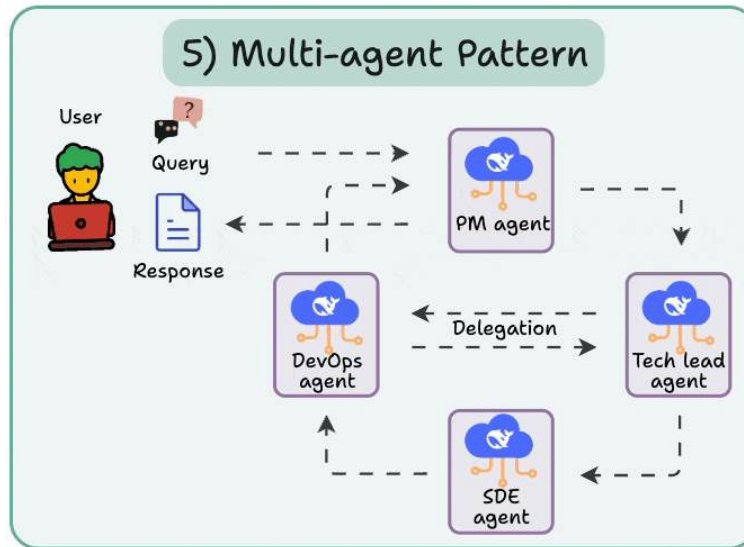
Instead of solving a task in one go, the AI creates a roadmap by:

- Subdividing tasks
- Outlining objectives

This strategic thinking solves tasks more effectively.

Note: In CrewAI, specify `planning=True` to use Planning.

#5) Multi-Agent pattern



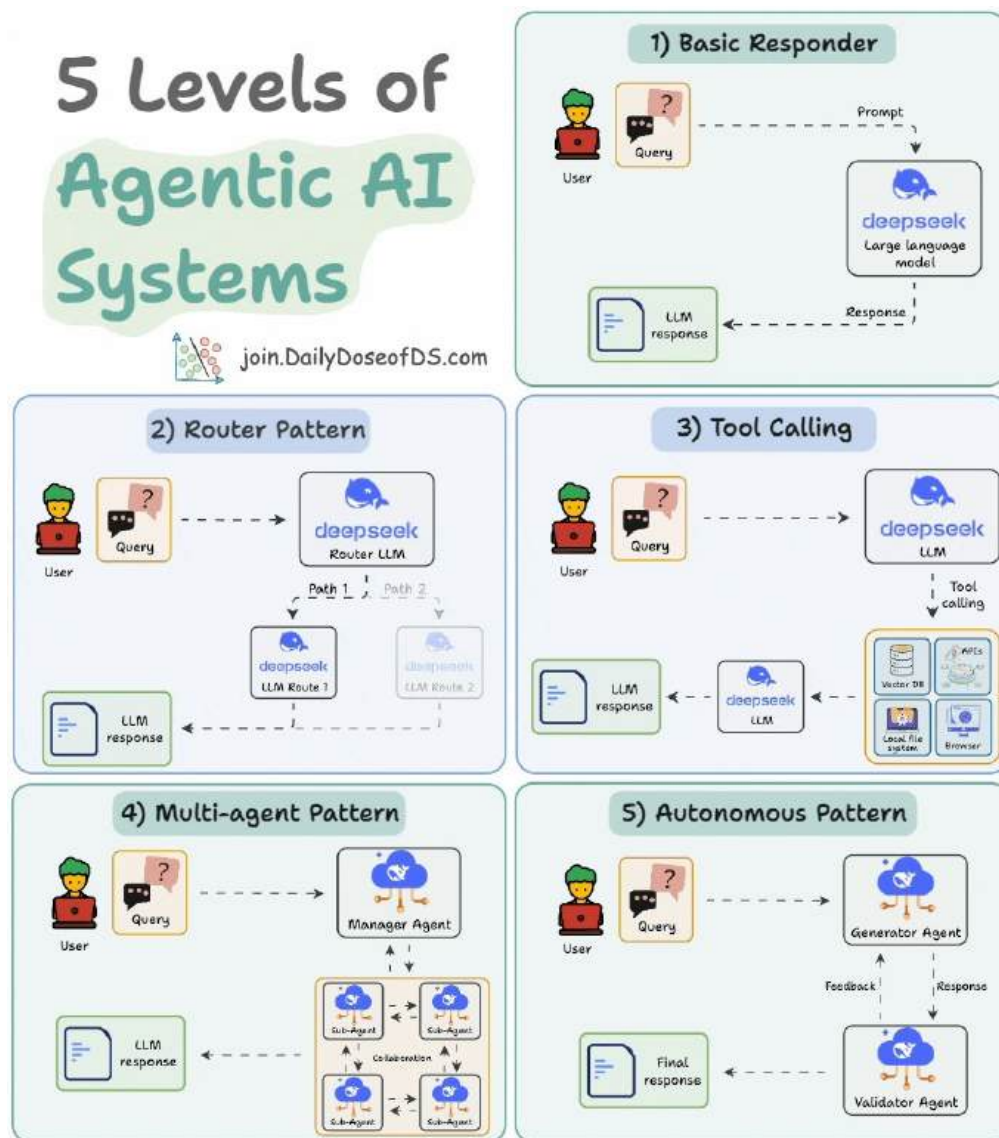
- There are several agents, each with a specific role and task.
- Each agent can also access tools.

All agents work together to deliver the final outcome, while delegating tasks to other agents if needed.

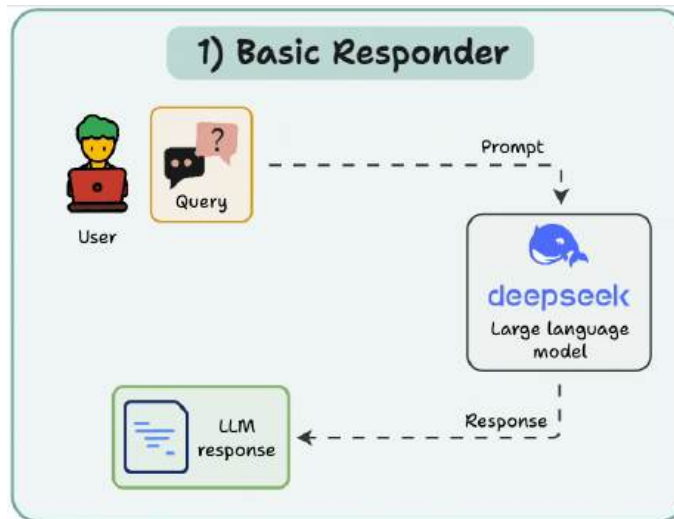
5 Levels of Agentic AI Systems

Agentic AI systems don't just generate text; they can make decisions, call functions, and even run autonomous workflows.

The visual explains 5 levels of AI agency—from simple responders to fully autonomous agents.



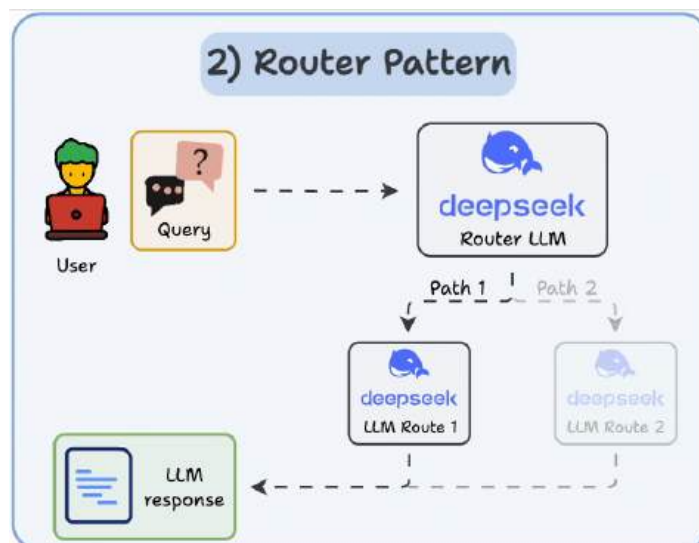
#1) Basic responder



A human guides the entire flow.

The LLM is just a generic responder that receives an input and produces an output. It has little control over the program flow.

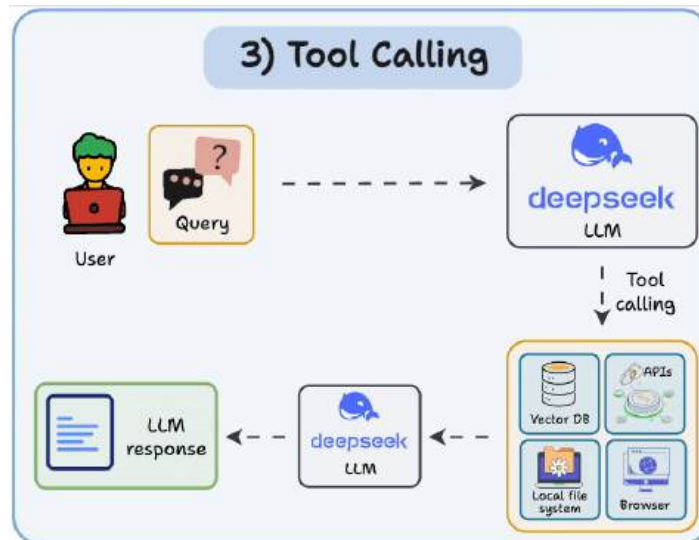
#2) Router pattern



A human defines the paths/functions that exist in the flow.

The LLM makes basic decisions on which function or path it can take.

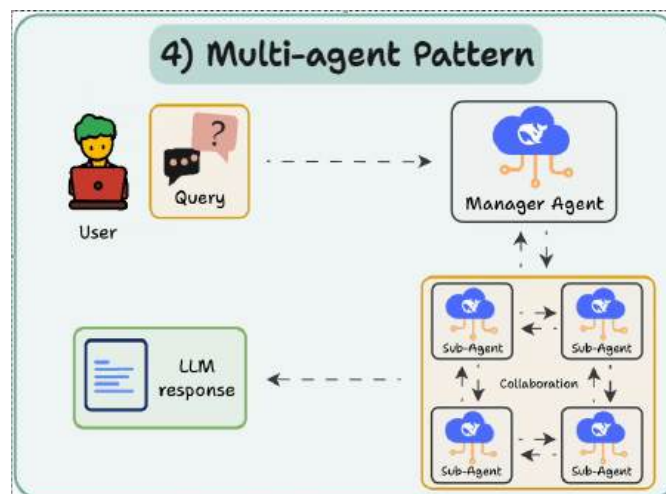
#3) Tool calling



A human defines a set of tools the LLM can access to complete a task.

LLM decides when to use them and also the arguments for execution.

#4) Multi-agent pattern

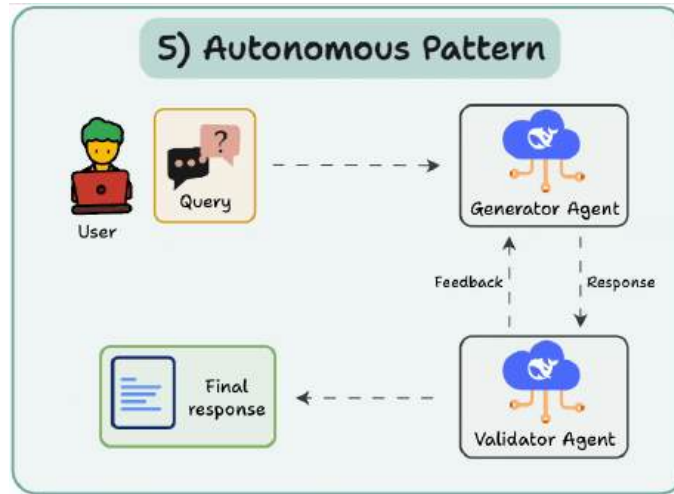


A manager agent coordinates multiple sub-agents and decides the next steps iteratively.

A human lays out the hierarchy between agents, their roles, tools, etc.

The LLM controls execution flow, deciding what to do next.

#5) Autonomous pattern

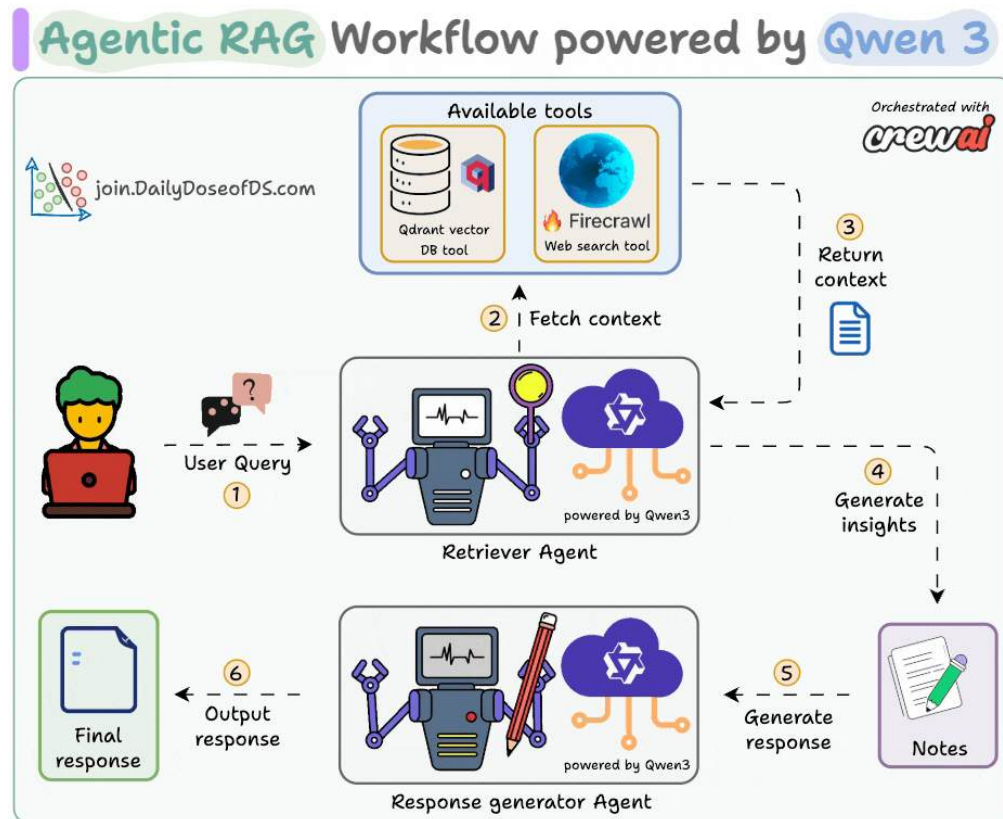


The most advanced pattern, wherein, the LLM generates and executes new code independently, effectively acting as an independent AI developer.

AI Agents Projects

#1) Agentic RAG

Build a RAG pipeline with agentic capabilities that can dynamically fetch context from different sources, like a vector DB and the internet.



Tech stack:

- CrewAI for Agent orchestration.
- Firecrawl for web search.
- LightningAI's LitServe for deployment.

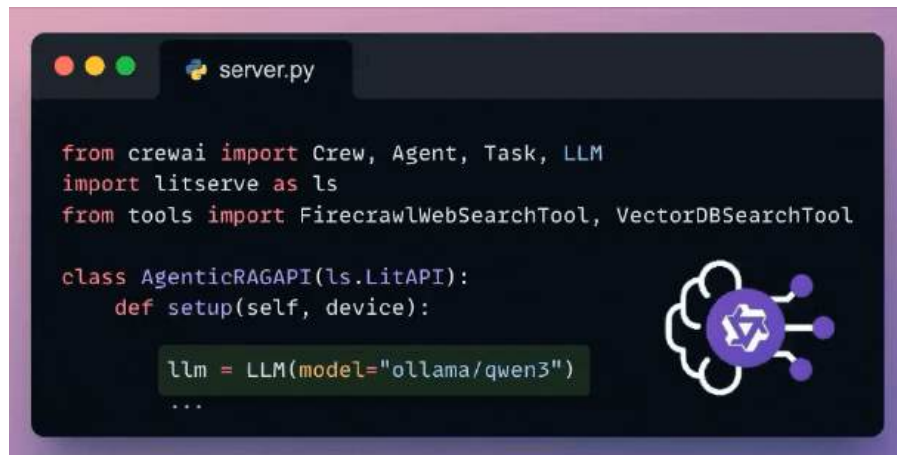
Workflow:

- The Retriever Agent accepts the user query.
- It invokes a relevant tool (Firecrawl web search or vector DB tool) to get context and generate insights.
- The Writer Agent generates a response.

Let's implement it!

#1) Set up LLM

CrewAI seamlessly integrates with all popular LLMs and providers. Here's how we set up a local Qwen 3 via Ollama:



```
server.py

from crewai import Crew, Agent, Task, LLM
import litserve as ls
from tools import FirecrawlWebSearchTool, VectorDBSearchTool

class AgenticRAGAPI(ls.LitAPI):
    def setup(self, device):

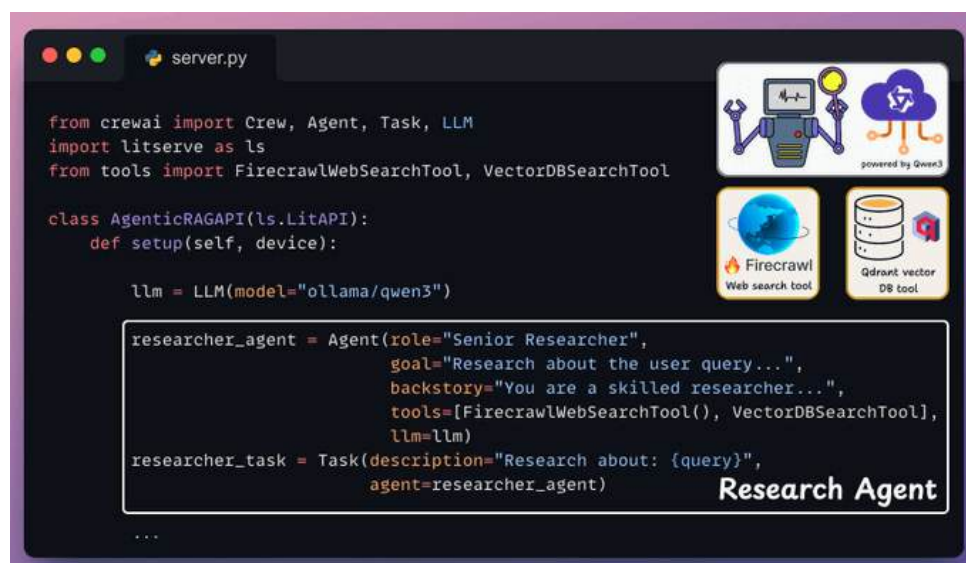
        llm = LLM(model="ollama/qwen3")
        ...
```

The screenshot shows a code editor window titled 'server.py' with a dark background. It contains Python code for setting up a CrewAI application. The code imports 'Crew', 'Agent', 'Task', and 'LLM' from 'crewai', and 'litserve as ls' and two tool classes from 'tools'. A class 'AgenticRAGAPI' inherits from 'ls.LitAPI' and has a 'setup' method. Inside the 'setup' method, an 'llm' object is created using 'LLM(model="ollama/qwen3")'. To the right of the code, there is a purple circular icon with a white brain-like shape inside, connected to several nodes.

#2) Define Research Agent and Task

This Agent accepts the user query and retrieves the relevant context using a vectorDB tool and a web search tool powered by Firecrawl.

Again, put this in the LitServe setup() method:



```
server.py

from crewai import Crew, Agent, Task, LLM
import litserve as ls
from tools import FirecrawlWebSearchTool, VectorDBSearchTool

class AgenticRAGAPI(ls.LitAPI):
    def setup(self, device):

        llm = LLM(model="ollama/qwen3")

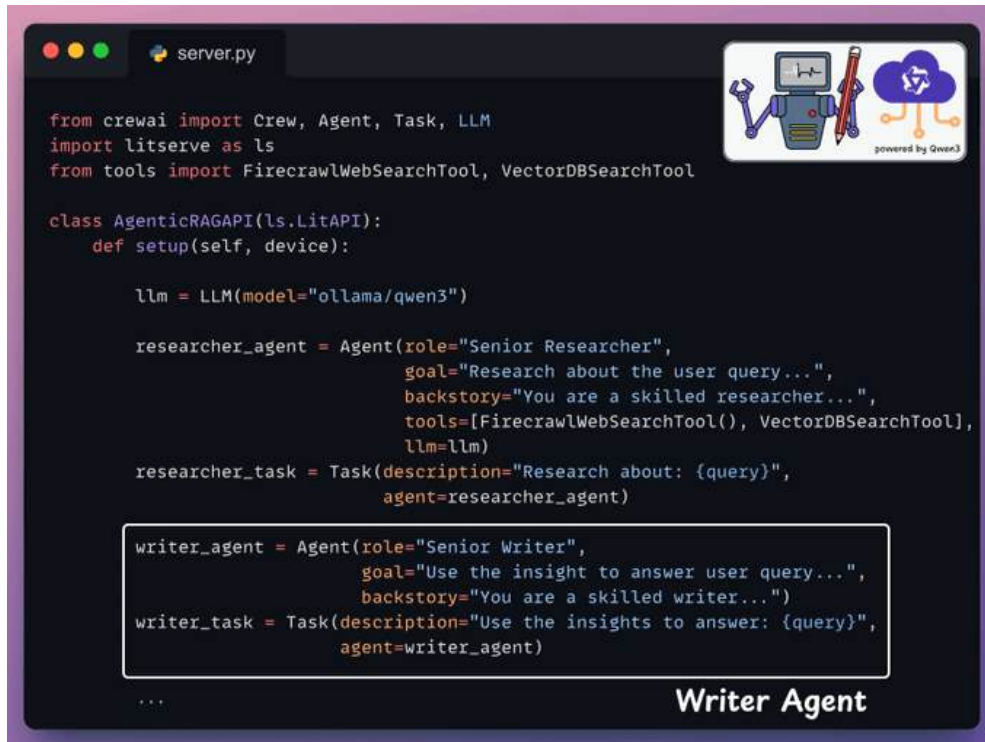
        researcher_agent = Agent(role="Senior Researcher",
                                goal="Research about the user query...",
                                backstory="You are a skilled researcher...",
                                tools=[FirecrawlWebSearchTool(), VectorDBSearchTool()],
                                llm=llm)
        researcher_task = Task(description="Research about: {query}",
                                agent=researcher_agent)
        ...
```

The screenshot shows a code editor window titled 'server.py' with a dark background. It contains Python code for setting up a CrewAI application. The code imports 'Crew', 'Agent', 'Task', and 'LLM' from 'crewai', and 'litserve as ls' and two tool classes from 'tools'. A class 'AgenticRAGAPI' inherits from 'ls.LitAPI' and has a 'setup' method. Inside the 'setup' method, an 'llm' object is created using 'LLM(model="ollama/qwen3")'. Then, a 'researcher_agent' is created with the role 'Senior Researcher', a goal, a backstory, and the two tool classes. Finally, a 'researcher_task' is created with a description and the 'researcher_agent' as the agent. To the right of the code, there are four icons: a robot head, a cloud with a lightning bolt, a globe with a magnifying glass, and a database cylinder. Below these icons are the labels 'powered by Qwen3', 'Firecrawl Web search tool', and 'Qdrant vector DB tool'. At the bottom right, there is a text label 'Research Agent'.

#3) Define Writer Agent and Task

Next, the Writer Agent accepts the insights from the Researcher Agent to generate a response.

Yet again, we add this in the LitServe setup method:



```
from crewai import Crew, Agent, Task, LLM
import litserve as ls
from tools import FirecrawlWebSearchTool, VectorDBSearchTool

class AgenticRAGAPI(ls.LitAPI):
    def setup(self, device):

        llm = LLM(model="ollama/qwen3")

        researcher_agent = Agent(role="Senior Researcher",
                                goal="Research about the user query...",
                                backstory="You are a skilled researcher...",
                                tools=[FirecrawlWebSearchTool(), VectorDBSearchTool],
                                llm=llm)

        researcher_task = Task(description="Research about: {query}",
                                agent=researcher_agent)

        writer_agent = Agent(role="Senior Writer",
                              goal="Use the insight to answer user query...",
                              backstory="You are a skilled writer...")
        writer_task = Task(description="Use the insights to answer: {query}",
                              agent=writer_agent)

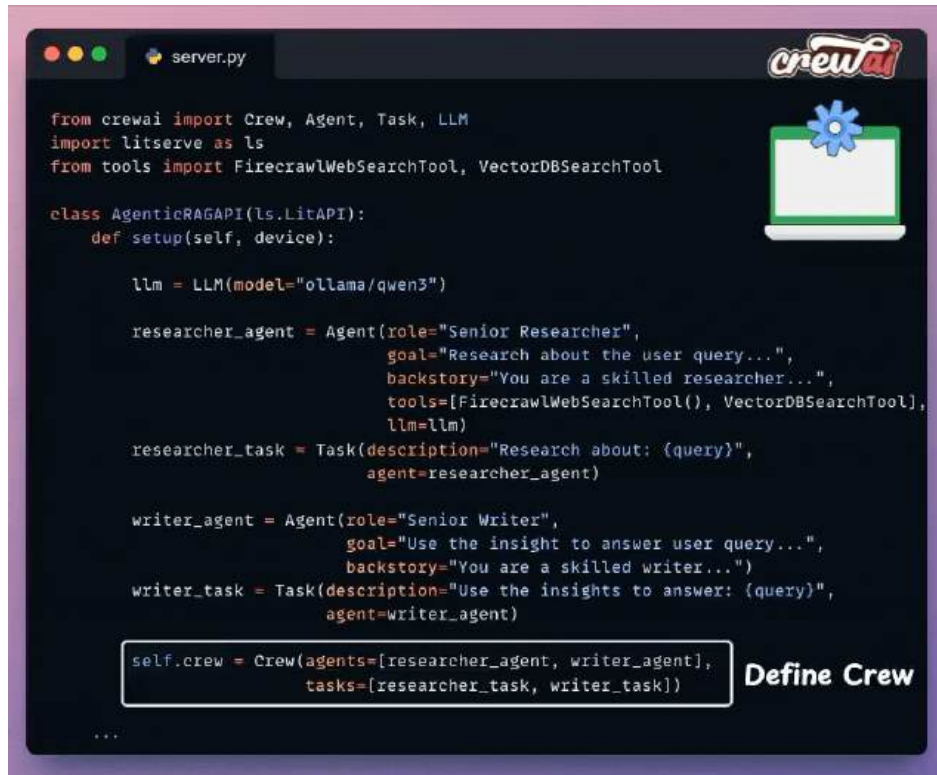
        ...
```

Writer Agent

#4) Set up the Crew

Once we have defined the Agents and their tasks, we orchestrate them into a crew using CrewAI and put that into a setup method.

Check this code:



```
from crewai import Crew, Agent, Task, LLM
import litserve as ls
from tools import FirecrawlWebSearchTool, VectorDBSearchTool

class AgenticRAGAPI(ls.LitAPI):
    def setup(self, device):

        llm = LLM(model="ollama/qwen3")

        researcher_agent = Agent(role="Senior Researcher",
                                goal="Research about the user query...",
                                backstory="You are a skilled researcher...",
                                tools=[FirecrawlWebSearchTool(), VectorDBSearchTool()],
                                llm=llm)

        researcher_task = Task(description="Research about: {query}",
                                agent=researcher_agent)

        writer_agent = Agent(role="Senior Writer",
                              goal="Use the insight to answer user query...",
                              backstory="You are a skilled writer...")

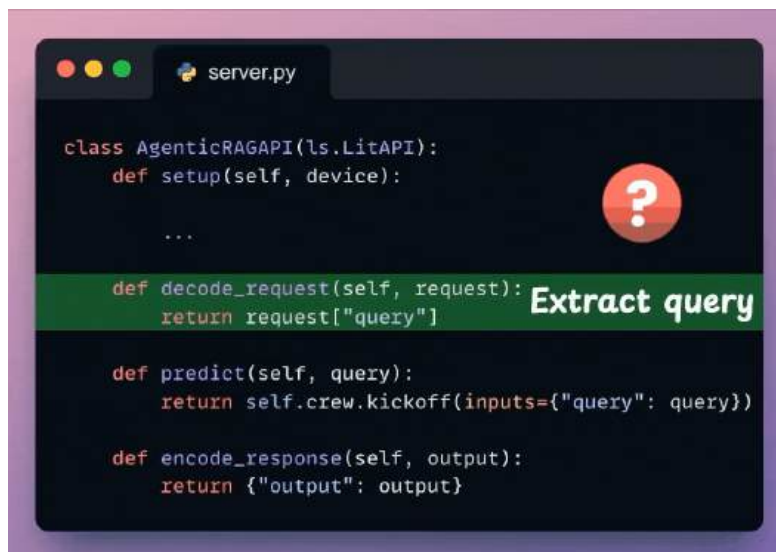
        writer_task = Task(description="Use the insights to answer: {query}",
                              agent=writer_agent)

        self.crew = Crew(agents=[researcher_agent, writer_agent],
                          tasks=[researcher_task, writer_task])
```

Define Crew

#5) Decode request

With that, we have orchestrated the Agentic RAG workflow, which will be executed upon an incoming request. Next, from the incoming request body, we extract the user query. Check the highlighted code below:



```
class AgenticRAGAPI(ls.LitAPI):
    def setup(self, device):
        ...

    def decode_request(self, request):
        return request["query"]

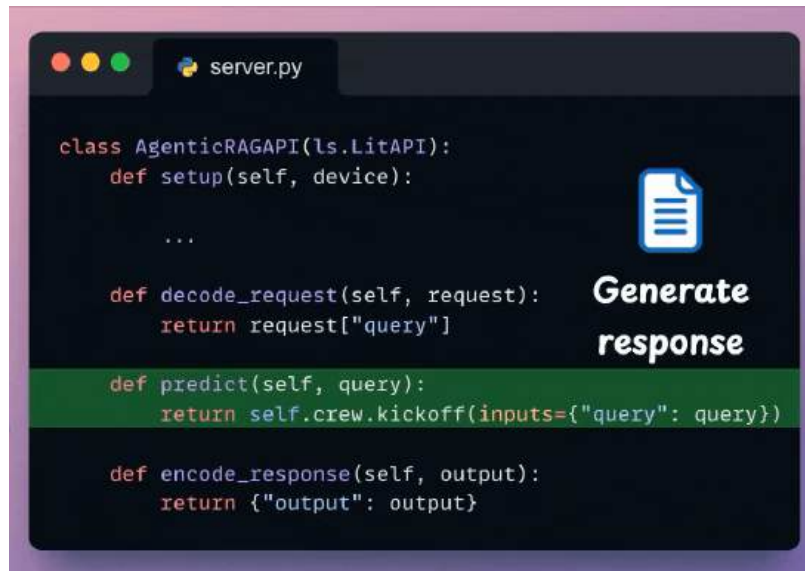
    def predict(self, query):
        return self.crew.kickoff(inputs={"query": query})

    def encode_response(self, output):
        return {"output": output}
```

Extract query

#6) Predict

We use the decoded user query and pass it to the Crew defined earlier to generate a response from the model. Check the highlighted code below:



```
class AgenticRAGAPI(ls.LitAPI):
    def setup(self, device):
        ...

    def decode_request(self, request):
        return request["query"]

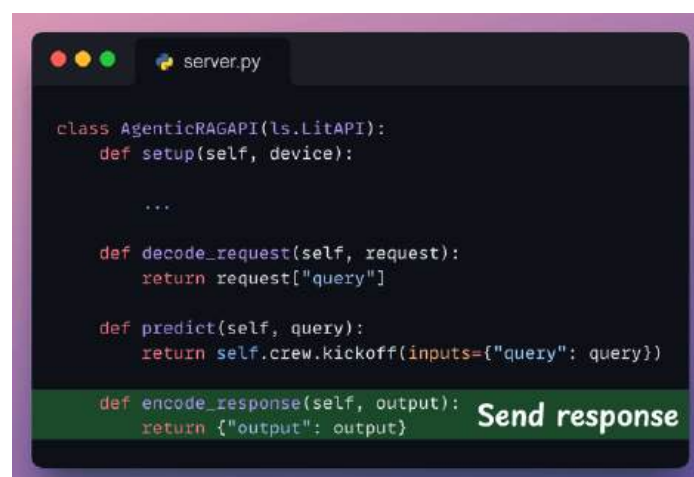
    def predict(self, query):
        return self.crew.kickoff(inputs={"query": query})

    def encode_response(self, output):
        return {"output": output}
```

#7) Encode response

Here, we can post-process the response & send it back to the client.

Note: LitServe internally invokes these methods in order: decode_request → predict → encode_request. Check the highlighted code below:



```
class AgenticRAGAPI(ls.LitAPI):
    def setup(self, device):
        ...

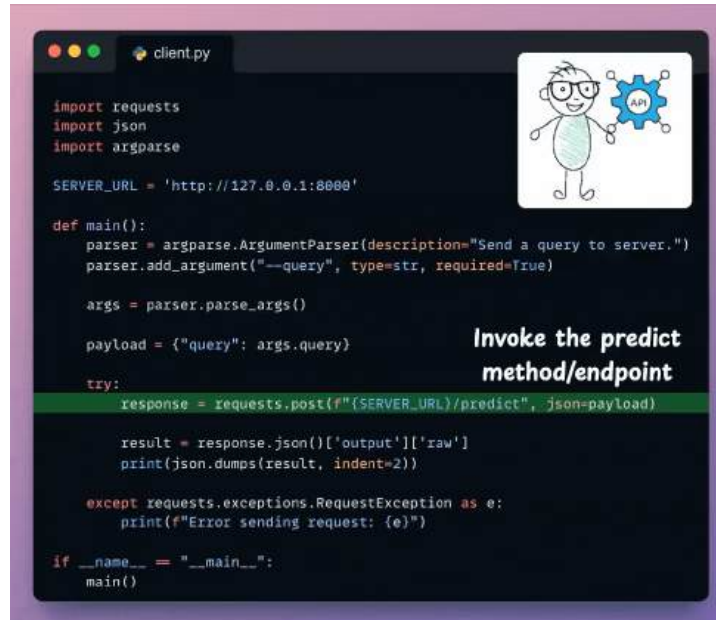
    def decode_request(self, request):
        return request["query"]

    def predict(self, query):
        return self.crew.kickoff(inputs={"query": query})

    def encode_response(self, output):
        return {"output": output}
```


#8) With that, we are done with the server code.

Next, we have the basic client code to invoke the API we created using the requests Python library. Check this:



```
import requests
import json
import argparse

SERVER_URL = 'http://127.0.0.1:8000'

def main():
    parser = argparse.ArgumentParser(description="Send a query to server.")
    parser.add_argument("--query", type=str, required=True)

    args = parser.parse_args()

    payload = {"query": args.query}

    try:
        response = requests.post(f"{SERVER_URL}/predict", json=payload)

        result = response.json()['output']['raw']
        print(json.dumps(result, indent=2))

    except requests.exceptions.RequestException as e:
        print(f"Error sending request: {e}")

if __name__ == "__main__":
    main()
```

Invoke the predict method/endpoint

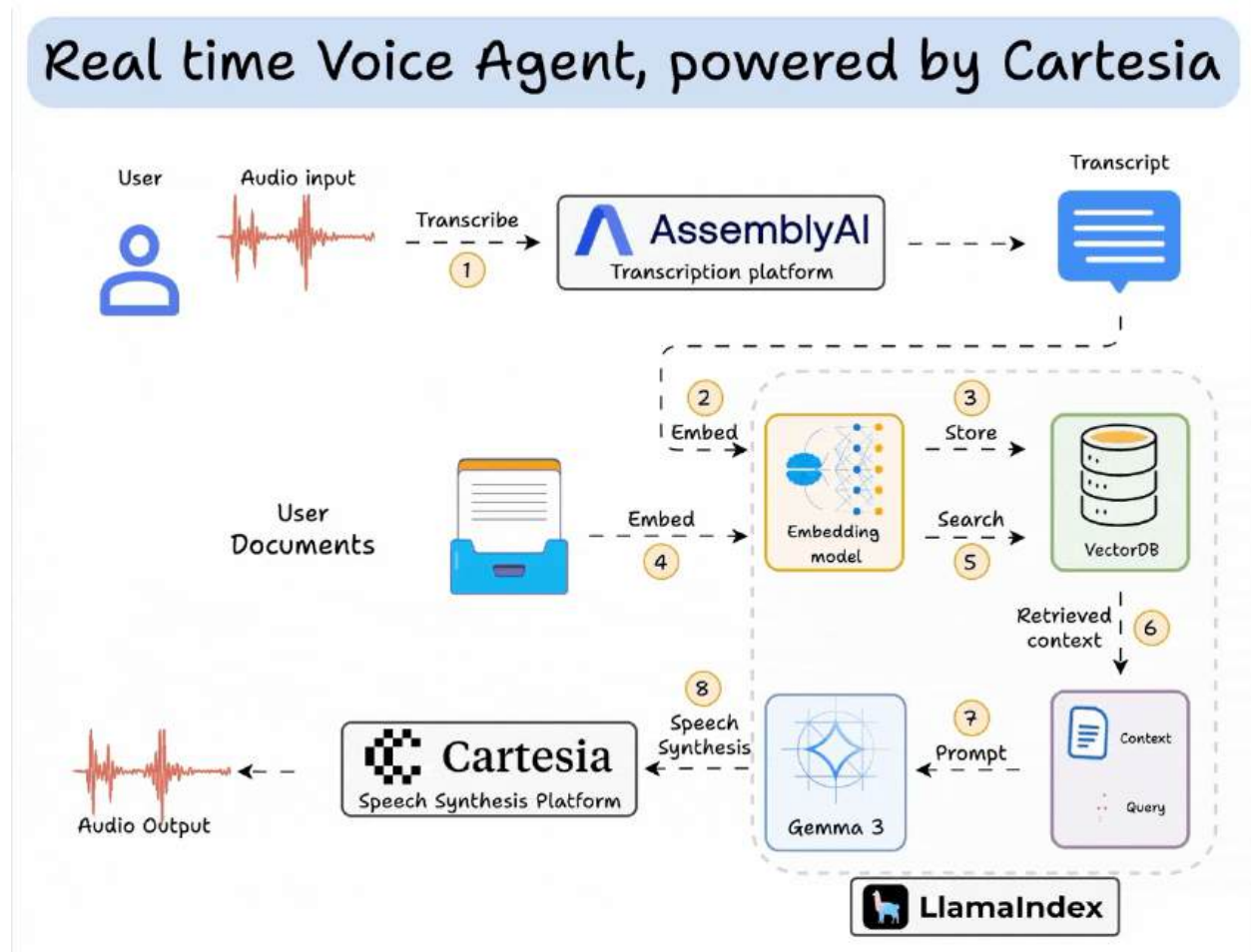
Done! We have deployed our fully private Qwen 3 Agentic RAG using LitServe.



The code is available here:
<https://www.dailydoseofds.com/p/deploy-a-qwen-3-agentic-rag/>

#2) Voice RAG Agent

Real-time voice interactions are becoming more and more popular in AI apps. Learn how to build a real-time Voice RAG Agent, step-by-step.



Tech stack:

- CartesiaAI for SOTA text-to-speech
- AssemblyAI for speech-to-text
- LlamaIndex to power RAG
- Livekit for orchestration

Workflow:

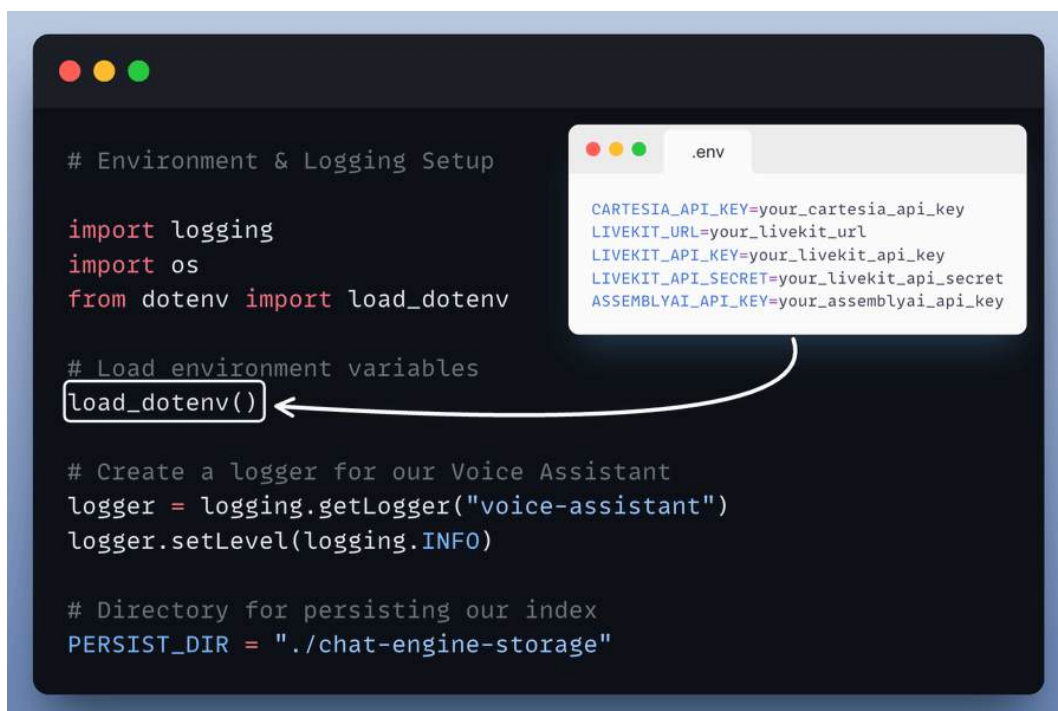
- Listens to real-time audio
- Transcribes it via AssemblyAI
- Uses your docs (via LlamaIndex) to craft an answer
- Speaks that answer back with Cartesia

Let's implement this!

#1) Set up environment and logging

This ensures we can load configurations from .env and keep track of everything in real time.

Check this out:



```
# Environment & Logging Setup

import logging
import os
from dotenv import load_dotenv

# Load environment variables
load_dotenv()

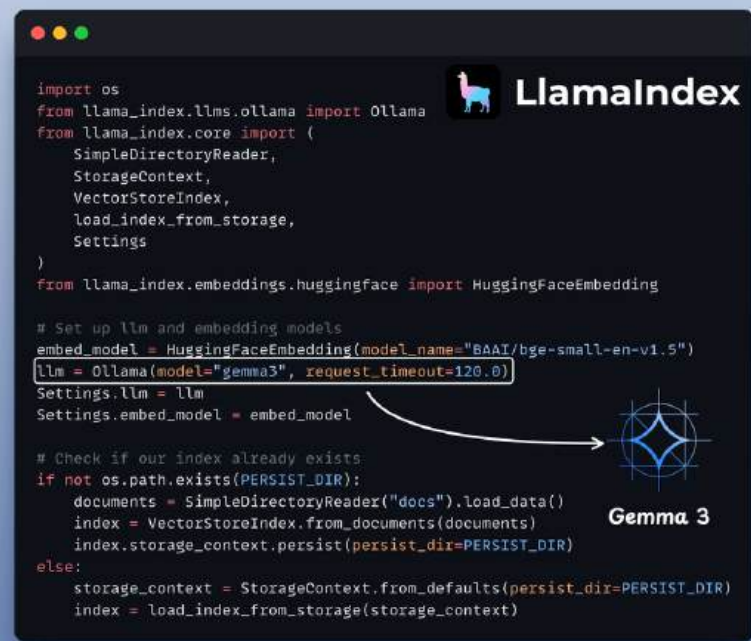
# Create a logger for our Voice Assistant
logger = logging.getLogger("voice-assistant")
logger.setLevel(logging.INFO)

# Directory for persisting our index
PERSIST_DIR = "./chat-engine-storage"
```

```
.env
CARTESIA_API_KEY=your_cartesia_api_key
LIVEKIT_URL=your_livekit_url
LIVEKIT_API_KEY=your_livekit_api_key
LIVEKIT_API_SECRET=your_livekit_api_secret
ASSEMBLYAI_API_KEY=your_assemblyai_api_key
```

#2) Setup RAG

This is where your documents get indexed for search and retrieval, powered by LlamaIndex. The agent's answers would be grounded to this knowledge base.



```
import os
from llama_index.llms.ollama import Ollama
from llama_index.core import (
    SimpleDirectoryReader,
    StorageContext,
    VectorStoreIndex,
    load_index_from_storage,
    Settings
)
from llama_index.embeddings.huggingface import HuggingFaceEmbedding

# Set up llm and embedding models
embed_model = HuggingFaceEmbedding(model_name="BAAI/bge-small-en-v1.5")
llm = Ollama(model="gemma3", request_timeout=120.0)
Settings.llm = llm
Settings.embed_model = embed_model

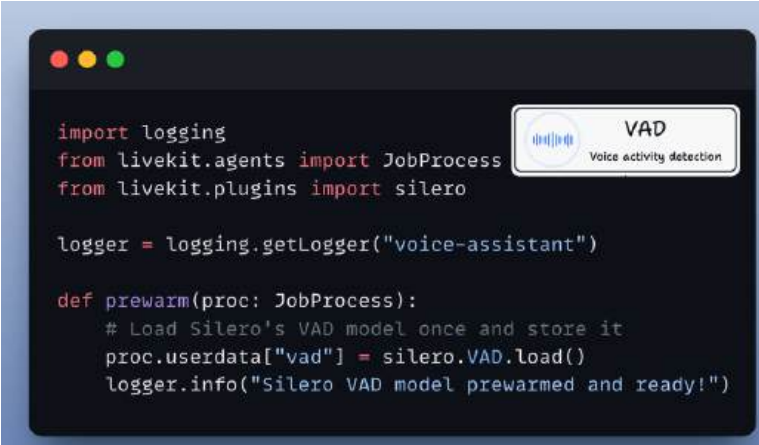
# Check if our index already exists
if not os.path.exists(PERSIST_DIR):
    documents = SimpleDirectoryReader("docs").load_data()
    index = VectorStoreIndex.from_documents(documents)
    index.storage_context.persist(persist_dir=PERSIST_DIR)
else:
    storage_context = StorageContext.from_defaults(persist_dir=PERSIST_DIR)
    index = load_index_from_storage(storage_context)
```

LlamaIndex

Gemma 3

#3) Setup Voice Activity Detection

We also want Voice Activity Detection (VAD) for smooth real-time experience—so we'll "prewarm" the Silero VAD model. This helps us detect when someone is actually speaking. Check this out:



```
import logging
from livekit.agents import JobProcess
from livekit.plugins import silero

logger = logging.getLogger("voice-assistant")

def prewarm(proc: JobProcess):
    # Load Silero's VAD model once and store it
    proc.userdata["vad"] = silero.VAD.load()
    logger.info("Silero VAD model prewarmed and ready!")
```

VAD
Voice activity detection

#4) The VoicePipelineAgent and Entry Point

This is where we bring it all together. The agent:

1. Listens to real-time audio.
2. Transcribes it using AssemblyAI.
3. Craft an answer with your documents via LlamaIndex.
4. Speaks that answer back using Cartesia.

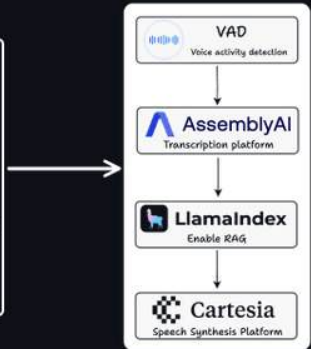
Check this out:

```
import logging
from livekit.agents import JobContext
from livekit.agents.pipeline import VoicePipelineAgent

async def entrypoint(ctx: JobContext):
    # Define system role for short, witty replies
    chat_context = ChatContext().append(
        role="system",
        text=(
            "You are a funny, witty assistant, who responds with short, concise answers."
        ),
    )
    # Convert index to a chat engine
    chat_engine = index.as_chat_engine(chat_mode=ChatMode.CONTEXT)
    await ctx.connect(auto_subscribe=AutoSubscribe.AUDIO_ONLY)

    participant = await ctx.wait_for_participant()
    logger.info(f"Participant {participant.identity} joined! Starting voice assistant.")

    # Build the voice pipeline: VAD → STT → RAG → TTS
    agent = VoicePipelineAgent(
        vad=ctx.proc.userdata["vad"],
        stt=assemblyai.STT(),
        llm=llama_index.LLM(chat_engine=chat_engine),
        tts=cartesia.TTS(
            model="sonic-2",
            voice="794f9389-aac1-45b6-b726-9d9369183238",
        ),
        chat_ctx=chat_context,
    )
    agent.start(ctx.room, participant)
    # Greet the user with a short message
    await agent.say("Hey there! How can I help you today?", allow_interruptions=True)
```



#5) Run the app

Finally, we tie it all together. We run our agent with, specifying the prewarm function and main entrypoint.



That's it—your Real-Time Voice RAG Agent is ready to roll!



The code is available here:
<https://www.dailydoseofds.com/p/building-a-real-time-voice-rag-agent/>

#3) Multi-agent Flight finder

Build a flight search pipeline with agentic capabilities that can parse natural language queries and fetch live results from Kayak.



Tech stack:

- CrewAIInc for multi-agent orchestration
- BrowserbaseHQ's headless browser tool
- Ollama to locally serve DeepSeek-R1

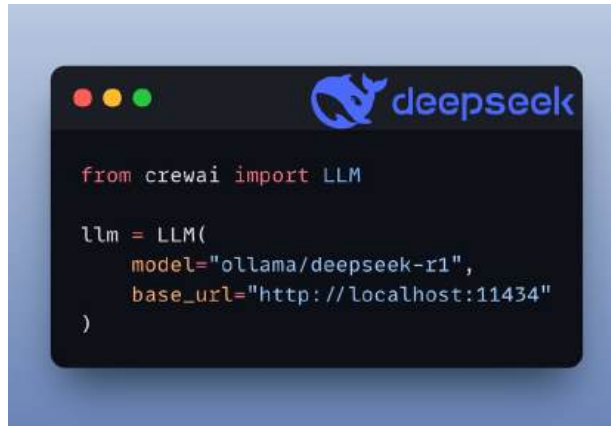
Workflow:

- Parse the query (SF to New York on 21st September) to create a Kayak search URL
- Visit the URL and extract top 5 flights
- For each flight, go to the details to find available airlines
- Summarize flight info

Let's implement this!

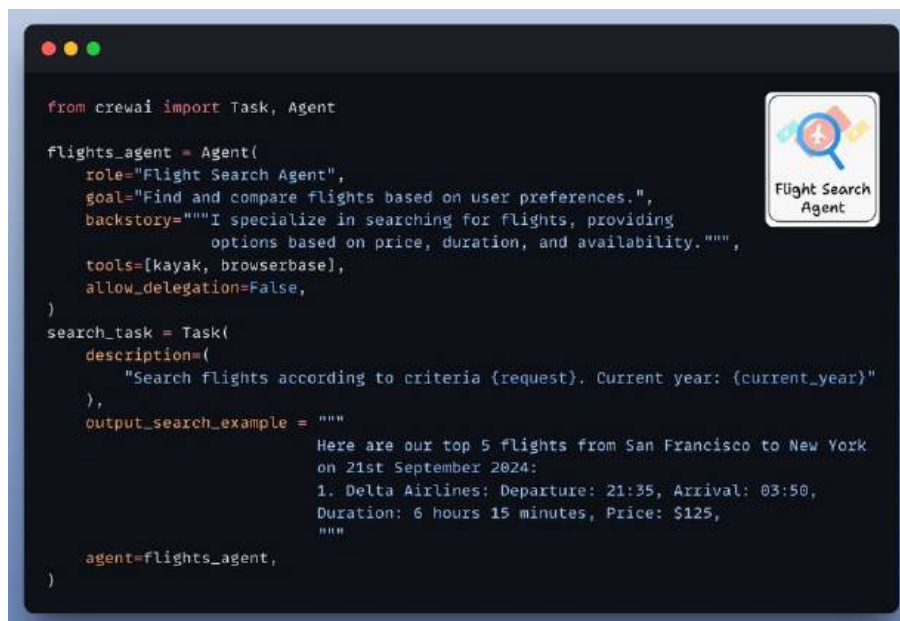
#1) Define LLM

CrewAI nicely integrates with all the popular LLMs and providers out there. Here's how you set up a local DeepSeek using Ollama:



#2) Flight Search Agent

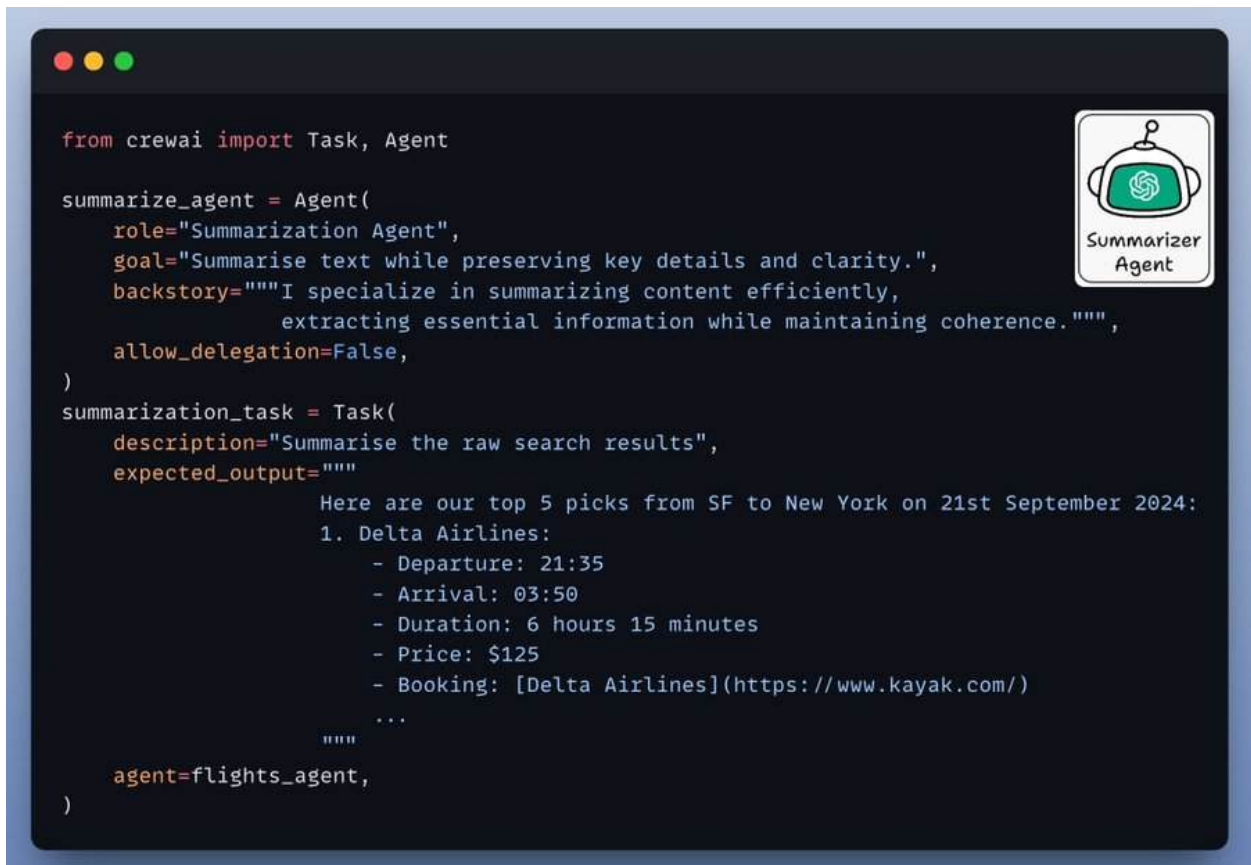
This agent mimics a real human searching for flights by browsing the web, powered by Browserbase's headless-browser tool and can look up flights on sites like Kayak.



#3) Summarisation Agent

After retrieving the flight details, we need a concise summary of all available options.

This is where our Summarization Agent steps in to make sense of the results for easy reading.



```
from crewai import Task, Agent

summarize_agent = Agent(
    role="Summarization Agent",
    goal="Summarise text while preserving key details and clarity.",
    backstory="""I specialize in summarizing content efficiently,
                extracting essential information while maintaining coherence.""",
    allow_delegation=False,
)

summarization_task = Task(
    description="Summarise the raw search results",
    expected_output="""
        Here are our top 5 picks from SF to New York on 21st September 2024:
        1. Delta Airlines:
            - Departure: 21:35
            - Arrival: 03:50
            - Duration: 6 hours 15 minutes
            - Price: $125
            - Booking: [Delta Airlines](https://www.kayak.com/)
            ...
        """,
    agent=flights_agent,
)
```



Now that we have both our agents ready, it's time to understand the tools powering them.

1. Kayak tool
2. Browserbase tool

Let's write their code one-by-one.

Tools



#4) Kayak tool

A custom Kayak tool to translate the user input into a valid Kayak search URL.

(FYI Kayak is a popular flight and hotel booking)

```
from crewai.tools import tool
from typing import Optional

@tool("Kayak tool")
def kayak_search(
    departure: str, destination: str, date: str, return_date: Optional[str] = None
) -> str:
    """
    Generates a Kayak URL for flights between departure and destination on the specified date.

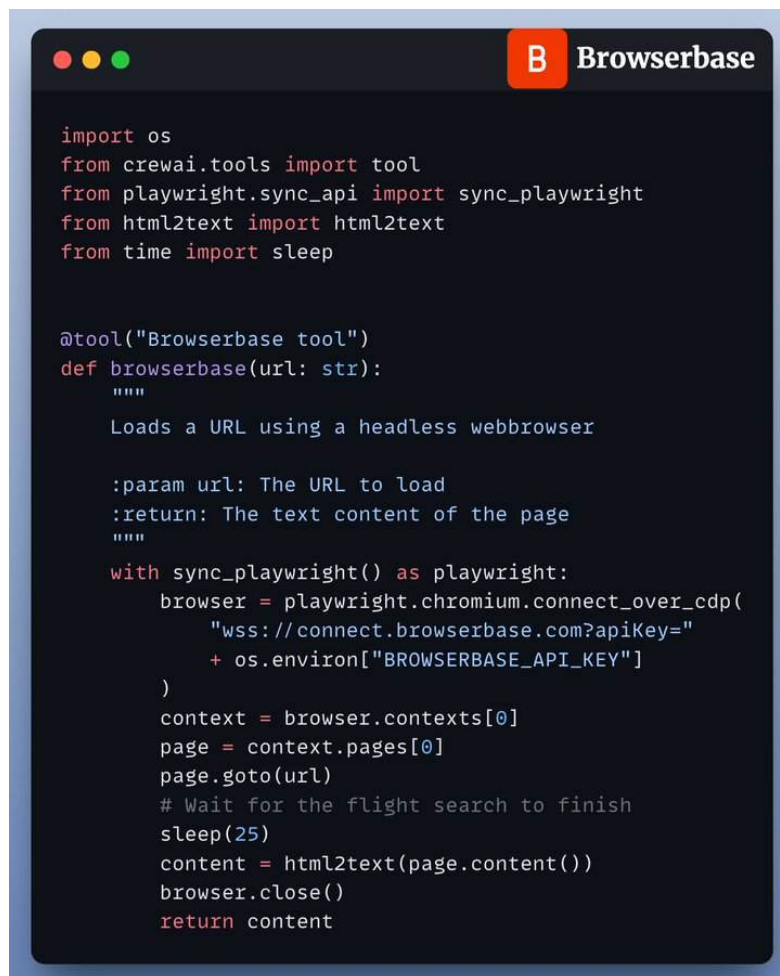
    :param departure: The IATA code for the departure airport (e.g., 'SOF' for Sofia)
    :param destination: The IATA code for the destination airport (e.g., 'BER' for Berlin)
    :param date: The date of the flight in the format 'YYYY-MM-DD'
    :return_date: Only for two-way tickets. The date of return flight in the format 'YYYY-MM-DD'
    :return: The Kayak URL for the flight search
    """
    print(f"Generating Kayak URL for {departure} to {destination} on {date}")
    URL = f"https://www.kayak.com/flights/{departure}-{destination}/{date}"
    if return_date:
        URL += f"/{return_date}"
    URL += "?currency=USD"
    return URL

# Export the decorated function
kayak = kayak_search
```

#5) Browserbase Tool

The flight search agent uses the Browserbase tool to simulate human browsing and gather flight data.

To be precise it automatically navigates the Kayak website and interacts with the web page. Check this out:

A screenshot of a code editor window with a dark background. The window has a title bar with three colored circles (red, yellow, green) on the left and a tab labeled 'B Browserbase' on the right. The code is written in Python and defines a tool named 'Browserbase tool'. It imports 'os', 'tool' from 'crewai.tools', 'sync_playwright' from 'playwright.sync_api', 'html2text' from 'html2text', and 'sleep' from 'time'. The function 'browserbase(url: str)' is decorated with '@tool("Browserbase tool")'. It has a docstring describing its purpose and parameters. The function body uses 'sync_playwright()' to create a playwright object, connects to Browserbase via CDP, navigates to the specified URL, waits for 25 seconds, and returns the HTML content of the page.

```
import os
from crewai.tools import tool
from playwright.sync_api import sync_playwright
from html2text import html2text
from time import sleep

@tool("Browserbase tool")
def browserbase(url: str):
    """
    Loads a URL using a headless webbrowser

    :param url: The URL to load
    :return: The text content of the page
    """
    with sync_playwright() as playwright:
        browser = playwright.chromium.connect_over_cdp(
            "wss://connect.browserbase.com?apiKey="
            + os.environ["BROWSERBASE_API_KEY"]
        )
        context = browser.contexts[0]
        page = context.pages[0]
        page.goto(url)
        # Wait for the flight search to finish
        sleep(25)
        content = html2text(page.content())
        browser.close()
        return content
```

#6) Setup Crew

Once the agents and tools are defined, we orchestrate them using CrewAI.

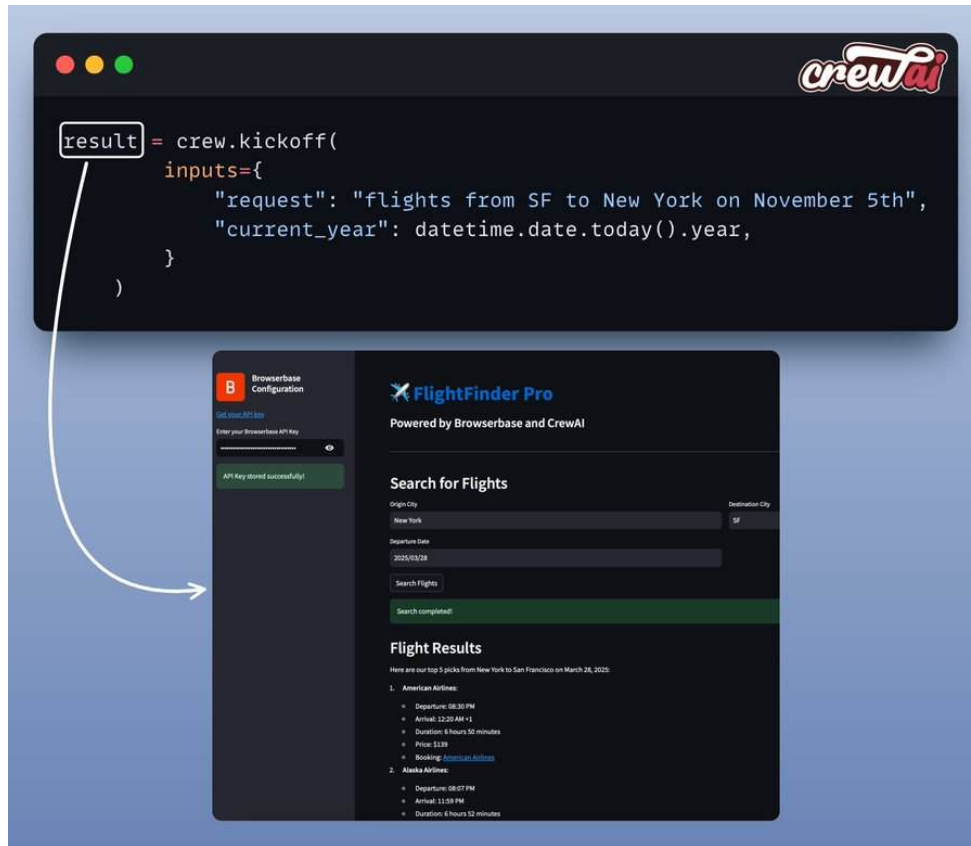
Define their tasks, sequence their actions, and watch them collaborate in real time. Check this out:



Note: here the 'planning = True' is using the planning pattern we discussed in the 5 Agentic AI Design Patterns above.

#7) Kickoff and results

Finally, we feed the user's request (departure city, arrival city, travel dates) into the Crew and let it run:



Streamlit UI

To make this accessible, we wrapped the entire system in a Streamlit interface.

It's a simple chat-like UI where you enter your flight details and see the results in real time.

Check this out:

**Browserbase Configuration**

[Get your API key](#)

Enter your Browserbase API Key

.....



API Key stored successfully!

**FlightFinder Pro**

Powered by Browserbase and CrewAI

Search for Flights

Origin City

New York

Destination City

SF

Departure Date

2025/03/28

Search Flights

Search completed!

Flight Results

Here are our top 5 picks from New York to San Francisco on March 28, 2025:

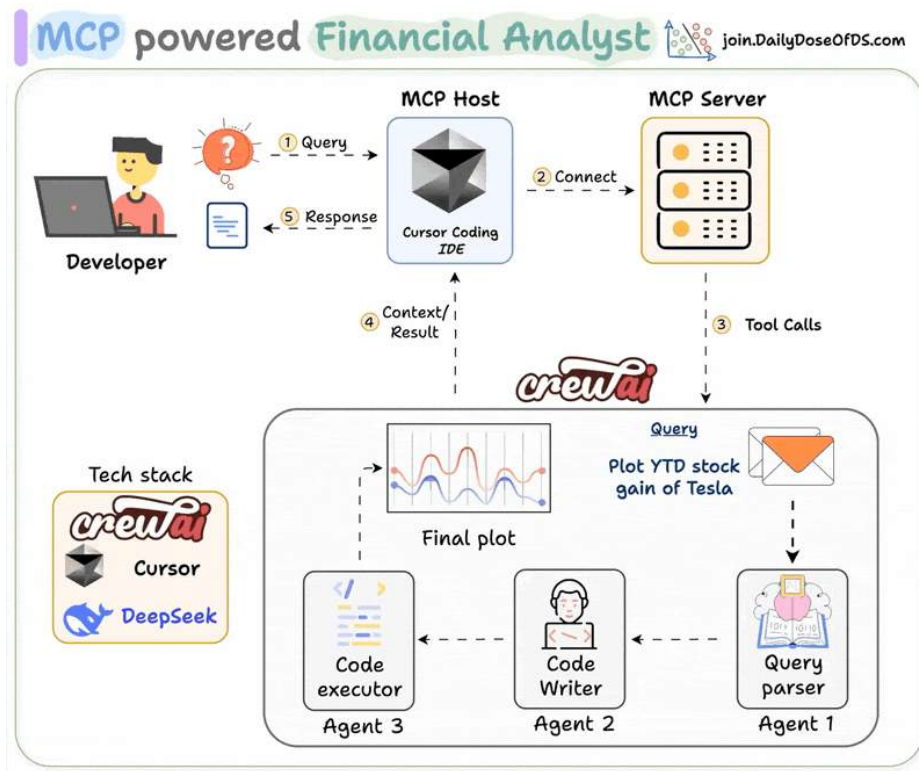
- American Airlines:



The code is available here:
<https://blog.dailydoseofds.com/p/hands-on-a-multi-agent-flight-finder/>

#4) Financial Analyst

Build an AI agent that fetches, analyzes & generates insights on stock market trends, right from Cursor or Claude Desktop.



Tech stack:

- CrewAI for multi-agent orchestration
- Ollama to locally serve DeepSeek-R1 LLM
- Cursor as the MCP host

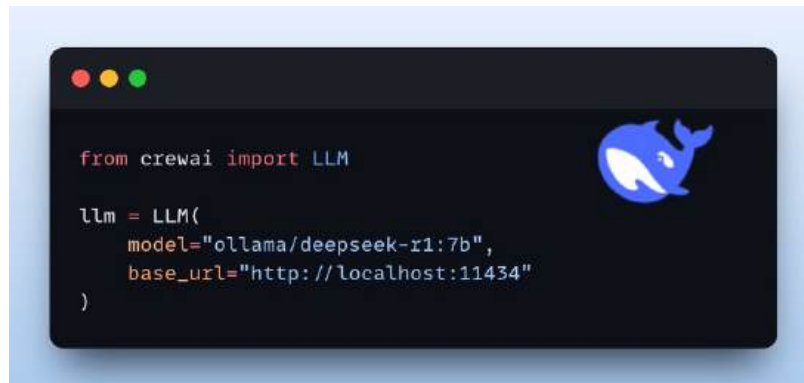
Workflow:

- The user submits a query.
- The MCP agent kicks off the financial analyst crew.
- The crew conducts research and creates an executable script.
- The agent runs the script to generate an analysis plot.

Let's implement this!

#1) Setup LLM

We will use Deepseek-R1 as the LLM, served locally using Ollama.



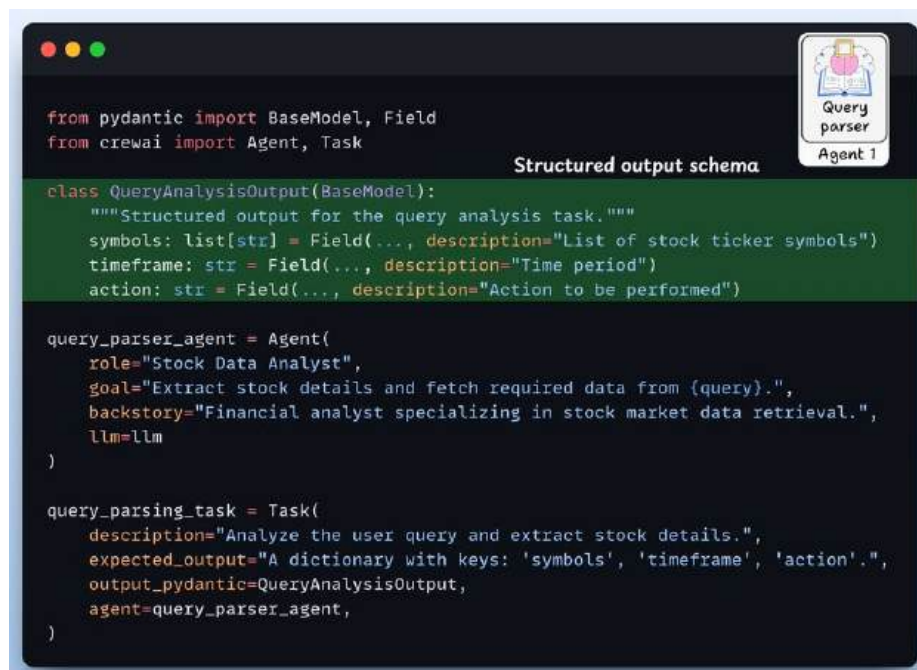
```
from crewai import LLM

llm = LLM(
    model="ollama/deepseek-r1:7b",
    base_url="http://localhost:11434"
)
```

Let's setup the Crew now

#2) Query Parser Agent

This agent accepts a natural language query and extracts structured output using Pydantic. This guarantees clean and structured inputs for further processing!



```
from pydantic import BaseModel, Field
from crewai import Agent, Task

class QueryAnalysisOutput(BaseModel):
    """Structured output for the query analysis task."""
    symbols: list[str] = Field(..., description="List of stock ticker symbols")
    timeframe: str = Field(..., description="Time period")
    action: str = Field(..., description="Action to be performed")

query_parser_agent = Agent(
    role="Stock Data Analyst",
    goal="Extract stock details and fetch required data from {query}.",
    backstory="Financial analyst specializing in stock market data retrieval.",
    llm=llm
)

query_parsing_task = Task(
    description="Analyze the user query and extract stock details.",
    expected_output="A dictionary with keys: 'symbols', 'timeframe', 'action'.",
    output_pydantic=QueryAnalysisOutput,
    agent=query_parser_agent,
)
```


#3) Code Writer Agent

This agent writes Python code to visualize stock data using Pandas, Matplotlib, and Yahoo Finance libraries.



```
from crewai import Agent, Task

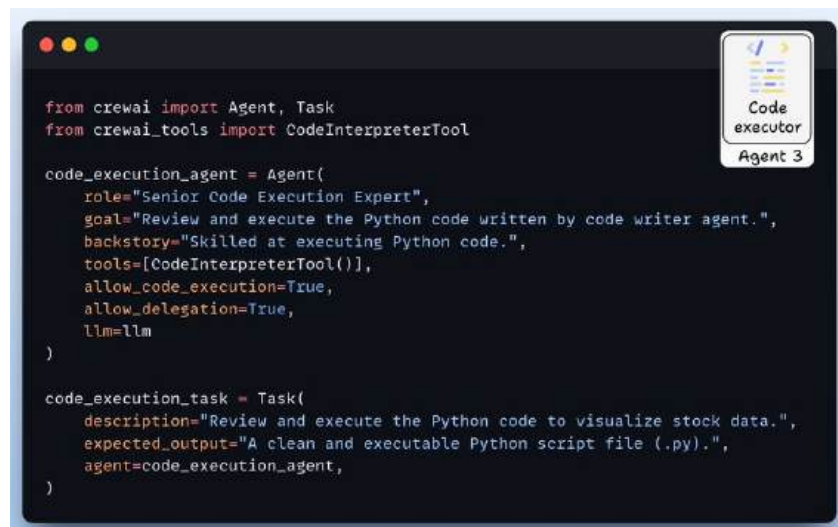
code_writer_agent = Agent(
    role="Senior Python Developer",
    goal="Write Python code to visualize stock data.",
    backstory="""Senior Python developer specialized in stock market data
        visualization and writing production-ready Python code.""",
    llm=llm
)

code_writer_task = Task(
    description="Write production-ready Python code to visualize stock data.",
    expected_output="An executable Python script for stock visualization.",
    agent=code_writer_agent,
)
```

#4) Code Executor Agent

This agent reviews and executes the generated Python code for stock data visualization.

It uses the code interpreter tool by CrewAI to execute the code in a secure sandbox environment.



```
from crewai import Agent, Task
from crewai_tools import CodeInterpreterTool

code_execution_agent = Agent(
    role="Senior Code Execution Expert",
    goal="Review and execute the Python code written by code writer agent.",
    backstory="Skilled at executing Python code.",
    tools=[CodeInterpreterTool()],
    allow_code_execution=True,
    allow_delegation=True,
    llm=llm
)

code_execution_task = Task(
    description="Review and execute the Python code to visualize stock data.",
    expected_output="A clean and executable Python script file (.py).",
    agent=code_execution_agent,
)
```

#5) Setup Crew and Kickoff

We set up and kick off our financial analysis crew to get the result shown below!



#6) Create MCP Server

Now, we encapsulate our financial analyst within an MCP tool and add two more tools to enhance the user experience.

- save_code -> Saves generated code to local directory
- run_code_and_show_plot -> Executes the code and generates a plot

```
server.py

from mcp.server.fastmcp import FastMCP
from finance_crew import run_financial_analysis

mcp = FastMCP("financial-analyst")

@mcp.tool()
def analyze_stock(query: str) -> str:
    """Analyzes stock market data based on query and generates
    executable Python code for analysis and visualization"""
    result = run_financial_analysis(query)
    return result

@mcp.tool()
def save_code(code: str) -> str:
    """Save the given code to a file stock_analysis.py"""
    with open('stock_analysis.py', 'w') as f:
        f.write(code)
    return "Code saved to stock_analysis.py"

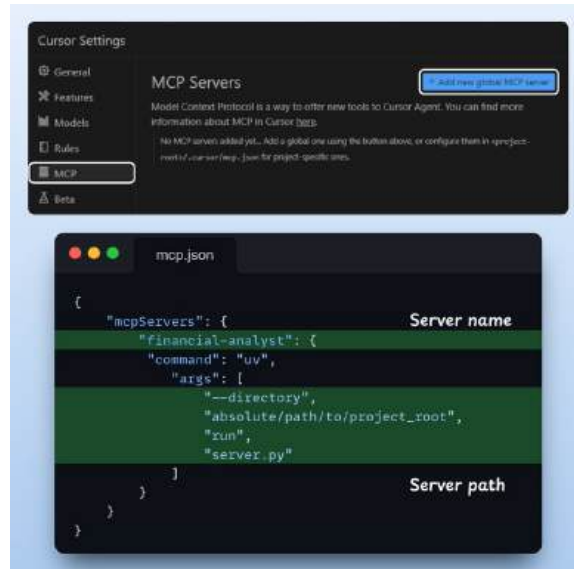
@mcp.tool()
def run_code_and_show_plot() -> str:
    """Execute code and generate a plot"""
    with open('stock_analysis.py', 'r') as f:
        exec(f.read())

if __name__ == "__main__":
    mcp.run(transport='stdio')
```

Model Context Protocol

#7) Integrate MCP server with Cursor

Go to: File → Preferences → Cursor Settings → MCP → Add new global MCP server. In the JSON file, add what's shown below



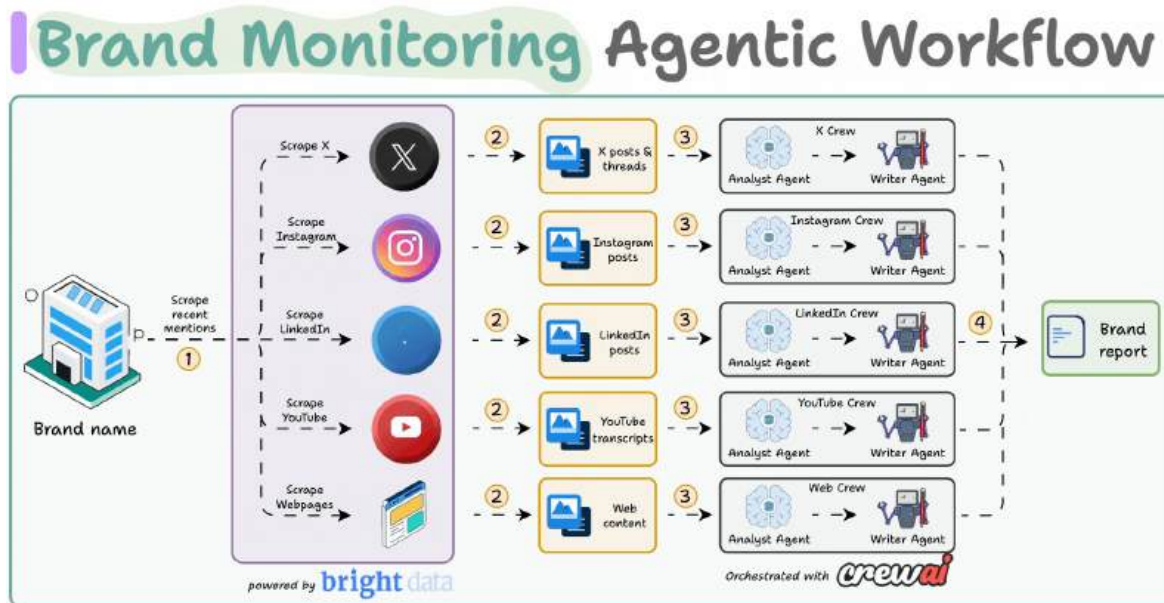
Done! Our financial analyst MCP server is live and connected to Cursor.



The code is available here:
<https://www.dailydoseofds.com/p/hands-on-building-an-mcp-powered-financial-analyst/>

#5) Brand Monitoring System

Build a multi agent brand monitoring app that scraps web mentions and produces insights about a company.



Tech stack:

- Bright Data to scrape data at scale
- CrewAI for orchestration
- Ollama to serve DeepSeek locally

Workflow:

- Use Bright Data to scrape brand mentions across X, Instagram, YouTube, websites, etc.
- Invoke platform-specific Crews to analyze the data and generate insights.
- Merge all insights to get the final report.

Let's implement this!

#1) Scraping tool

To monitor a brand, we must scrape data across various sources—X, YouTube, Instagram, websites, etc.

Thus, we'll first gather recent search results from Bright Data's SERP API.

See this code:

A screenshot of a code editor window titled 'book_writer.py'. The editor has a dark background with light blue and green accents. The code is written in Python and defines a tool for scraping search results from Bright Data's SERP API. The code includes imports for BaseTool, BaseModel, Field, and requests. It defines a BrightDataWebSearchToolInput class with a title field and a BrightDataWebSearchTool class that inherits from BaseTool. The tool class has attributes for name, description, and args_schema. It also has a _run method that sets up a proxy, sends a GET request to the SERP API, and returns the organic search results.

```
from crewai.tools import BaseTool
from pydantic import BaseModel, Field
import requests

class BrightDataWebSearchToolInput(BaseModel):
    """Input schema for BrightDataWebSearchTool."""
    title: str = Field(..., description="Topic of the book to write about.")

class BrightDataWebSearchTool(BaseTool):
    name: str = "Web Search Tool"
    description: str = "Tool to search Google and retrieve the results."
    args_schema: Type[BaseModel] = BrightDataWebSearchToolInput

    def _run(self, title):

        host = 'brd.superproxy.io'
        port = 33335

        username = '<Get from brightdata.com in SERP API>'
        password = '<Get from brightdata.com in SERP API>'

        proxies = {
            'http': f'http://{username}:{password}@{host}:{port}',
            'https': f'http://{username}:{password}@{host}:{port}'
        }

        url = f"https://www.google.com/search?q={title}&brd_json=1&num=500"
        response = requests.get(url, proxies=proxies, verify=False)

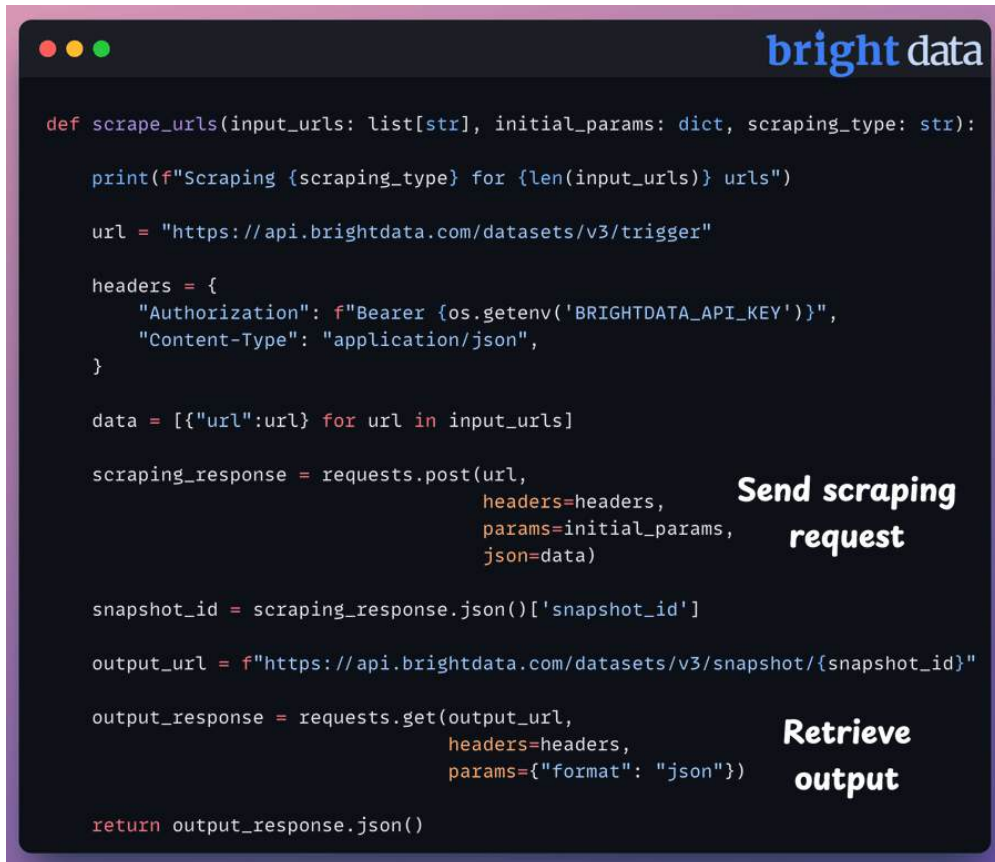
        return response.json()['organic']
```

#2) Platform-specific scraping function

The above output will contain links to web pages, X posts, YouTube videos, Instagram posts, etc.

To scrape those sources, we use Bright Data's platform-specific scrapers.

Check this code:



```
def scrape_urls(input_urls: list[str], initial_params: dict, scraping_type: str):  
    print(f"Scraping {scraping_type} for {len(input_urls)} urls")  
  
    url = "https://api.brightdata.com/datasets/v3/trigger"  
  
    headers = {  
        "Authorization": f"Bearer {os.getenv('BRIGHTDATA_API_KEY')}",  
        "Content-Type": "application/json",  
    }  
  
    data = [{"url":url} for url in input_urls]  
  
    scraping_response = requests.post(url,  
                                     headers=headers,  
                                     params=initial_params,  
                                     json=data) Send scraping request  
  
    snapshot_id = scraping_response.json()['snapshot_id']  
  
    output_url = f"https://api.brightdata.com/datasets/v3/snapshot/{snapshot_id}"  
  
    output_response = requests.get(output_url,  
                                   headers=headers,  
                                   params={"format": "json"}) Retrieve output  
  
    return output_response.json()
```

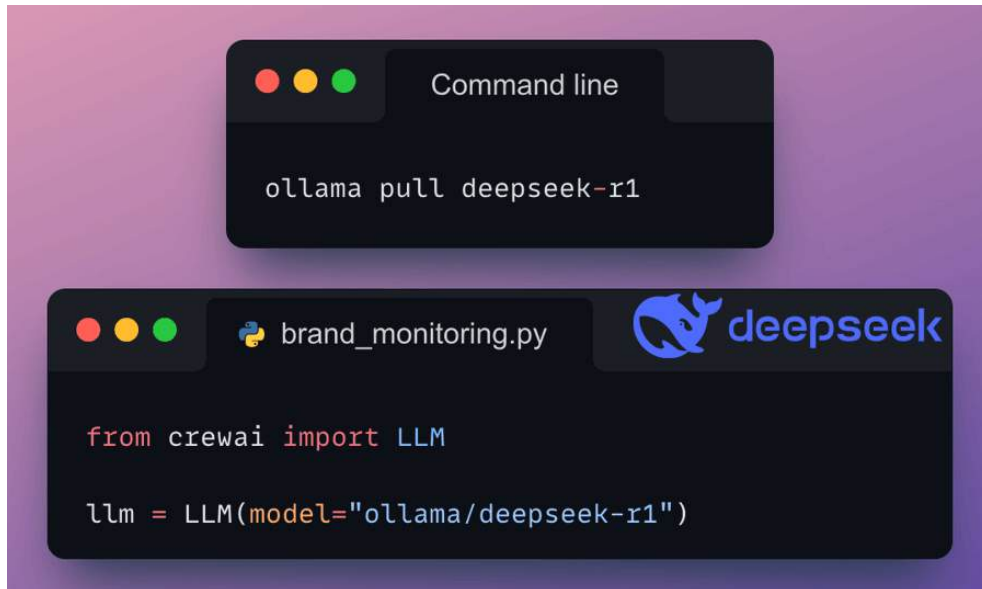
#3) Set up DeepSeek R1 locally

We'll serve R1 locally through Ollama.

To do this:

- First, we download it locally.
- Next, we define it with the CrewAI's LLM class.

Here's the code:



```
ollama pull deepseek-r1
```

```
from crewai import LLM

llm = LLM(model="ollama/deepseek-r1")
```

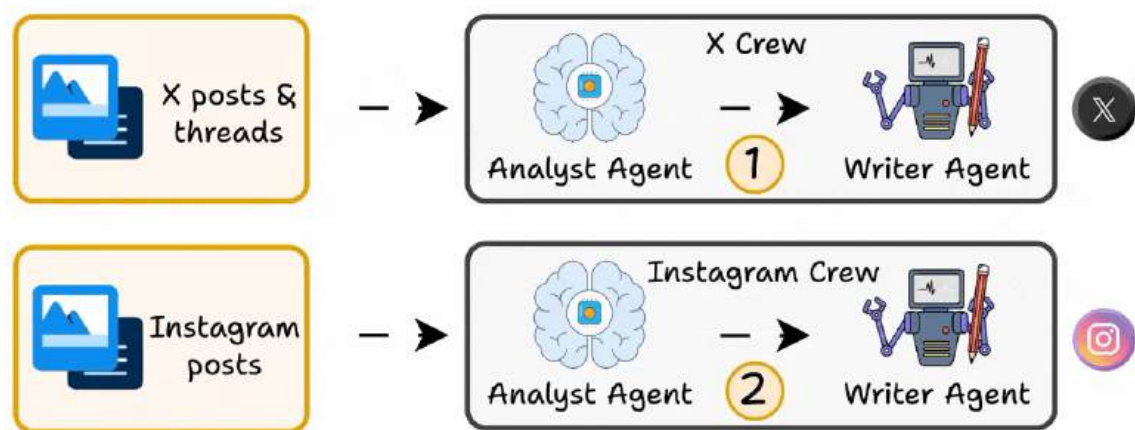
#4) Crew Setup

We will have multiple Crews, one for each platform (X, Instagram, YouTube, etc.)

Each Crew will have two Agents:

- Analysis Agent → It analyses the scraped content.
- Writer Agent → It produces insights from the analysis.

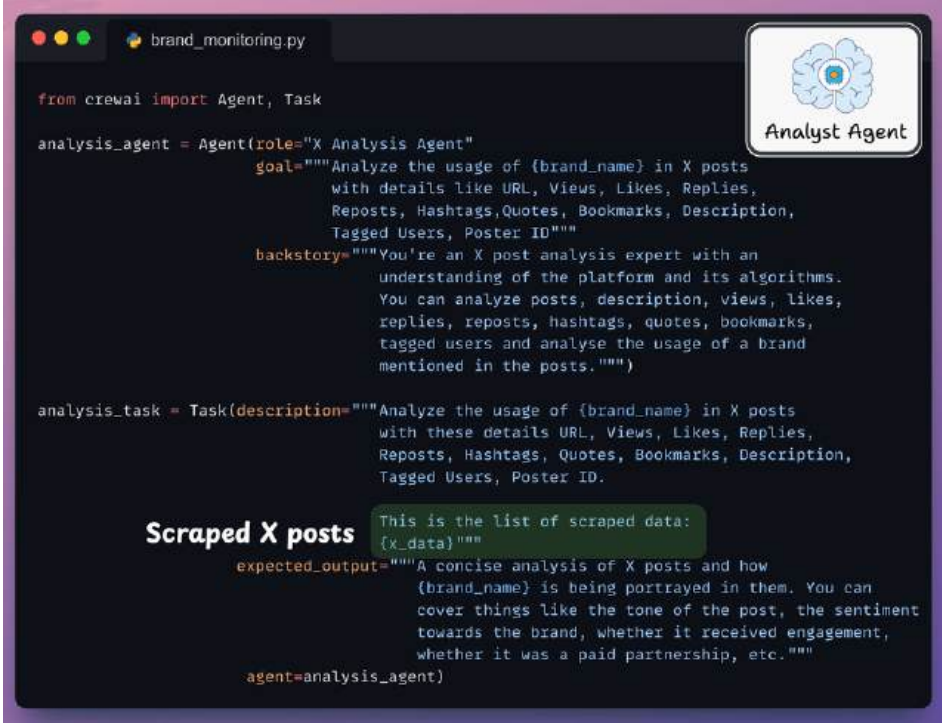
Below, let's implement the X Crew!



#5) X Analyst Agent

This Agent analyzes the posts scraped by Bright Data and extracts key insights. It is also assigned a task to do so.

Here's how it's done:



```
from crewai import Agent, Task

analysis_agent = Agent(role="X Analysis Agent"
    goal="""Analyze the usage of {brand_name} in X posts
    with details like URL, Views, Likes, Replies,
    Reposts, Hashtags, Quotes, Bookmarks, Description,
    Tagged Users, Poster ID"""
    backstory="""You're an X post analysis expert with an
    understanding of the platform and its algorithms.
    You can analyze posts, description, views, likes,
    replies, reposts, hashtags, quotes, bookmarks,
    tagged users and analyse the usage of a brand
    mentioned in the posts.""")

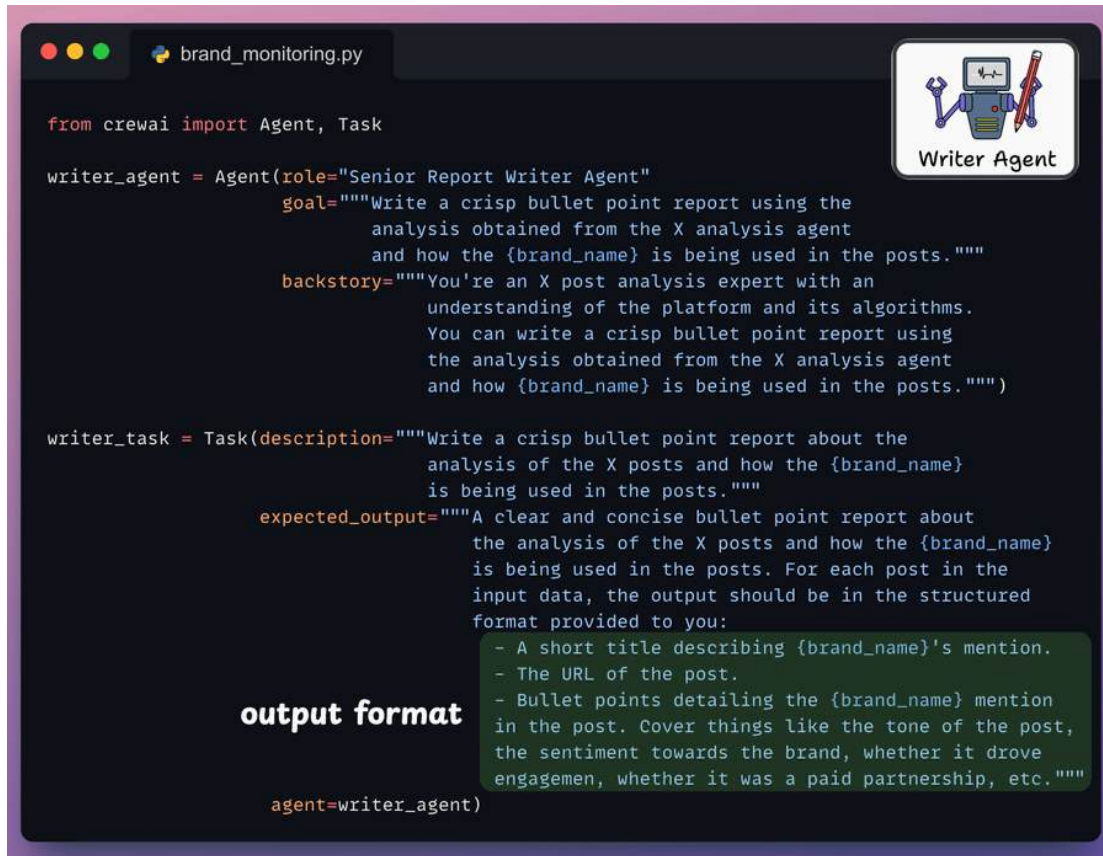
analysis_task = Task(description="""Analyze the usage of {brand_name} in X posts
    with these details URL, Views, Likes, Replies,
    Reposts, Hashtags, Quotes, Bookmarks, Description,
    Tagged Users, Poster ID.

    Scraped X posts {x_data}""",
    expected_output="""A concise analysis of X posts and how
    {brand_name} is being portrayed in them. You can
    cover things like the tone of the post, the sentiment
    towards the brand, whether it received engagement,
    whether it was a paid partnership, etc.""",
    agent=analysis_agent)
```

#6) X Writer Agent

The Agent takes the output of the X analyst agent and generates insights.

Here's the code:



```
from crewai import Agent, Task

writer_agent = Agent(role="Senior Report Writer Agent"
                    goal="""Write a crisp bullet point report using the
                        analysis obtained from the X analysis agent
                        and how the {brand_name} is being used in the posts."""
                    backstory="""You're an X post analysis expert with an
                        understanding of the platform and its algorithms.
                        You can write a crisp bullet point report using
                        the analysis obtained from the X analysis agent
                        and how {brand_name} is being used in the posts.""")

writer_task = Task(description="""Write a crisp bullet point report about the
                        analysis of the X posts and how the {brand_name}
                        is being used in the posts.""",
                    expected_output="""A clear and concise bullet point report about
                        the analysis of the X posts and how the {brand_name}
                        is being used in the posts. For each post in the
                        input data, the output should be in the structured
                        format provided to you:
                        - A short title describing {brand_name}'s mention.
                        - The URL of the post.
                        - Bullet points detailing the {brand_name} mention
                        in the post. Cover things like the tone of the post,
                        the sentiment towards the brand, whether it drove
                        engagement, whether it was a paid partnership, etc.""",
                    agent=writer_agent)
```

output format

#7) Create a Flow

Finally, we use CrewAI Flows to orchestrate the workflow:

- We start the Flow by using the Scraping tool.
- Next, we invoke platform-specific scrapers.
- Finally, we invoke platform-specific Crews.

Check this out:

```
brand_monitoring.py

from crewai.flow import Flow, listen, start
from pydantic import BaseModel

class BrandMonitoringFlow(Flow[BrandMonitoringState]):

    @start()
    def scrape_data(self):
        search_tool = BrightDataWebSearchTool() Scrape URLs of brand mentions
        self.state.search_result = search_tool(self.state.brand_name)

    @listen(scrape_data)
    def scrape_platform_data_and_analyse(self):

        # X scraping
        X_posts = scrape_urls(self.state.search_response.x_urls,
                              X_params, "X")

        X_crew = XCrew().crew()
        X_reponse = X_crew.kickoff(inputs={"X_data":X_posts, Scrape X posts
                                          "brand_name": self.state.brand_name})

        # YouTube scraping
        YouTube_transcripts = scrape_urls(self.state.search_response.YouTube_urls,
                                           YouTube_params, "X") Scrape YouTube videos

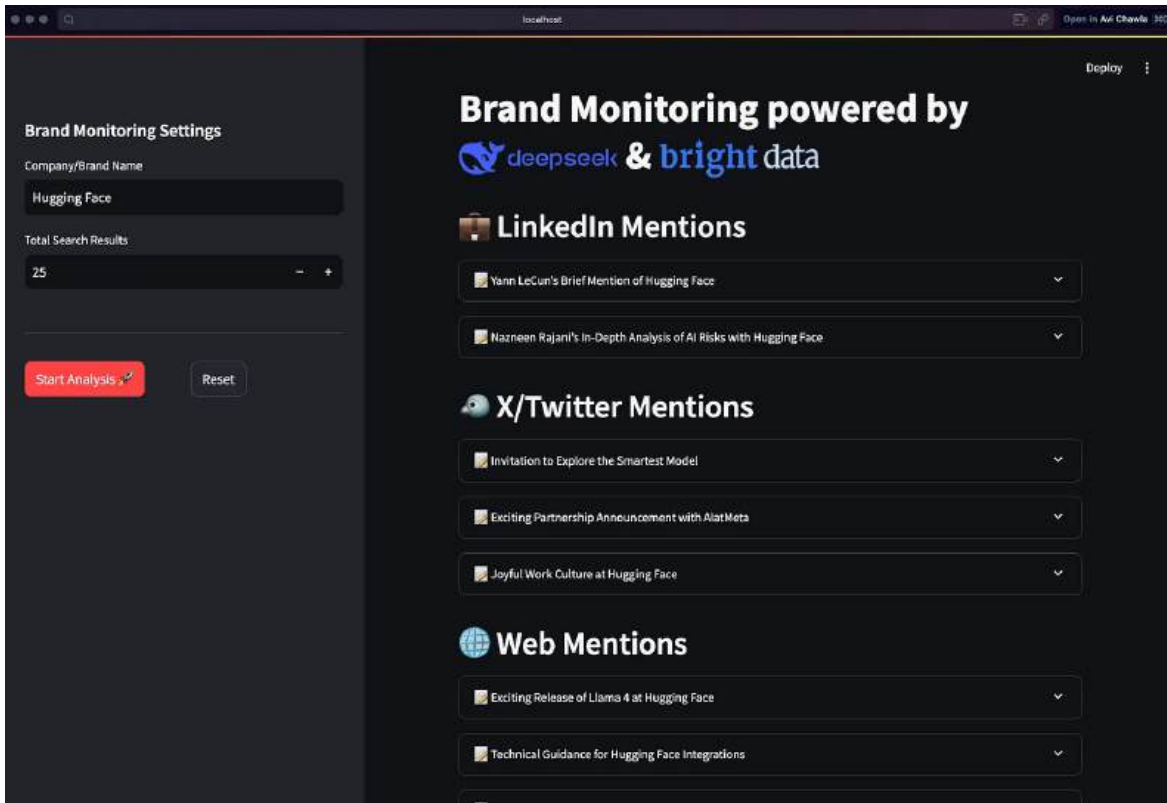
        YouTube_crew = YouTubeCrew().crew()
        YouTube_reponse = YouTube_crew.kickoff(inputs={"X_data":YouTube_posts,
                                                       "brand_name": self.state.brand_name})

        # repeat for all plaforms
```

#8) Streamlit UI and Kick off the Flow

Finally, we wrap the app in a clear Streamlit interface for interactivity and run the Flow.

Check the final outcome:



Done!

You're now ready to monitor any brand, track all mentions, and generate insights about a company.



The code is available here:
<https://www.dailydoseofds.com/p/hands-on-build-a-multi-agent-brand-monitoring-system/>

#6) Multi-agent Hotel Finder

Build an Agentic workflow that parses a travel query, fetches live flights and hotel data from Kayak, and summarizes the best options.



Tech stack:

- CrewAI for multi-agent orchestration
- Browserbase's headless browser tool
- Ollama to locally serve DeepSeek-R1

Workflow:

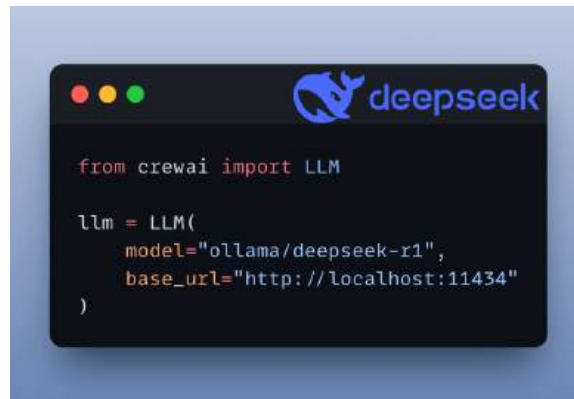
- Parse the query (location, dates etc.) to create a Kayak search URL
- Visit the URL and extract top 5 flights
- For each hotel, find pricing and more info
- Summarize hotel info

Let's implement this!

#1) Define LLM

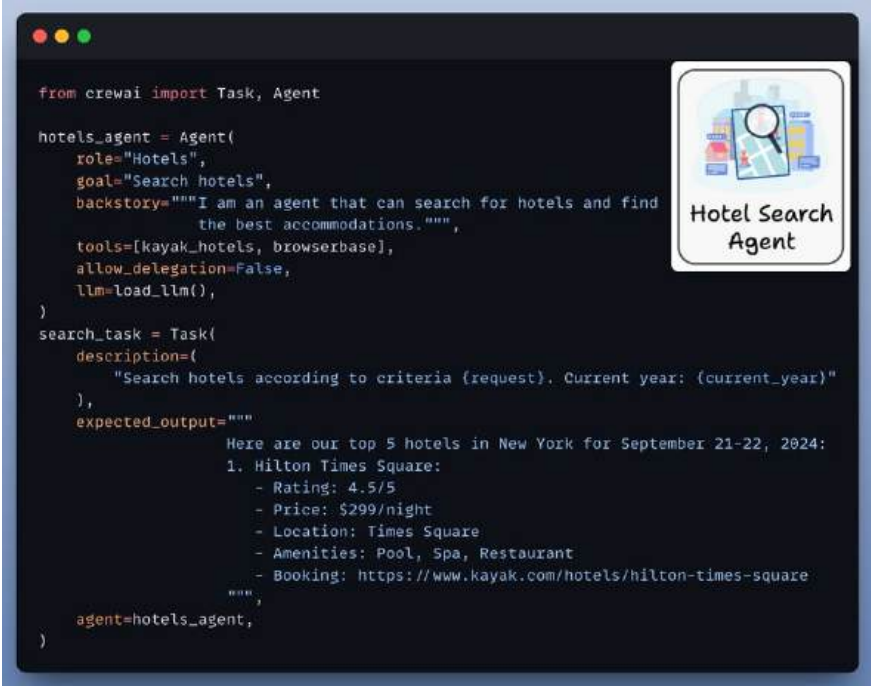
CrewAI nicely integrates with all the popular LLMs and providers out there!

Here's how you set up a local DeepSeek using Ollama:



#2) Hotel Search Agent

This agent mimics a real human searching for hotels by browsing the web. It's powered by browserbase's headless-browser tool and can look up hotels on sites like Kayak.



```
from crewai import Task, Agent

hotels_agent = Agent(
    role="Hotels",
    goal="Search hotels",
    backstory="""I am an agent that can search for hotels and find
                the best accommodations.""",
    tools=[kayak_hotels, browserbase],
    allow_delegation=False,
    llm=load_llm(),
)

search_task = Task(
    description=(
        "Search hotels according to criteria {request}. Current year: {current_year}"
    ),
    expected_output="""
        Here are our top 5 hotels in New York for September 21-22, 2024:
        1. Hilton Times Square:
            - Rating: 4.5/5
            - Price: $299/night
            - Location: Times Square
            - Amenities: Pool, Spa, Restaurant
            - Booking: https://www.kayak.com/hotels/hilton-times-square
        """
    ,
    agent=hotels_agent,
)
```

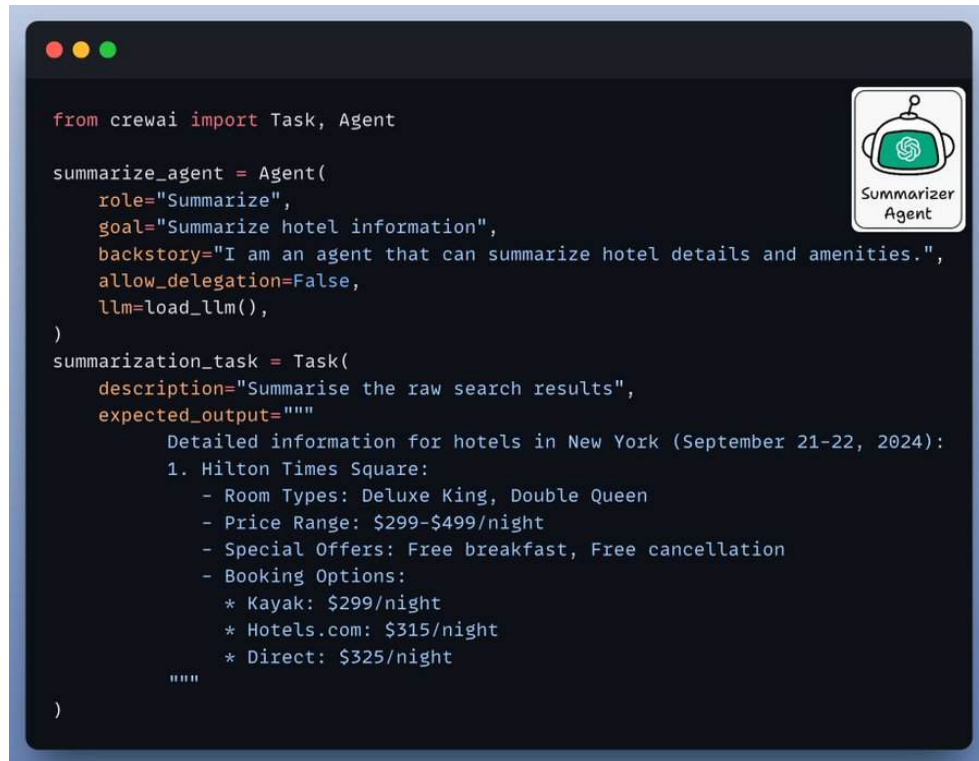


#3) Summarisation Agent

After retrieving the hotel details, we need a concise summary of all available options.

This is where our Summarization Agent steps in to make sense of the results for easy reading.

Check this out:



```
from crewai import Task, Agent

summarize_agent = Agent(
    role="Summarize",
    goal="Summarize hotel information",
    backstory="I am an agent that can summarize hotel details and amenities.",
    allow_delegation=False,
    llm=load_llm(),
)

summarization_task = Task(
    description="Summarise the raw search results",
    expected_output="""
    Detailed information for hotels in New York (September 21-22, 2024):
    1. Hilton Times Square:
        - Room Types: Deluxe King, Double Queen
        - Price Range: $299-$499/night
        - Special Offers: Free breakfast, Free cancellation
        - Booking Options:
            * Kayak: $299/night
            * Hotels.com: $315/night
            * Direct: $325/night
    """)
```

Now that we have both our agents ready, it's time to understand the tools powering them.

1. Kayak tool

2. Browserbase tool

Let's write their code one-by-one.



#4) Kayak tool

A custom Kayak tool to translate the user input into a valid Kayak search URL.

(FYI Kayak is a popular for hotel and flight booking)

```
from crewai.tools import tool
from typing import Optional

# --- New Hotel Search Function ---
@tool("Kayak Hotel Tool")
def kayak_hotel_search(
    location: str, check_in_date: str, check_out_date: str, num_adults: int = 2
) -> str:
    """
    Generates a Kayak URL for hotel searches based on location, dates, and number of adults.

    :param location_query: The location string used by Kayak (e.g., 'Hisar,Haryana,India-p15321')
    :param check_in_date: The check-in date in 'YYYY-MM-DD' format
    :param check_out_date: The check-out date in 'YYYY-MM-DD' format
    :param num_adults: The number of adults (defaults to 2)
    :return: The Kayak URL for the hotel search
    """
    URL = f"https://www.kayak.co.in/hotels/{location}/{check_in_date}/{check_out_date}"
    return URL

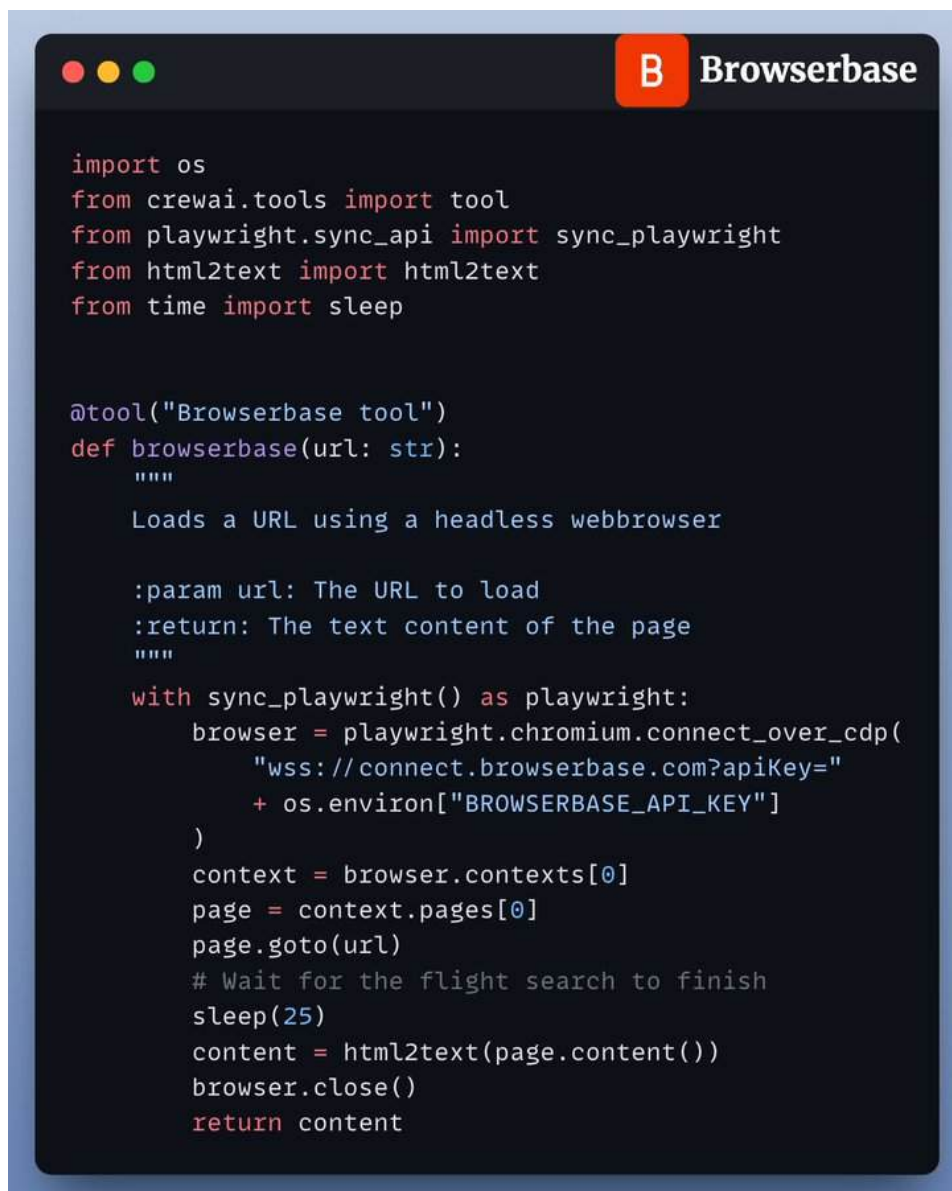
# Export the new decorated function
kayak_hotels = kayak_hotel_search
```

#5) Browserbase Tool

The hotel search agent uses the Browserbase tool to simulate human browsing and gather hotel data.

To be precise it automatically navigates the Kayak website and interacts with the web page.

Check this out:

A screenshot of a code editor window with a dark background and light-colored text. The window has a title bar with three colored circles (red, yellow, green) on the left and a tab labeled 'B Browserbase' on the right. The code is written in Python and defines a tool named 'Browserbase tool'. It imports modules like 'os', 'crewai.tools', 'playwright.sync_api', 'html2text', and 'time'. The tool function 'browserbase(url: str)' is decorated with '@tool' and has a docstring describing its purpose: 'Loads a URL using a headless webbrowser'. It includes a parameter 'url' and a return value 'The text content of the page'. The function body uses 'sync_playwright()' to create a playwright object, connects to a headless browser via 'connect_over_cdp' using a 'wss' endpoint and an API key from the environment. It then navigates to the specified URL, waits for 25 seconds, and returns the page content converted to text using 'html2text'.

```
import os
from crewai.tools import tool
from playwright.sync_api import sync_playwright
from html2text import html2text
from time import sleep

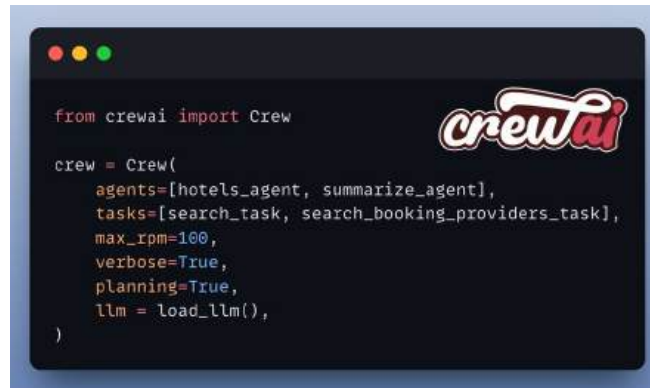
@tool("Browserbase tool")
def browserbase(url: str):
    """
    Loads a URL using a headless webbrowser

    :param url: The URL to load
    :return: The text content of the page
    """
    with sync_playwright() as playwright:
        browser = playwright.chromium.connect_over_cdp(
            "wss://connect.browserbase.com?apiKey="
            + os.environ["BROWSERBASE_API_KEY"]
        )
        context = browser.contexts[0]
        page = context.pages[0]
        page.goto(url)
        # Wait for the flight search to finish
        sleep(25)
        content = html2text(page.content())
        browser.close()
        return content
```

#6) Setup Crew

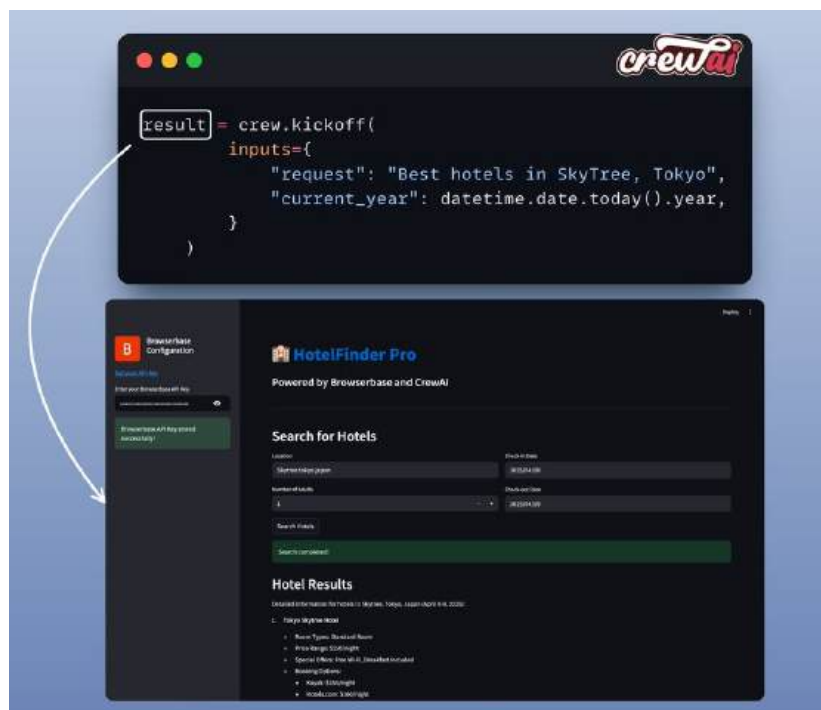
Once the agents and tools are defined, we orchestrate them using CrewAI.

Define their tasks, sequence their actions, and watch them collaborate in real time! Check this out:



#7) Kickoff and results

Finally, we feed the user's request (location, dates etc.) into the Crew and let it run! Check this out:

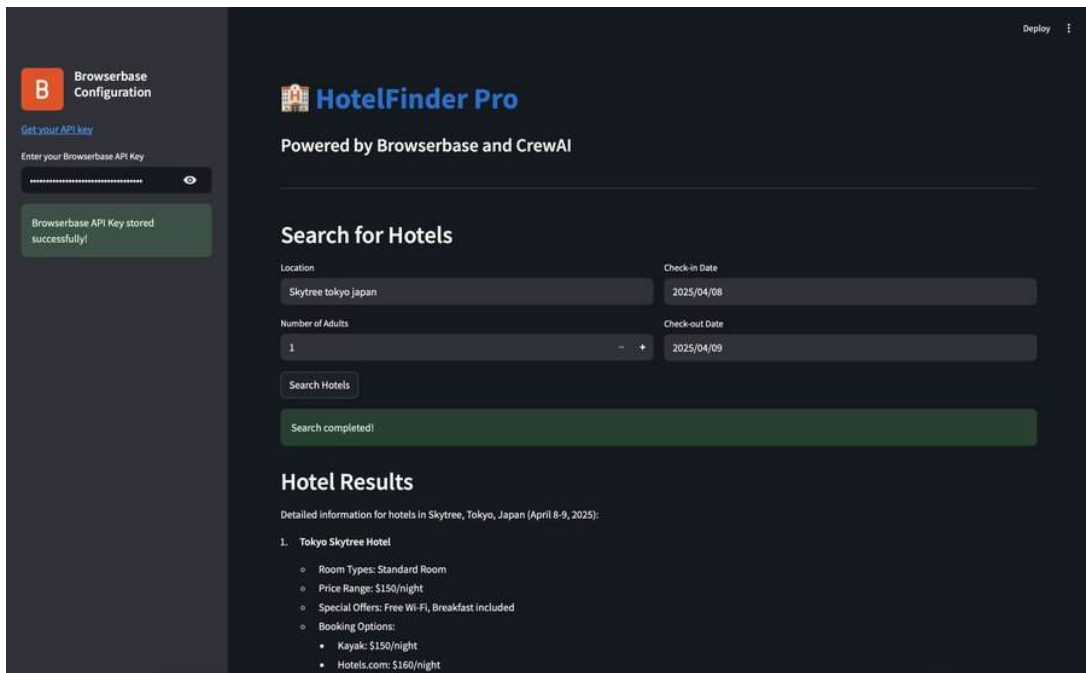


Streamlit UI

To make this accessible, we wrapped the entire system in a @Streamlit interface.

It's a simple chat-like UI where you enter your location and other details and see the results in real time!

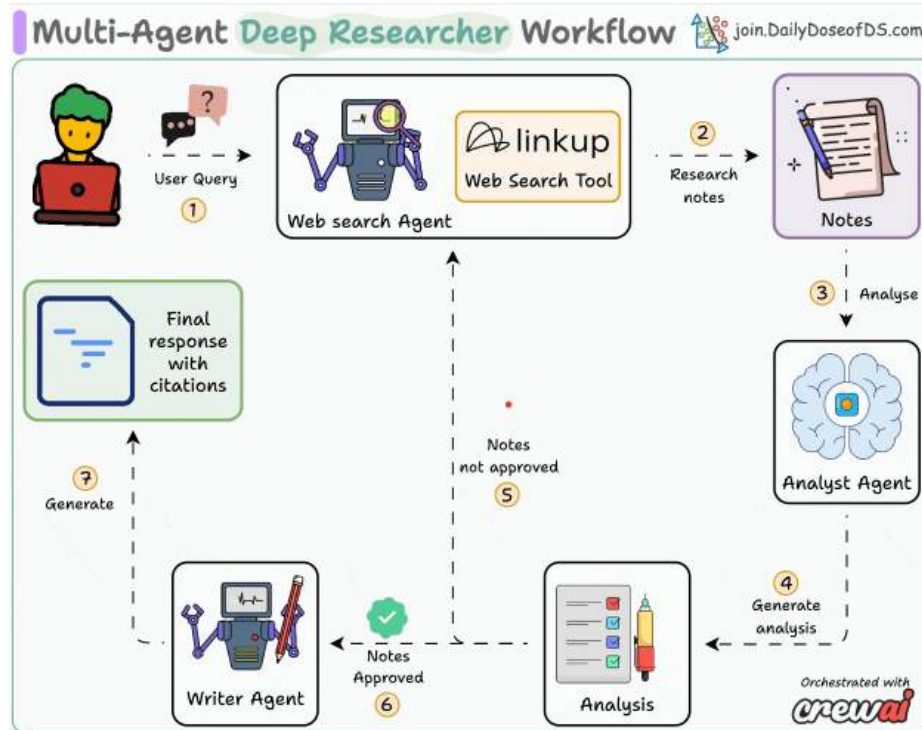
Check this out:



The code is available here:
<https://github.com/patchy631/ai-engineering-hub/tree/main/hotel-booking-crew>

#7) Multi-agent Deep Researcher

ChatGPT has a deep research feature. It helps you get detailed insights on any topic. Learn how you can build a 100% local alternative to it.



Tech stack:

- Linkup platform for deep web research
- CrewAI for multi-agent orchestration
- Ollama to locally serve DeepSeek
- Cursor as MCP host

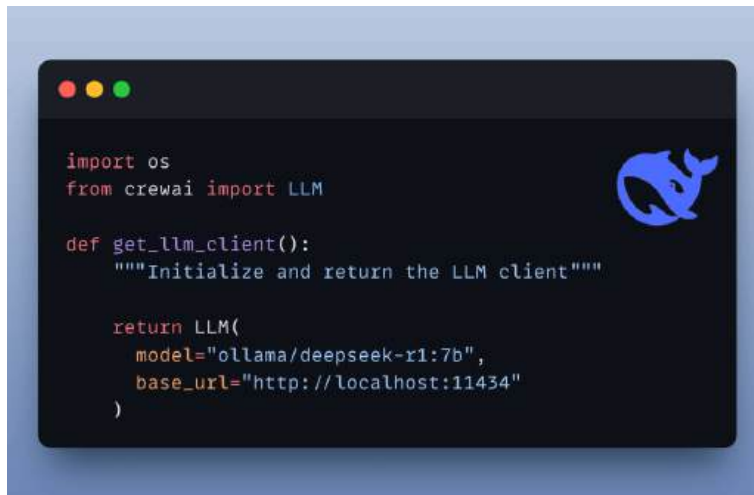
Workflow:

- User submits a query
- Web search agent runs deep web search via Linkup
- Research analyst verifies and deduplicates results
- Technical writer crafts a coherent response with citations

Let's implement this!

#1) Setup LLM

We'll use a locally served DeepSeek-R1 using Ollama.



#2) Define Web Search Tool

We'll use Linkup platform's powerful search capabilities, which rival Perplexity and OpenAI, to power our web search agent. This is done by defining a custom tool that our agent can use.

```
import os
from typing import Type
from pydantic import BaseModel, Field
from linkup import LinkupClient
from crewai.tools import BaseTool

class LinkupSearchInput(BaseModel):
    """Input schema for LinkUp Search Tool."""
    query: str = Field(description="The search query to perform")
    depth: str = Field(default="standard", description="Depth of search: 'standard' or 'deep'")
    output_type: str = Field(
        default="searchResults",
        description="Output type: 'searchResults' or 'sourcedAnswer'"
    )

class LinkupSearchTool(BaseTool):
    name: str = "LinkUp Search"
    description: str = "Retrieve info from the web using LinkUp and return results"
    args_schema: Type[BaseModel] = LinkupSearchInput

    def _run(self, query: str, depth: str = "standard",
            output_type: str = "searchResults") -> str:
        """Execute LinkUp search and return results."""
        # Initialize Linkup client with API key from environment variables
        linkup_client = LinkupClient(api_key=os.getenv("LINKUP_API_KEY"))

        # Perform search
        search_response = linkup_client.search(query=query,
                                              depth=depth,
                                              output_type=output_type)

        return str(search_response)
```

#3) Define Web Search Agent

The web search agent gathers up-to-date information from the internet based on user query. The linkup tool we defined earlier is used by this agent.

```
from crewai import Agent, Task

linkup_search_tool = LinkupSearchTool()

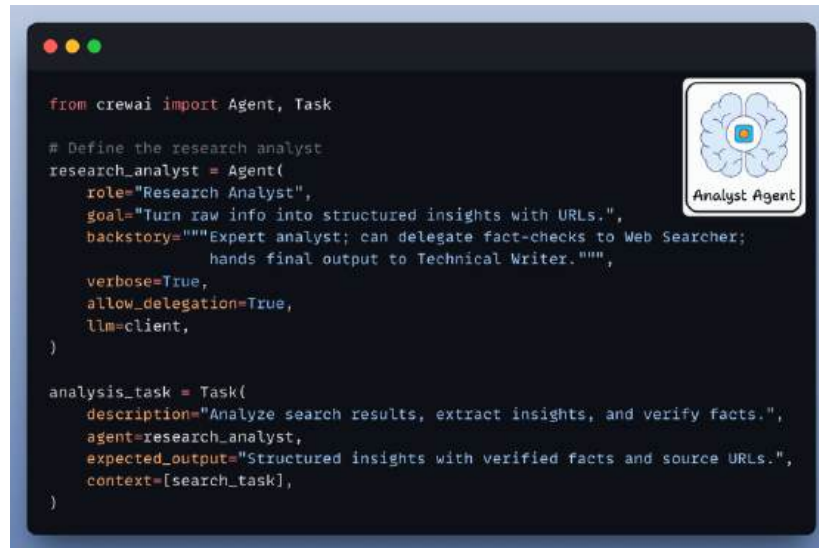
client = get_llm_client()

web_searcher = Agent(
    role="Web Searcher",
    goal="Retrieve relevant info with citations (source URLs)",
    backstory="Expert searcher; forwards results to Research Analyst only.",
    verbose=True,
    allow_delegation=True,
    tools=[linkup_search_tool],
)

search_task = Task(
    description=f"Search for comprehensive information about: {query}.",
    agent=web_searcher,
    expected_output="Detailed raw search results including sources (urls).",
    tools=[linkup_search_tool]
)
```

#4) Define Research Analyst Agent

This agent transforms raw web search results into structured insights, with source URLs. It can also delegate tasks back to the web search agent for verification and fact-checking.



```
from crewai import Agent, Task

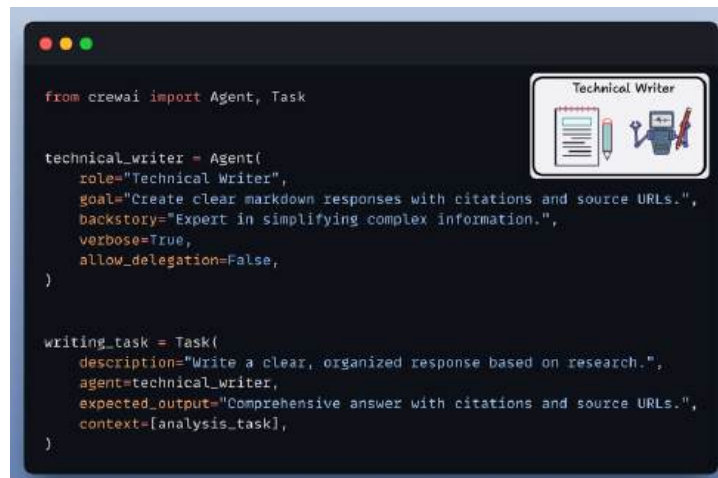
# Define the research analyst
research_analyst = Agent(
    role="Research Analyst",
    goal="Turn raw info into structured insights with URLs.",
    backstory="Expert analyst; can delegate fact-checks to Web Searcher; hands final output to Technical Writer.",
    verbose=True,
    allow_delegation=True,
    llm=client,
)

analysis_task = Task(
    description="Analyze search results, extract insights, and verify facts.",
    agent=research_analyst,
    expected_output="Structured insights with verified facts and source URLs.",
    context=[search_task],
)
```



#5) Define Technical Writer Agent

It takes the analyzed and verified results from the analyst agent and drafts a coherent response with citations for the end user.



```
from crewai import Agent, Task

technical_writer = Agent(
    role="Technical Writer",
    goal="Create clear markdown responses with citations and source URLs.",
    backstory="Expert in simplifying complex information.",
    verbose=True,
    allow_delegation=False,
)

writing_task = Task(
    description="Write a clear, organized response based on research.",
    agent=technical_writer,
    expected_output="Comprehensive answer with citations and source URLs.",
    context=[analysis_task],
)
```



#6) Setup Crew

Finally, once we have all the agents and tools defined we set up and kickoff our deep researcher crew.



#7) Create MCP Server

Now, we'll encapsulate our deep research team within an MCP tool. With just a few lines of code, our MCP server will be ready.

Let's see how to connect it with Cursor.

```
from mcp.server.fastmcp import FastMCP
from agents import run_research

# Create FastMCP instance
mcp = FastMCP("crew_research")

@mcp.tool()
def crew_research(query: str) → str:
    """
    Run CrewAI-based deep-research system for given user query.

    Args:
        query (str): The research query or question.

    Returns:
        str: The research response from the CrewAI pipeline.
    """
    return run_research(query)

# Run the server
if __name__ == "__main__":
    mcp.run(transport="stdio")
```

 Model Context Protocol

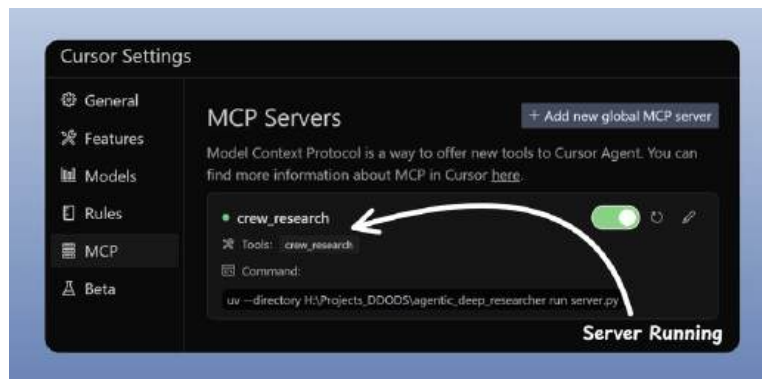
#8) Integrate MCP server with Cursor

Go to: File → Preferences → Cursor Settings → MCP → Add new global MCP server

In the JSON file, add what's shown below



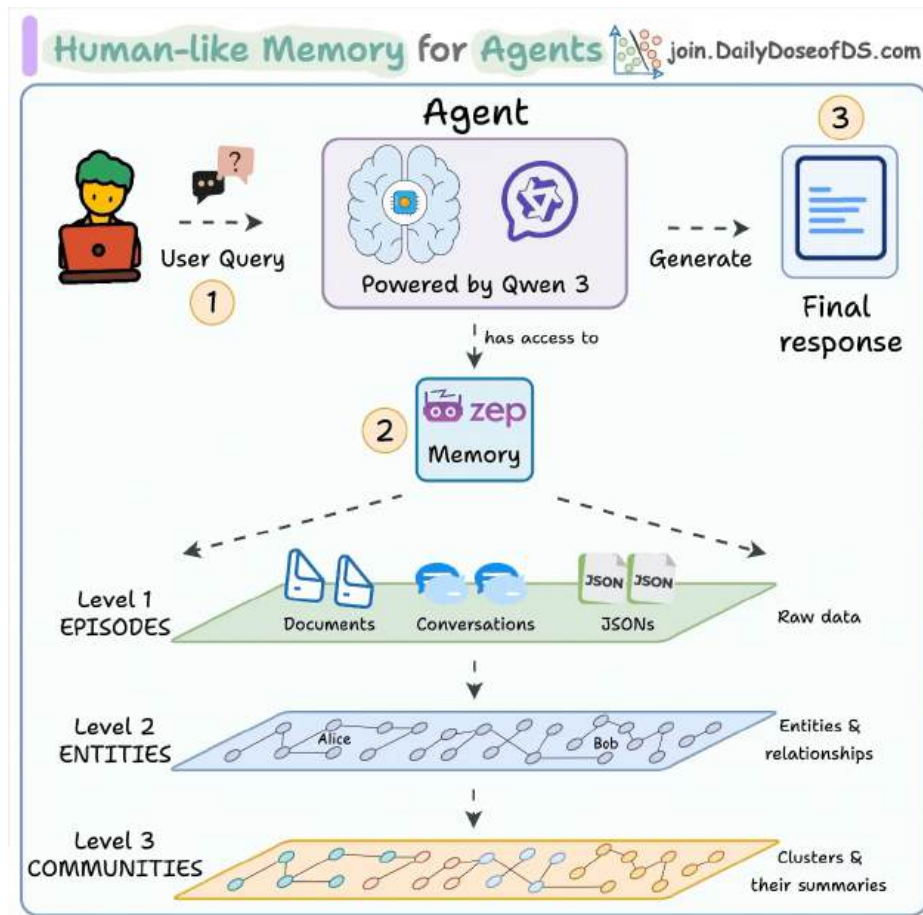
Done! Your deep research MCP server is live and connected to Cursor.



The code is available here:
<https://www.dailydoseofds.com/p/hands-on-mcp-powered-deep-researcher/>

#8) Human-like Memory for Agents

If a memory-less AI Agent is deployed in production, every interaction with the Agent will be a blank slate. Learn how to build an AI Agent with human-like memory to solve this.



Tech stack:

- Zep AI for the memory layer to AI agent
- Microsoft AutoGen for agent orchestration
- Ollama to locally serve Qwen3

Workflow:

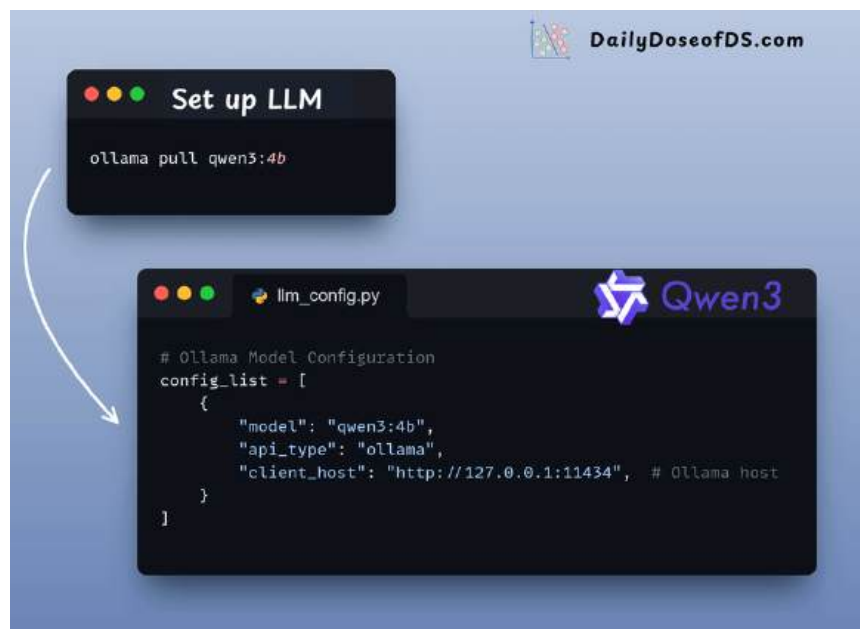
- User submits a query

- Agent saves the conversation and extracts facts into memory
- Agent retrieves facts and summarizes
- Uses facts and history for informed responses

Let's implement this!

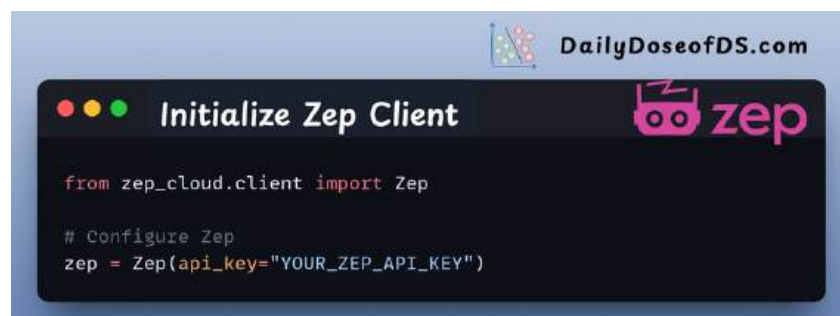
#1) Setup LLM

We'll use a locally served Qwen 3 via Ollama. Check this out:



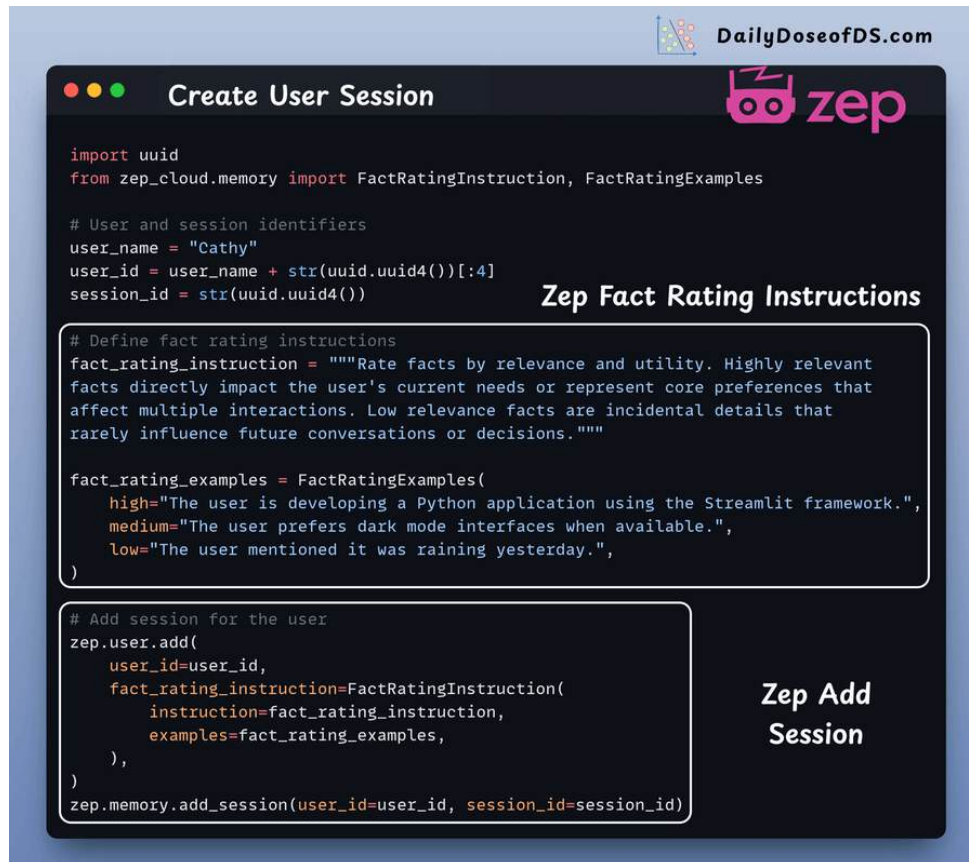
#2) Initialise Zep Client

We're leveraging zep_ai's Foundational Memory Layer to equip our Autogen agent with genuine task-completion capabilities.



#3) Create User Session

Create a Zep client session for the user, which the agent will use to manage memory. A user can have multiple sessions. Here's how it looks:



#4) Define Zep Conversable Agent

Our Zep Memory Agent builds on Autogen's Conversable Agent, drawing live memory context from Zep Cloud with each user query.

It remains efficient by utilizing the session we just established.

Here's how it comes together:



#5) Setting up Agents

We initialize the Conversable Agent and a Stand-in Human Agent to manage chat interactions.

Here's the setup:



Setting up Agents

```

from zep_cloud.client import Zep
from autogen import ConversableAgent
from agent import ZepConversableAgent
from llm_config import config_list

# Create the autogen agent with Zep memory
agent = ZepConversableAgent(
    name="Zep Agent",
    system_message="""
    You are 'Zep Agent', a helpful and professional assistant. Support the
    user by remembering past interactions, offering accurate and clear
    answers, and maintaining a friendly, empathetic tone. Prioritize recent
    context, avoid repeating known info, and never reveal internal workings.
    """,
    llm_config={"config_list": config_list},
    zep_session_id=session_id,
    zep_client=zep,
    min_fact_rating=0.7,
    function_map=None,
    human_input_mode="NEVER",
)

# Create Stand-in Human agent
user = ConversableAgent(
    name="Cathy",
    system_message="""
    You are a helpful mental health bot. You are
    seeking counsel from a mental health bot. Ask
    the bot about your previous conversation.
    """,
    llm_config={"config_list": config_list},
    human_input_mode="NEVER",
)
  
```

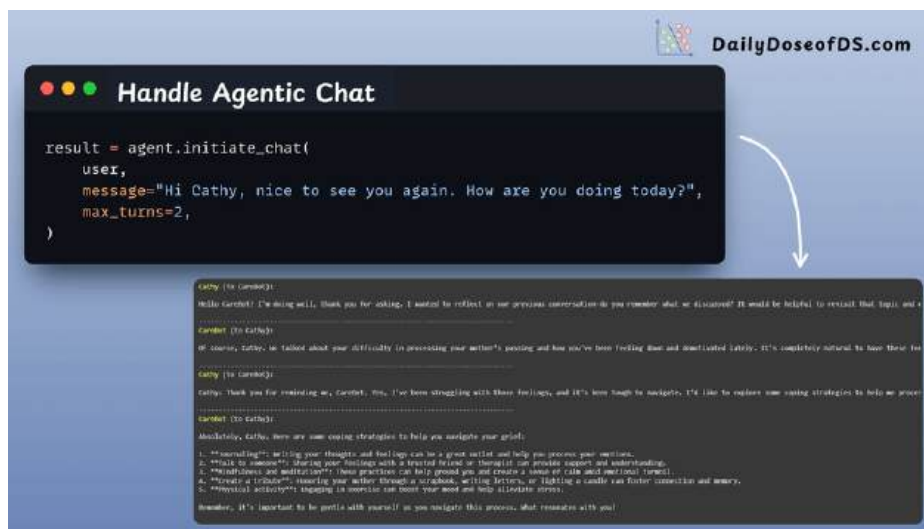
1. Zep Conversable Agent

2. Stand-in Human Agent

#6) Handle Agentic Chat

The Zep Conversable Agent steps in to create a coherent, personalized response.

It seamlessly integrates memory and conversation. Here's how it works:



Handle Agentic Chat

```

result = agent.initiate_chat(
    user,
    message="Hi Cathy, nice to see you again. How are you doing today?",
    max_turns=2,
)
  
```

Conversation Log:

```

Cathy (to Cardbot):
Hello Cardbot! I'm doing well, thank you for asking. I needed to reflect on our previous conversation do you remember what we discussed? It would be helpful to revisit that topic and...

Cardbot (to Cathy):
Of course, Cathy, we talked about your difficulty in processing your mother's passing and how you've been feeling blue and overwhelmed lately. It's completely natural to have these feelings...

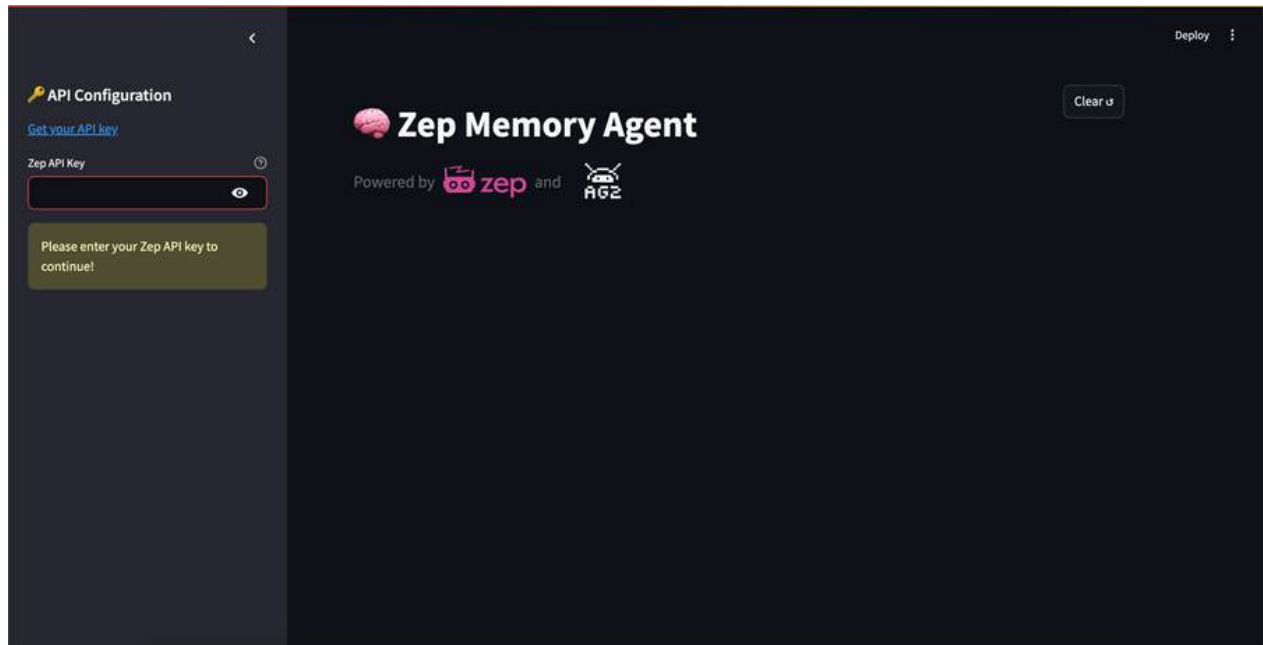
Cathy (to Cardbot):
Thank you for reminding me, Cardbot. Yes, I've been struggling with those feelings, and it's been tough to navigate. I'd like to explore some coping strategies to help me process...

Cardbot (to Cathy):
Absolutely, Cathy, here are some coping strategies to help you navigate your grief:
1. "Journaling": Writing your thoughts and feelings can be a great outlet and help you process your emotions.
2. "Mindfulness and meditation": These practices can help ground you and create a sense of calm and emotional balance.
3. "Physical activity": Exercising your body through a walk, jog, or yoga can help release endorphins and improve your mood.
4. "Social support": Reaching out to friends and family can provide you with emotional support and help alleviate stress.
Remember, it's important to be gentle with yourself as you navigate this process. What resonates with you?
  
```

#7) Streamlit UI

We created a streamlined Streamlit UI to ensure smooth and simple interactions with the Agent.

Here's what it looks like:

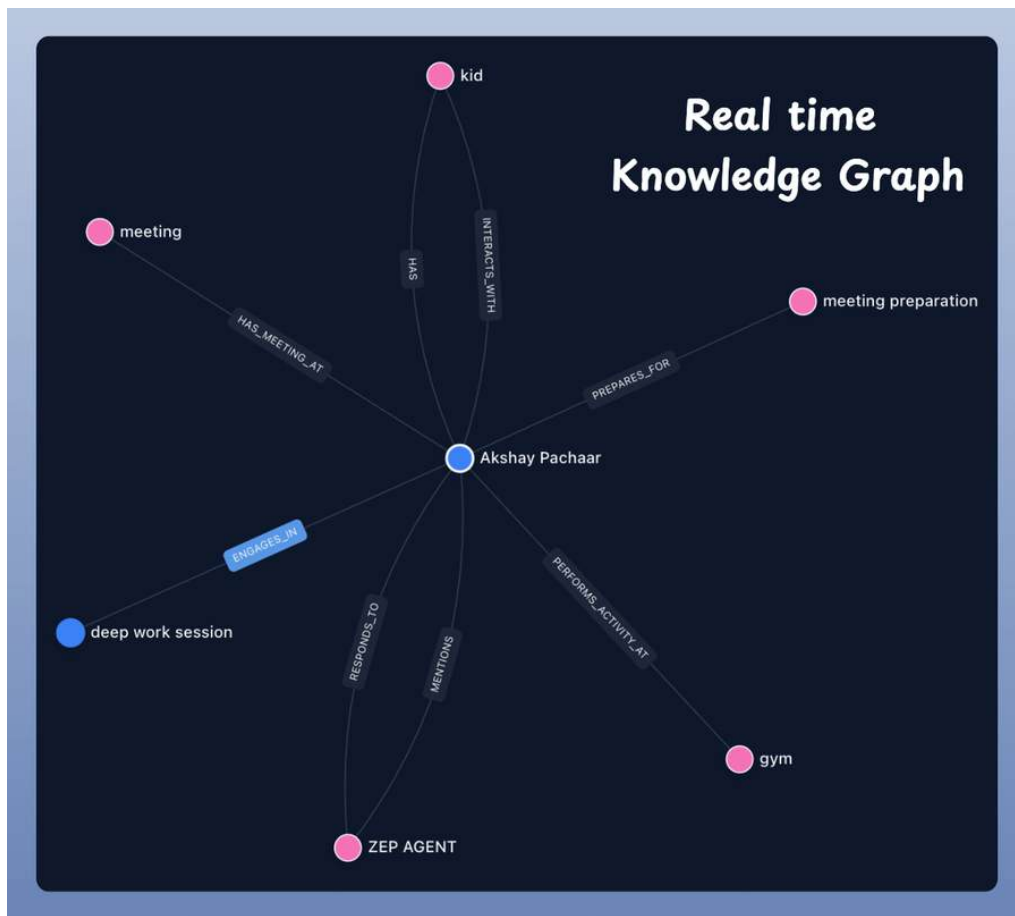


#8) Visualize Knowledge Graph

We can interactively map users' conversations across multiple sessions with Zep Cloud's UI.

This powerful tool allows us to visualize how knowledge evolves through a graph.

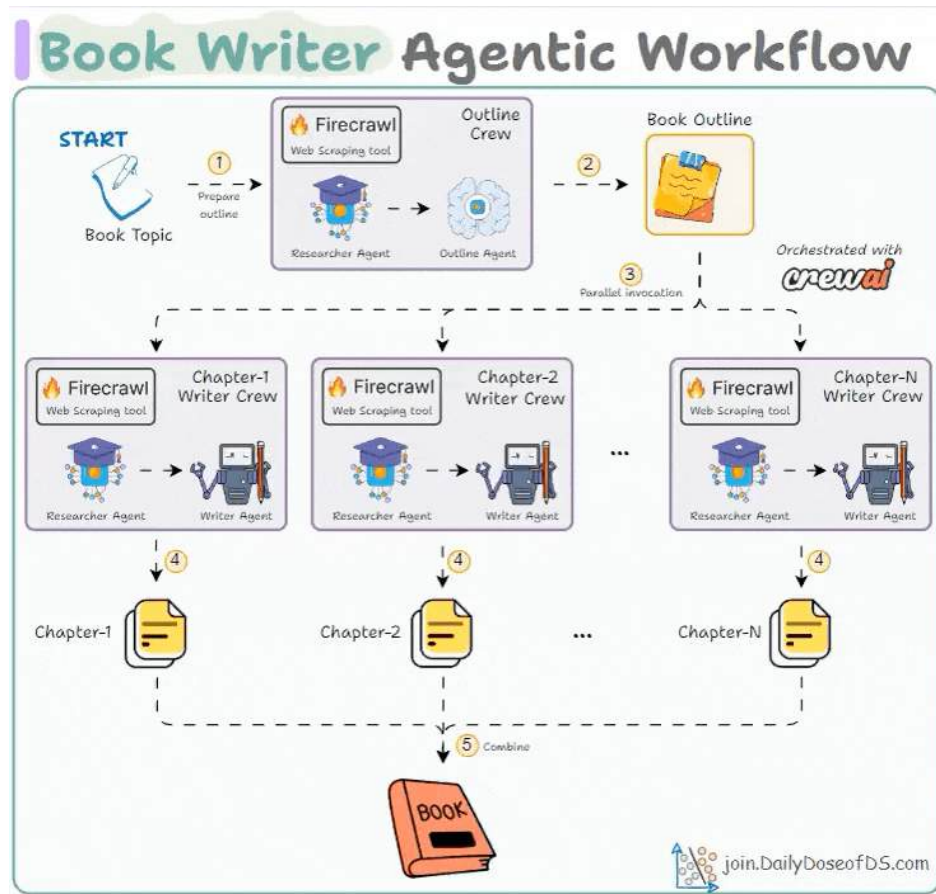
Take a look:



The code is available here:
<https://www.dailydoseofds.com/p/hands-on-building-an-ai-agent-with-human-like-memory/>

#9) Multi-agent Book Writer

Build an Agentic workflow that writes a 20k-word book from a 3-5 word book title.



Tech stack:

- Firecrawl for web scraping.
- CrewAI for orchestration.
- Ollama to serve Qwen 3 locally.
- LightningAI for development and hosting

Workflow:

- Using Firecrawl, Outline Crew scrapes data related to the book title and decides the chapter count and titles.
- Multiple writer Crews work in parallel to write one chapter each.
- Combine all chapters to get the book.

Let's implement this!

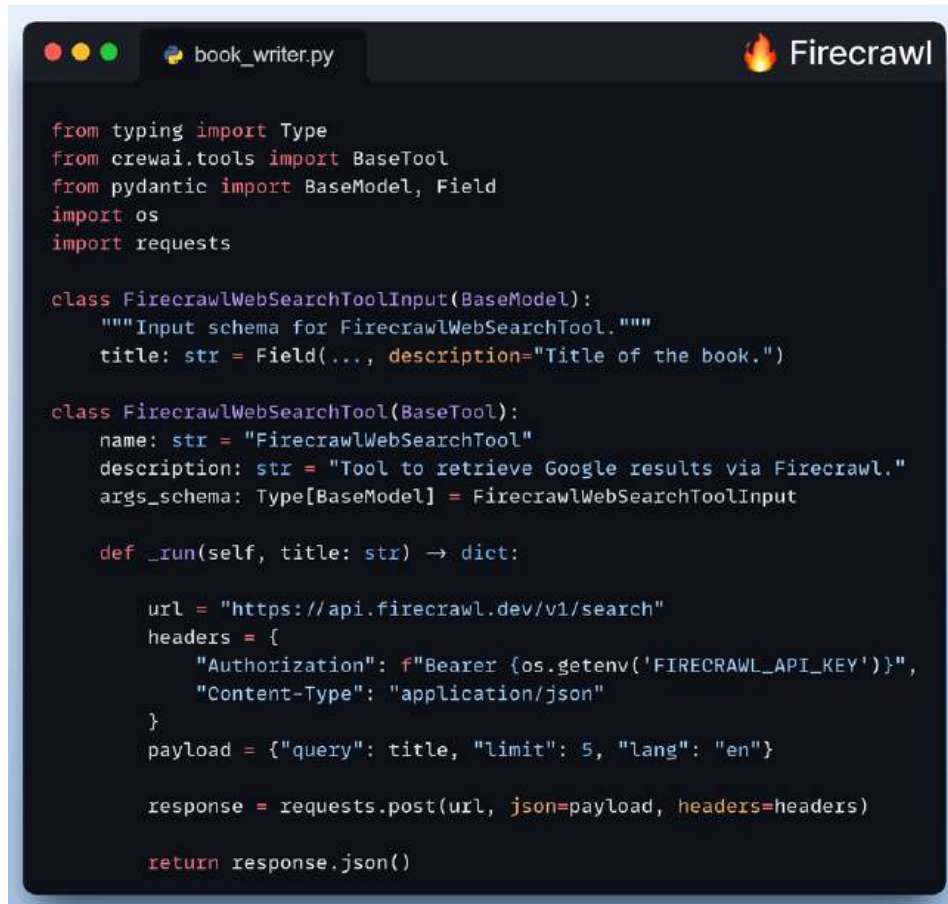
#1) Scraping tool - SERP API

Books demand research. Thus, we'll use Firecrawl's SERP API to scrape data.

Tool usage:

- Outline Crew → to research the book title and prepare an outline.
- Writer Crew → to research the chapter title and write it.

See this code:

A screenshot of a code editor window titled 'book_writer.py' with a 'Firecrawl' logo in the top right corner. The code defines a Pydantic model for search input and a tool class that uses the Firecrawl API to search for books.

```
from typing import Type
from crewai.tools import BaseTool
from pydantic import BaseModel, Field
import os
import requests

class FirecrawlWebSearchToolInput(BaseModel):
    """Input schema for FirecrawlWebSearchTool."""
    title: str = Field(..., description="Title of the book.")

class FirecrawlWebSearchTool(BaseTool):
    name: str = "FirecrawlWebSearchTool"
    description: str = "Tool to retrieve Google results via Firecrawl."
    args_schema: Type[BaseModel] = FirecrawlWebSearchToolInput

    def _run(self, title: str) → dict:

        url = "https://api.firecrawl.dev/v1/search"
        headers = {
            "Authorization": f"Bearer {os.getenv('FIRECRAWL_API_KEY')}",
            "Content-Type": "application/json"
        }
        payload = {"query": title, "limit": 5, "lang": "en"}

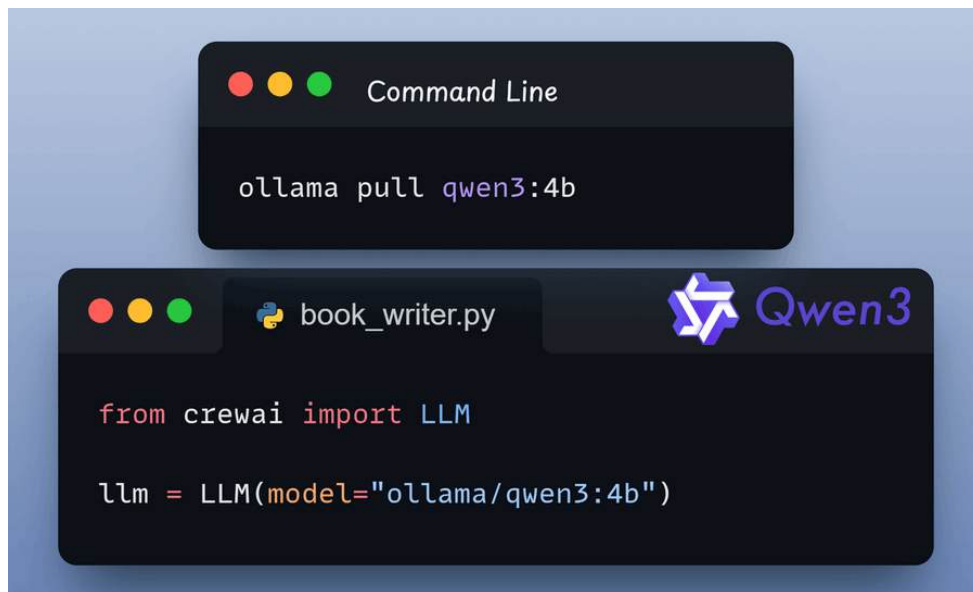
        response = requests.post(url, json=payload, headers=headers)

        return response.json()
```

#2) Setup Qwen 3 locally

We'll serve Qwen 3 locally through Ollama. To do this:

- First, we download it locally.
- Next, we define it with the CrewAI's LLM class.



#3) Outline Crew

This Crew has two Agents:

- Research Agent → Uses the Firecrawl scraping tool to scrape data related to the book's title and prepare insights.
- Outline Agent → Uses the insights to output total chapters and titles as Pydantic output.



#4) Writer Crew

This Crew has two Agents:

- Research Agent → Uses the Firecrawl Scraping tool to scrape data related to a chapter's title and prepare insights.
- Write Agent → Uses the insights to write a chapter.

Check this code:



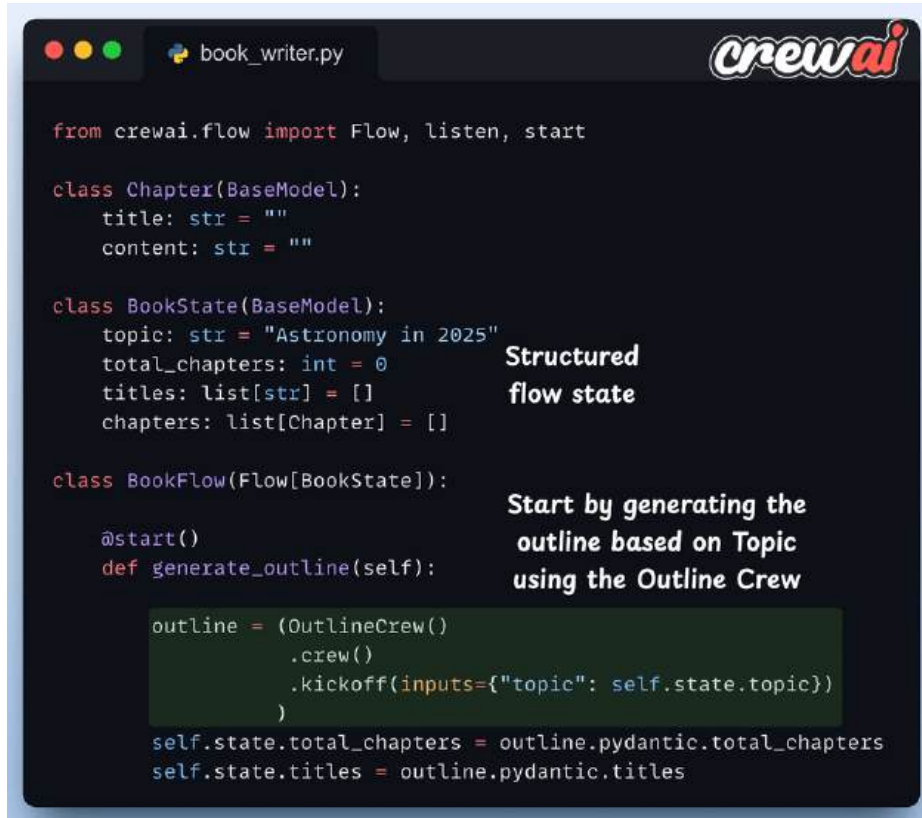
#5) Create a Flow

We use CrewAI Flows to orchestrate the workflow.

First, the outline method invokes the Outline Crew, which:

- research the topic using the scraping tool.
- returns the total number of chapters and the corresponding titles.

This is implemented below:



```
from crewai.flow import Flow, listen, start

class Chapter(BaseModel):
    title: str = ""
    content: str = ""

class BookState(BaseModel):
    topic: str = "Astronomy in 2025"
    total_chapters: int = 0
    titles: list[str] = []
    chapters: list[Chapter] = []

class BookFlow(Flow[BookState]):

    @start()
    def generate_outline(self):

        outline = (OutlineCrew()
                   .crew()
                   .kickoff(inputs={"topic": self.state.topic})
                  )

        self.state.total_chapters = outline.pydantic.total_chapters
        self.state.titles = outline.pydantic.titles
```

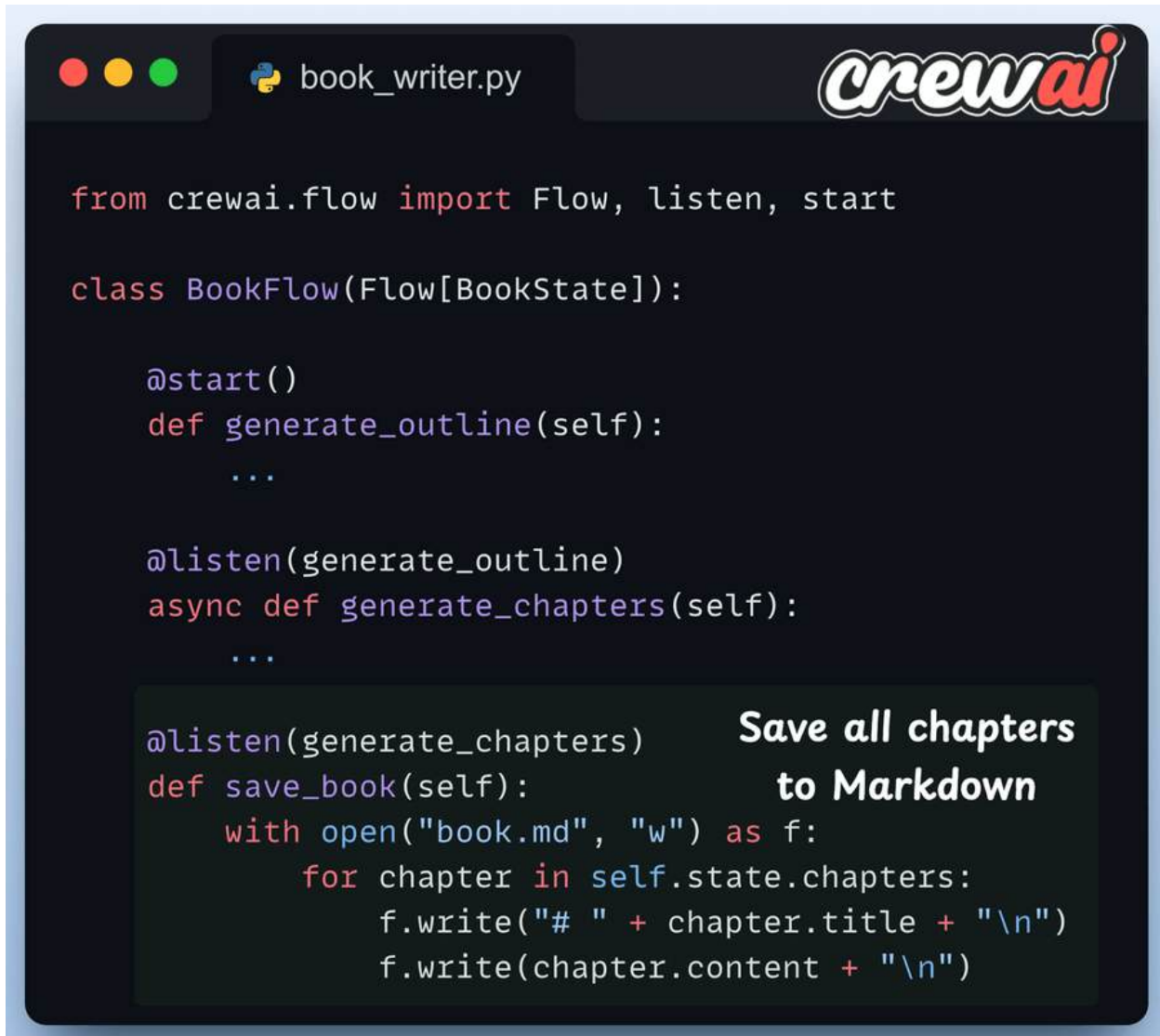
Structured flow state

Start by generating the outline based on Topic using the Outline Crew

#6) Save the book

Once all Writer Crews have finished execution, we save the book as a Markdown file.

Check this code:



```
from crewai.flow import Flow, listen, start

class BookFlow(Flow[BookState]):

    @start()
    def generate_outline(self):
        ...

    @listen(generate_outline)
    async def generate_chapters(self):
        ...

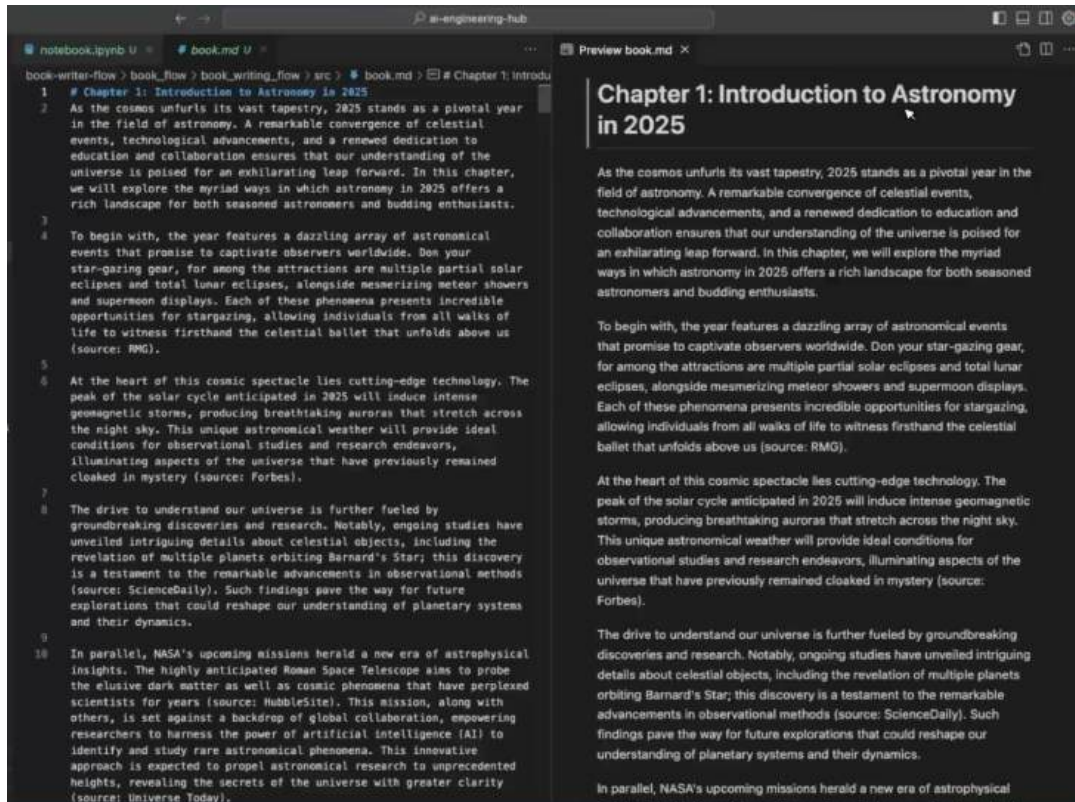
    @listen(generate_chapters)
    def save_book(self):
        Save all chapters to Markdown
        with open("book.md", "w") as f:
            for chapter in self.state.chapters:
                f.write("# " + chapter.title + "\n")
                f.write(chapter.content + "\n")
```

#7) Kickoff the Flow

Finally, we run the Flow.

- First, the Outline Crew is invoked. It utilizes the Firecrawl scraping tool to prepare the outline.
- Next, many Writer Crews are invoked in parallel to write one chapter each.

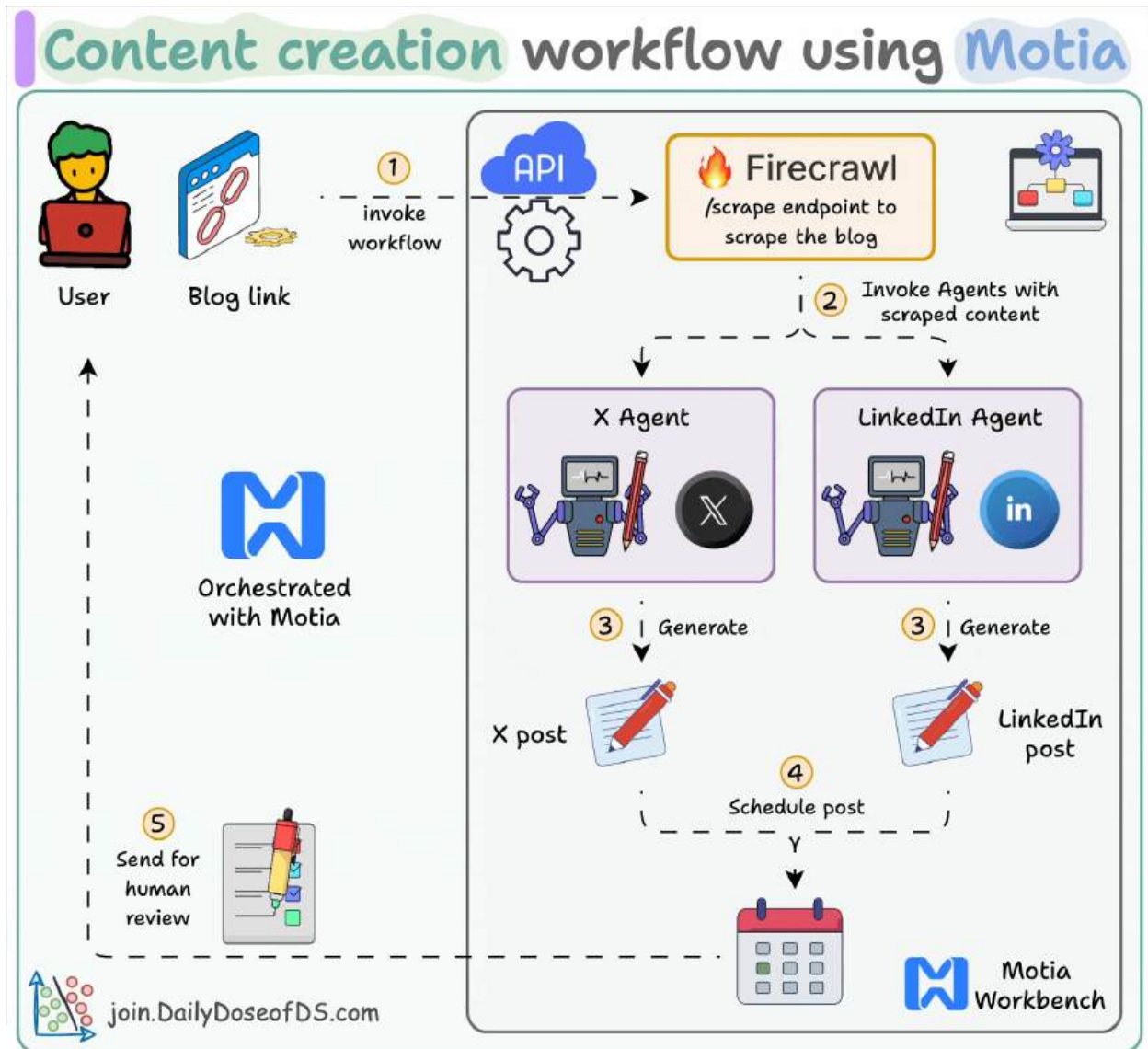
This workflow runs for ~2 minutes, and we get a neatly written book about the specified topic—"Astronomy in 2025." Here's the book our Agentic workflow wrote:



The code is available here:
<https://blog.dailydoseofds.com/p/building-a-multi-agent-book-writer/>

#10) Multi-agent Content Creation System

Build an Agentic workflow that turns any URL into social media posts and auto-schedules them via Typefully.



Tech stack:

- Motia as the unified backend framework
- Firecrawl to scrape web content
- Ollama to locally serve Deepseek-R1 LLM

Workflow:

- User submits URL to scrape
- Firecrawl scrapes content and converts it to markdown
- Twitter and LinkedIn agents run in parallel to generate content
- Generated content gets scheduled via Typefully

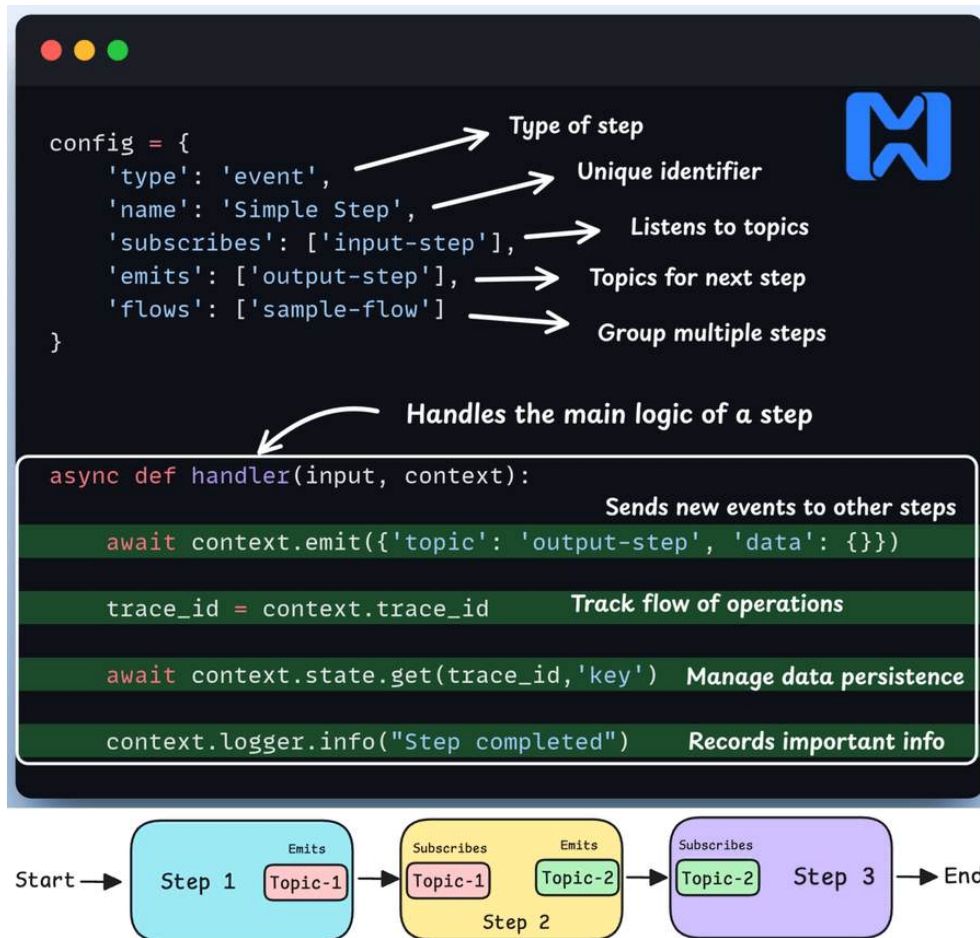
Let's implement this:

Steps are the fundamental building blocks of Motia.

They consist of two main components:

- The Config object: It instructs Motia on how to interact with a step.
- The handler function: It defines the main logic of a step.

Check this out:

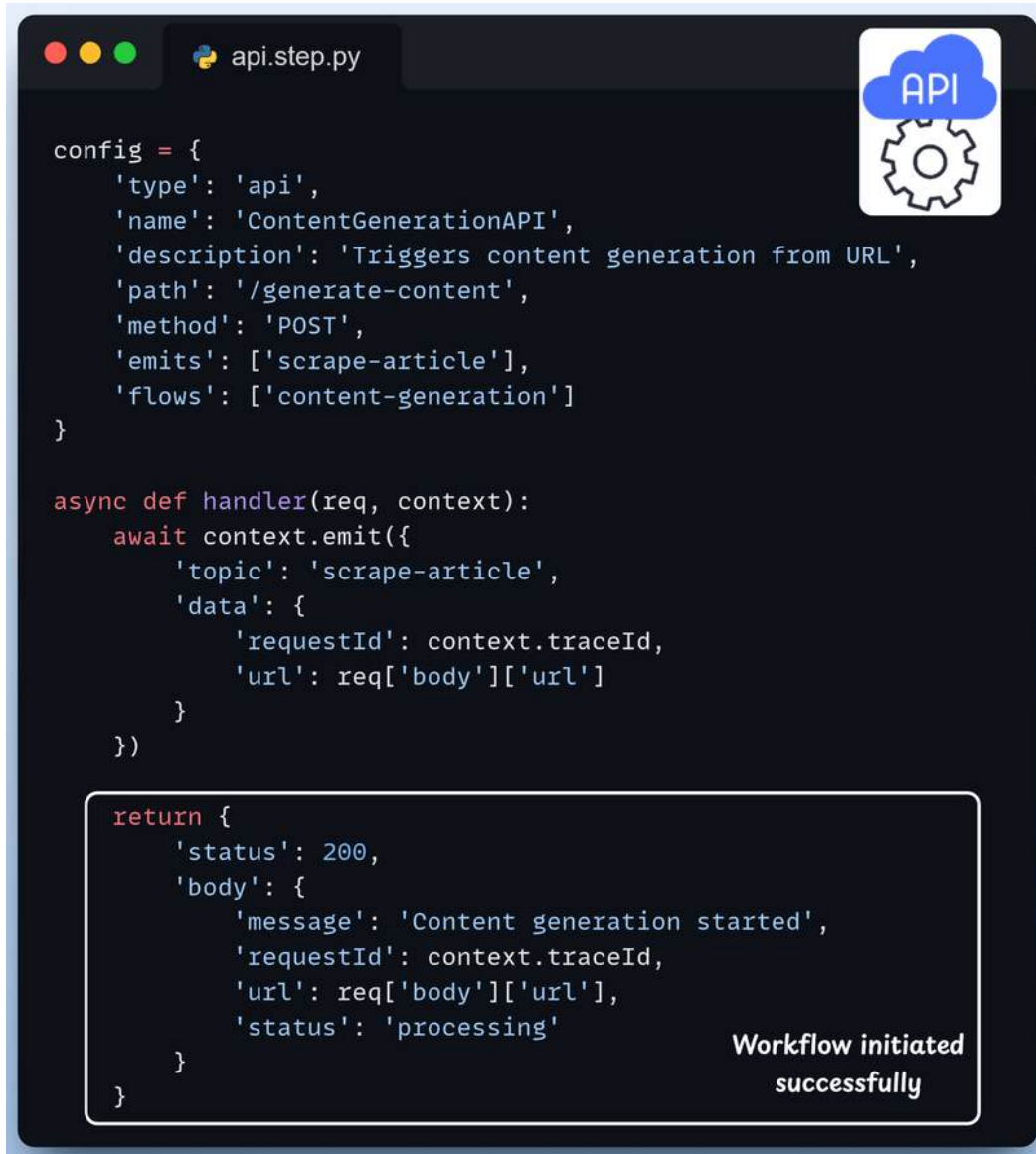


With that understanding in mind, let's start building our content creation workflow.

#1) Entry point (API)

We start our content generation workflow by defining an API step that takes in a URL from the user via a POST request.

Check this out:



```
api.step.py

config = {
    'type': 'api',
    'name': 'ContentGenerationAPI',
    'description': 'Triggers content generation from URL',
    'path': '/generate-content',
    'method': 'POST',
    'emits': ['scrape-article'],
    'flows': ['content-generation']
}

async def handler(req, context):
    await context.emit({
        'topic': 'scrape-article',
        'data': {
            'requestId': context.traceId,
            'url': req['body']['url']
        }
    })

    return {
        'status': 200,
        'body': {
            'message': 'Content generation started',
            'requestId': context.traceId,
            'url': req['body']['url'],
            'status': 'processing'
        }
    }
```

Workflow initiated successfully

#2) Web scraping

This step scrapes the article content using Firecrawl and emits the next step in the workflow.

Steps can be connected together in a sequence, where the output of one step becomes the input for another.

Check this out:



```
from firecrawl import FirecrawlApp

config = {
    'type': 'event',
    'name': 'ScrapeArticle',
    'description': 'Scrapes content using Firecrawl',
    'subscribes': ['scrape-article'],
    'emits': ['generate-content'],
    'input': ScrapeInput,
    'flows': ['content-generation']
}

async def handler(input, context):
    context.logger.info(f"Scraping article: {input['url']}")

    app = FirecrawlApp(api_key=FIRECRAWL_API_KEY)
    scrapeResult = await app.scrape_url(input['url'])

    await context.emit({
        'topic': 'generate-content',
        'data': {
            'requestId': input['requestId'],
            'url': str(input['url']),
            'title': scrapeResult.metadata.get('title'),
            'content': scrapeResult.markdown,
        }
    })
```

Sent as input to the next step

#3) Content generation

The scraped content gets fed to the X and LinkedIn agents that run in parallel and generate curated posts.

We define all our prompting and AI logic in the handler that runs automatically when a step is triggered.

Check this out:

```
generate.step.py

import asyncio
from ollama import AsyncClient

config = {
    'type': 'event',
    'name': 'GenerateContent',
    'description': 'Generates social media content',
    'subscribes': ['generate-content'],
    'emits': ['schedule-content'],
    'input': GenerateInput,
    'flows': ['content-generation']
}

async def handler(input, context):
    twitter_resp, linkedin_resp = await asyncio.gather(
        AsyncClient().chat(model="deepseek-r1",
            messages=[{
                'role': 'user',
                'content': twitter_prompt
            }]),
        AsyncClient().chat(model="deepseek-r1",
            messages=[{
                'role': 'user',
                'content': linkedin_prompt
            }])

    await context.emit({
        'topic': 'schedule-content',
        'data': { 'content': {twitter_content, linkedin_content} }
    })
```

Both agents invoked in parallel

X Agent



LinkedIn Agent



#4) Scheduling

After the content is generated we draft it in Typefully where we can easily review our social media posts.

Motia also allows us to mix and match different languages within the same workflow providing great flexibility.

Check this typescript code:



```
import { EventConfig, Handlers } from 'motia'
import axios from 'axios'

export const config: EventConfig = {
  type: 'event',
  name: 'Schedule',
  description: 'Schedules social media posts using Typefully',
  subscribes: ['schedule-content'],
  emits: [],
  flows: ['content-generation']
}

export const handler: Handlers['Schedule'] = async (input, {logger}) => {
  const typefullyHeaders = {
    'X-API-KEY': `Bearer ${TYPEFULLY_API_KEY}`,
    'Content-Type': 'application/json'
  }

  await axios.post('https://api.typefully.com/v1/drafts/', {
    content: twitterTweets.join('\n\n\n\n'),
    schedule_date: twitterScheduleTime
  }, { headers: typefullyHeaders })

  await axios.post('https://api.typefully.com/v1/drafts/', {
    content: input.content.linkedin.post,
    schedule_date: linkedinScheduleTime,
  }, { headers: typefullyHeaders })

  logger.info(`Content scheduling completed successfully!\n`)
}
```

After defining our steps, we install required dependencies using `npm install` and run the Motia workbench using `npm run dev` commands.

Check this out:



```
command line
$ npm install      → Install dependencies
$ npx motia dev    → Run development server

> dev
> . python_modules/bin/activate && motia dev

Activating Python environment...
→[CREATED] Flow content-generation created
→[CREATED] Step (Event) steps/scrape.step.py created
→[CREATED] Step (Event) steps/schedule.step.ts created
→[CREATED] Step (Event) steps/generate.step.py created
→[CREATED] Step (API) steps/api.step.py created
🔥 Server ready and listening on port 3000
🔗 Open http://localhost:3000/ to open workbench 🌟
```

Motia workbench provides an interactive UI to help build, monitor and debug our flows.

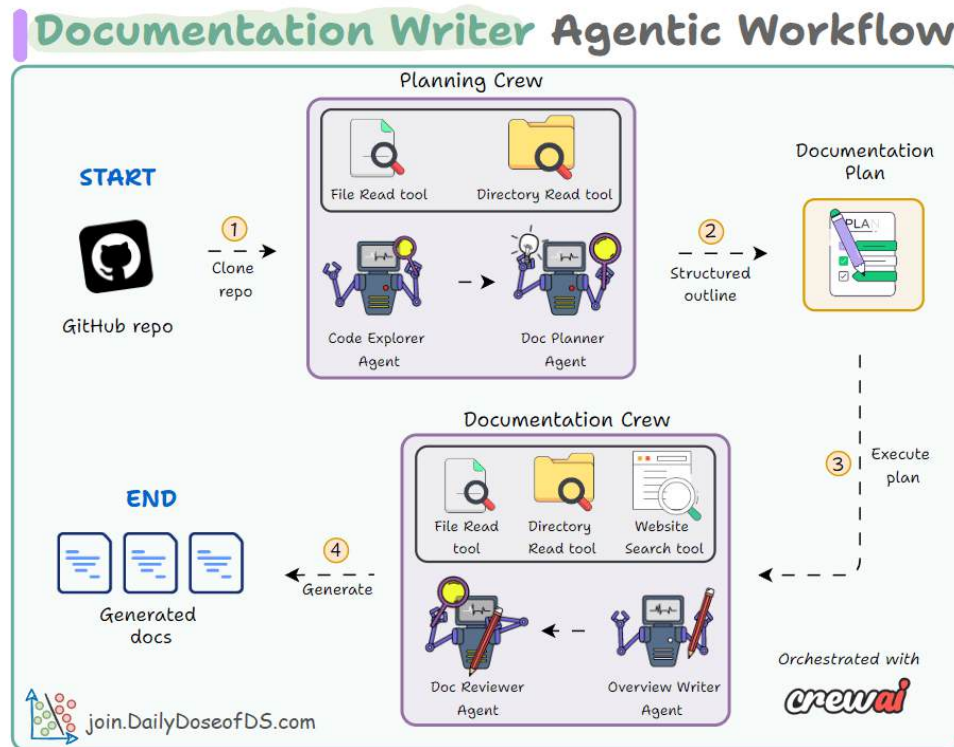
With one-click you can also deploy it to the cloud!



The code is available here:
<https://blog.dailydoseofds.com/p/build-a-multi-agent-content-creation/>

#11) Documentation Writer Flow

Build an Agentic workflow that generates full project documentation from just a GitHub repo URL.



Tech stack:

- CrewAI for multi-agent orchestration
- Ollama to locally serve DeepSeek-R1 LLM

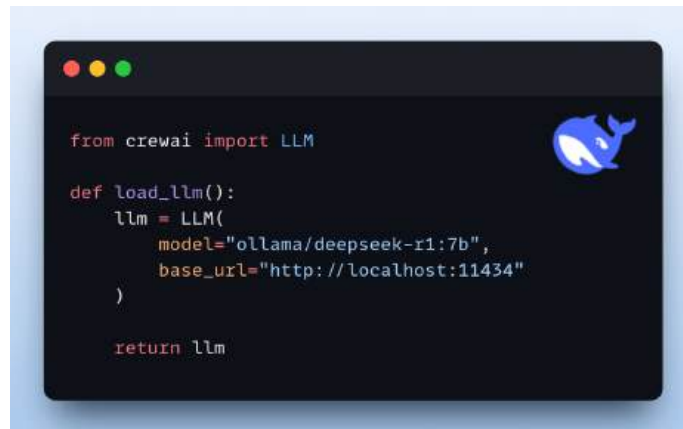
Workflow:

- User specifies GitHub repo
- Planning crew creates documentation plan
- Documentation crew writes documentation according to plan
- Generated docs get saved to local directory

Let's implement this!

#1) Setup LLM

We will use Deepseek-R1 as the LLM, served locally using Ollama.



```
from crewai import LLM

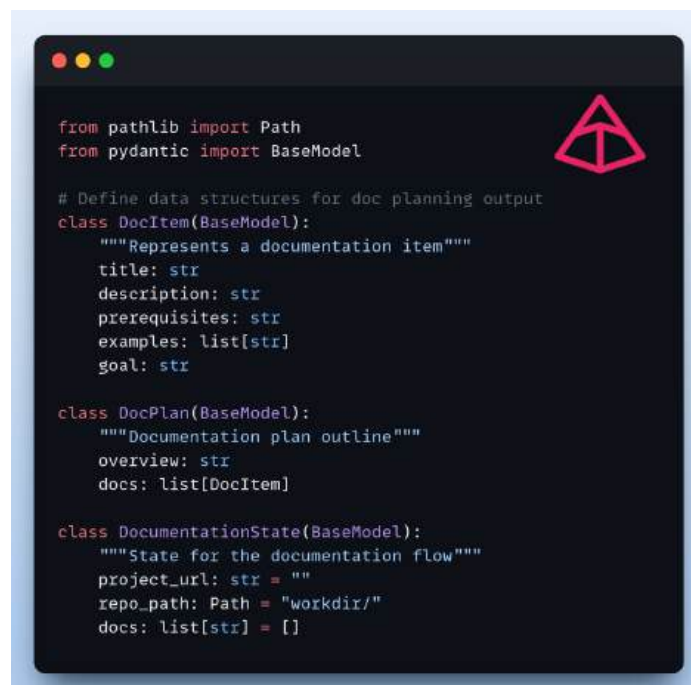
def load_llm():
    llm = LLM(
        model="ollama/deepseek-r1:7b",
        base_url="http://localhost:11434"
    )

    return llm
```

#2) Define Pydantic schema

We define the following pydantic schemas for robust structured outputs.

This ensures data validation and integrity before generating the documentation files. Check this code:



```
from pathlib import Path
from pydantic import BaseModel

# Define data structures for doc planning output
class DocItem(BaseModel):
    """Represents a documentation item"""
    title: str
    description: str
    prerequisites: str
    examples: list[str]
    goal: str

class DocPlan(BaseModel):
    """Documentation plan outline"""
    overview: str
    docs: list[DocItem]

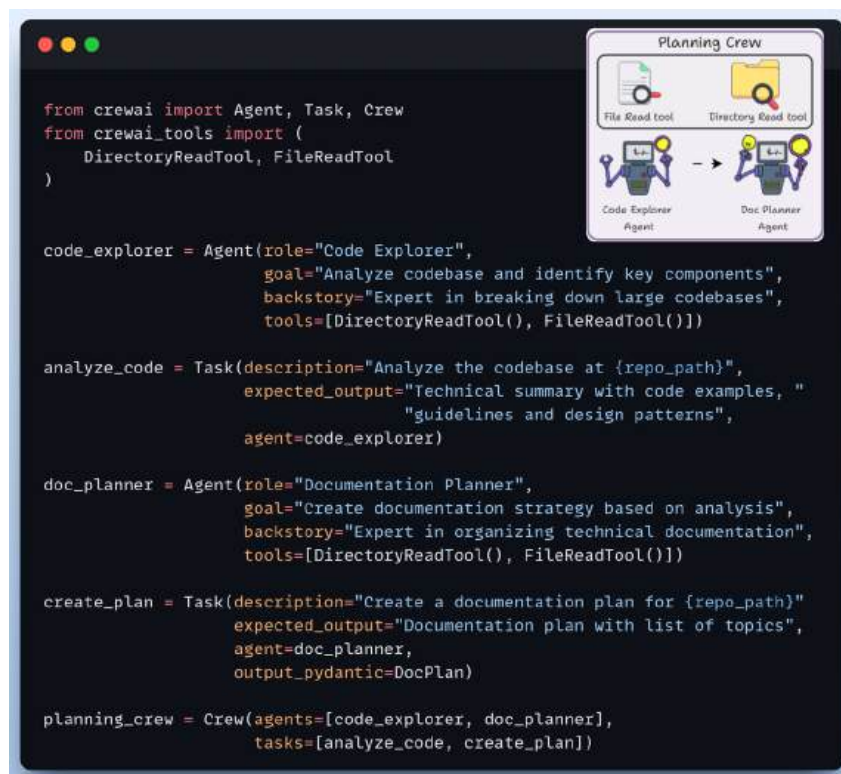
class DocumentationState(BaseModel):
    """State for the documentation flow"""
    project_url: str = ""
    repo_path: Path = "workdir/"
    docs: list[str] = []
```

#3) Planning Crew

This crew oversees strategy for the outline via:

- Code explorer agent -> Analyzes codebase for key components, patterns and relationships.
- Doc planner agent -> Creates outline based on codebase analysis as pydantic output.

Check this out:



```
from crewai import Agent, Task, Crew
from crewai_tools import (
    DirectoryReadTool, FileReadTool
)

code_explorer = Agent(role="Code Explorer",
    goal="Analyze codebase and identify key components",
    backstory="Expert in breaking down large codebases",
    tools=[DirectoryReadTool(), FileReadTool()])

analyze_code = Task(description="Analyze the codebase at {repo_path}",
    expected_output="Technical summary with code examples, "
    "guidelines and design patterns",
    agent=code_explorer)

doc_planner = Agent(role="Documentation Planner",
    goal="Create documentation strategy based on analysis",
    backstory="Expert in organizing technical documentation",
    tools=[DirectoryReadTool(), FileReadTool()])

create_plan = Task(description="Create a documentation plan for {repo_path}"
    expected_output="Documentation plan with list of topics",
    agent=doc_planner,
    output_pydantic=DocPlan)

planning_crew = Crew(agents=[code_explorer, doc_planner],
    tasks=[analyze_code, create_plan])
```

The diagram titled "Planning Crew" shows the workflow between two agents. At the top, it lists the tools: "File Read tool" and "Directory Read tool". Below, it shows the "Code Explorer Agent" and the "Doc Planner Agent" connected by a right-pointing arrow, indicating the flow of information from the code explorer to the documentation planner.

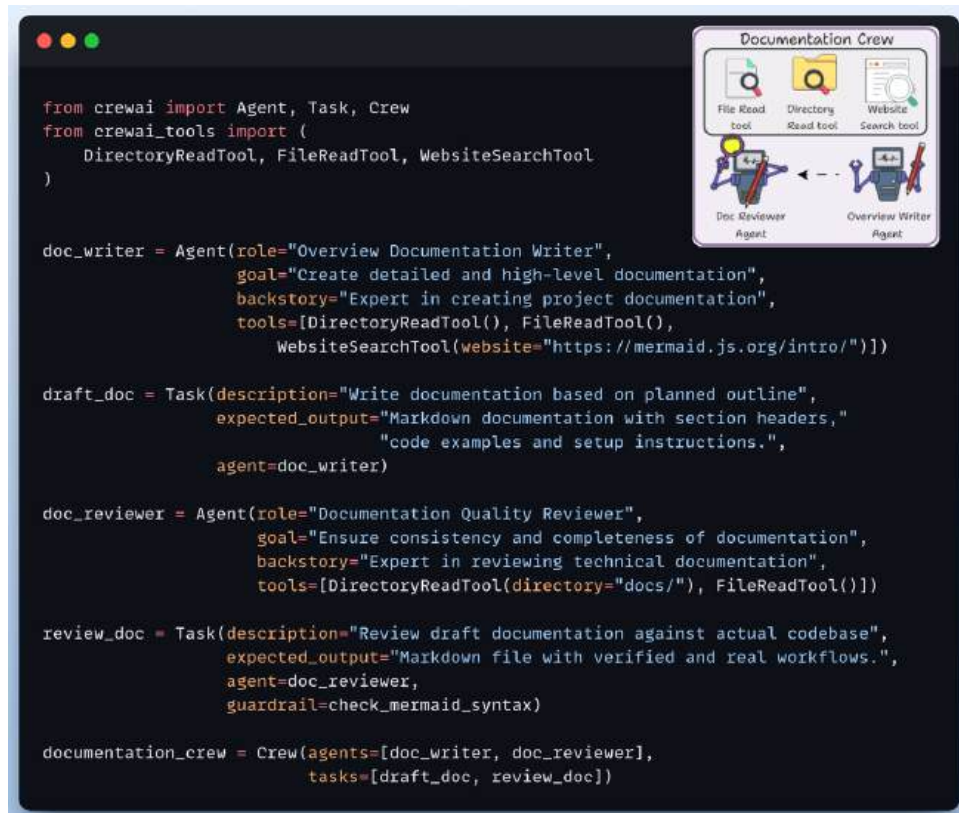
#4) Documentation Crew

This crew writes and reviews the documentation via:

- Doc writer agent -> Generates a high-level draft based on the planned outline.

- Doc reviewer agent -> Reviews the draft for consistency, accuracy and completeness.

Check this out:



```
from crewai import Agent, Task, Crew
from crewai_tools import (
    DirectoryReadTool, FileReadTool, WebsiteSearchTool
)

doc_writer = Agent(role="Overview Documentation Writer",
    goal="Create detailed and high-level documentation",
    backstory="Expert in creating project documentation",
    tools=[DirectoryReadTool(), FileReadTool(),
    WebsiteSearchTool(website="https://mermaid.js.org/intro/")]

draft_doc = Task(description="Write documentation based on planned outline",
    expected_output="Markdown documentation with section headers,"
    "code examples and setup instructions.",
    agent=doc_writer)

doc_reviewer = Agent(role="Documentation Quality Reviewer",
    goal="Ensure consistency and completeness of documentation",
    backstory="Expert in reviewing technical documentation",
    tools=[DirectoryReadTool(directory="docs/"), FileReadTool()])

review_doc = Task(description="Review draft documentation against actual codebase",
    expected_output="Markdown file with verified and real workflows.",
    agent=doc_reviewer,
    guardrail=check_mermaid_syntax)

documentation_crew = Crew(agents=[doc_writer, doc_reviewer],
    tasks=[draft_doc, review_doc])
```

The diagram titled "Documentation Crew" shows the workflow. At the top, three tools are listed: "File Read tool", "Directory Read tool", and "Website Search tool". Below these, two agents are shown: "Doc Reviewer Agent" and "Overview Writer Agent". An arrow points from the "Overview Writer Agent" to the "Doc Reviewer Agent", indicating the flow of the process.

#5) Create Documentation Flow

After setting up our crews, we create the main workflow that:

- Clones the GitHub repo
- Plans and saves the outline
- Generates documentation based on the outline
- Saves final docs to local directory

Check this code:



#6) Kickoff the flow

Finally, when we have everything ready, we kick off our documentation flow with the GitHub repo URL.

Check this out:

```
repo_url = "https://github.com/username/repo"

flow = CreateDocumentationFlow()

flow.kickoff(inputs={"project_url": repo_url})
```

Flow Execution

```
Starting Flow Execution
Name: CreateDocumentationFlow
ID: 0f54b988-afe9-4dfb-9433-158f3feeb42f
```

Flow: CreateDocumentationFlow
ID: 0f54b988-afe9-4dfb-9433-158f3feeb42f
Starting Flow...

Flow started with ID: 0f54b988-afe9-4dfb-9433-158f3feeb42f

Flow: CreateDocumentationFlow
ID: 0f54b988-afe9-4dfb-9433-158f3feeb42f
Starting Flow...
Running: clone_repo

Cloning repository: https://github.com/nikhil-1e9/machine-learning-daily

Cloning into 'workdir/machine-learning-daily'...

remote: Enumerating objects: 621, done.
remote: Counting objects: 100% (621/621), done.
remote: Compressing objects: 100% (516/516), done.
remote: Total 621 (delta 135), reused 570 (delta 87), pack-reused 0 (from 0)
Receiving objects: 100% (621/621), 1.74 MiB | 2.32 MiB/s, done.
Resolving deltas: 100% (135/135), done.

Flow: CreateDocumentationFlow
ID: 0f54b988-afe9-4dfb-9433-158f3feeb42f
Flow Method Step
Completed: clone_repo

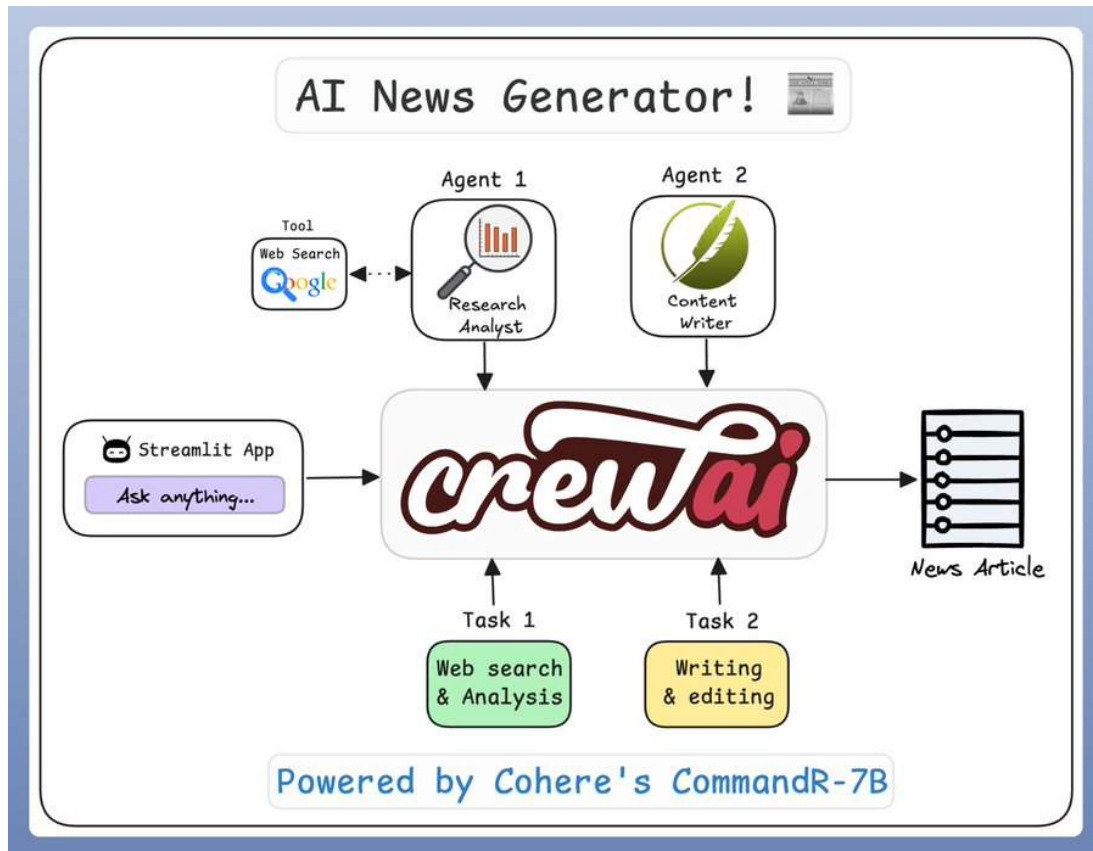
Flow: CreateDocumentationFlow
ID: 0f54b988-afe9-4dfb-9433-158f3feeb42f
Flow Method Step
Completed: clone_repo
Running: plan_docs



The code is available here:
<https://github.com/patchy631/ai-engineering-hub/tree/main/documentation-writer-flow>

#12) News Generator

The app takes a user query, searches the web for it, and turns it into a well-crafted news article, with citations.



Tech stack:

- Cohere ultra-fast Command R 7B as LLM
- CrewAI for multi-agent orchestration

Workflow:

We'll have two agents in this multi-agent app:

Research analyst agent:

- Accepts a user query.
- Uses the Serper web search tool to fetch results from the internet.
- Consolidates the results.

Content writer agent:

- Uses the curated results to prepare a polished, publication-ready article.

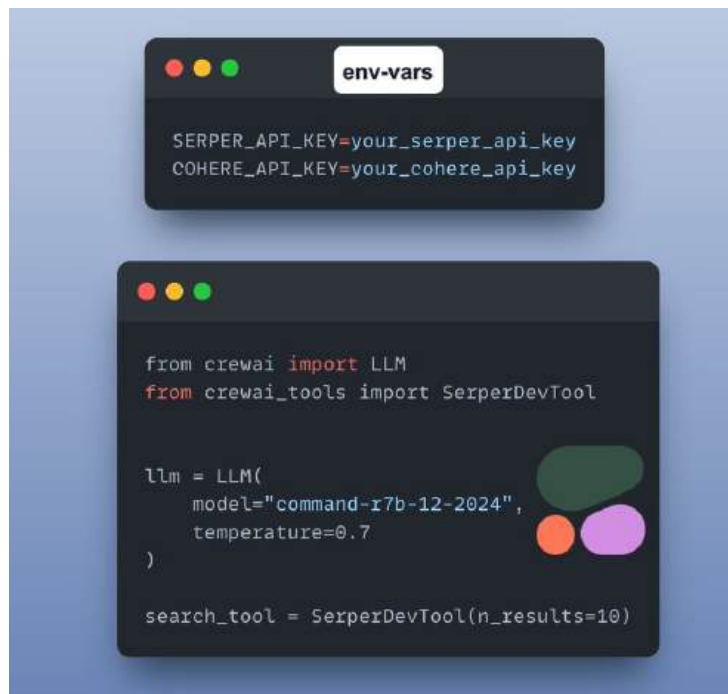
Let's implement this:

#1) Setup

Create a .env file for their corresponding API keys:

- Cohere API key
- Serper API key

Next, setup the LLM and web search tool as follows:



The image shows two terminal windows. The top window, titled 'env-vars', contains the following text: `SERPER_API_KEY=your_serper_api_key` and `COHERE_API_KEY=your_cohere_api_key`. The bottom window contains Python code for importing LLM and SerperDevTool, and initializing them with specific parameters.

```
from crewai import LLM
from crewai_tools import SerperDevTool

llm = LLM(
    model="command-r7b-12-2024",
    temperature=0.7
)

search_tool = SerperDevTool(n_results=10)
```

#2) Senior Research Analyst Agent

The web-search agent takes a user query and then uses the Serper web search tool to fetch results from the internet and consolidate them. Check this out:

```
from crewai import Agent

senior_research_analyst = Agent(
    role="Senior Research Analyst",
    goal="Research, analyze, and synthesize comprehensive information on {topic} from \n"
        "reliable web sources",
    backstory="You're an expert research analyst with advanced web research skills. "
        "You excel at finding, analyzing, and synthesizing information from "
        "across the internet using search tools. You're skilled at "
        "distinguishing reliable sources from unreliable ones, "
        "fact-checking, cross-referencing information, and "
        "identifying key patterns and insights. You provide "
        "well-organized research briefs with proper citations "
        "and source verification. Your analysis includes both "
        "raw data and interpreted insights, making complex "
        "information accessible and actionable.",
    allow_delegation=False,
    verbose=True,
    tools=[search_tool],
    llm=llm
)
```

A circular icon with a magnifying glass over a bar chart, with the text "Research Analyst" below it.

This is the research task that we assign to our senior research analyst agent, with description and expected output.

```
from crewai import Task

research_task = Task(
    description="""
        1. Conduct comprehensive research on {topic} including:
            - Recent developments and news
            - Key industry trends and innovations
            - Expert opinions and analyses
            - Statistical data and market insights
        2. Evaluate source credibility and fact-check all information
        3. Organize findings into a structured research brief
        4. Include all relevant citations and sources
    """,
    expected_output="""A detailed research report containing:
        - Executive summary of key findings
        - Comprehensive analysis of current trends and developments
        - List of verified facts and statistics
        - All citations and links to original sources
        - Clear categorization of main themes and patterns
        Please format with clear sections and bullet points for easy reference.""",
    agent=senior_research_analyst
)
```

#3) Content writer agent

The role of content writer is to use the curated results and turn them into a polished, publication-ready news article .

```
from crewai import Agent

# Second Agent: Content Writer
content_writer = Agent(
    role="Content Writer",
    goal="Transform research findings into engaging blog posts while maintaining accuracy",
    backstory="You're a skilled content writer specialized in creating "
        "engaging, accessible content from technical research. "
        "You work closely with the Senior Research Analyst and excel at maintaining \n"
        "the perfect balance between informative and entertaining writing, "
        "while ensuring all facts and citations from the research "
        "are properly incorporated. You have a talent for making "
        "complex topics approachable without oversimplifying them.",
    allow_delegation=False,
    verbose=True,
    llm=llm
)
```



This is how we describe the writing task with all the details and expected output:

```
from crewai import Task

# Writing Task
writing_task = Task(
    description="""
    Using the research brief provided, create an engaging blog post that:
    1. Transforms technical information into accessible content
    2. Maintains all factual accuracy and citations from the research
    3. Includes:
        - Attention-grabbing introduction
        - Well-structured body sections with clear headings
        - Compelling conclusion
    4. Preserves all source citations in [Source: URL] format
    5. Includes a References section at the end
    """,
    expected_output="""A polished blog post in markdown format that:
    - Engages readers while maintaining accuracy
    - Contains properly structured sections
    - Includes inline citations hyperlinked to the original source url
    - Presents information in an accessible yet informative way
    - Follows proper markdown formatting, use H1 for the title and H3 for the sub-sections""",
    agent=content_writer
)
```

#4) Setup Crew

And we're done!

Just build a crew and kick it off!

```
from crewai import Crew
from IPython.display import Markdown

# Create Crew
crew = Crew(
    agents=[senior_research_analyst, content_writer],
    tasks=[research_task, writing_task],
    verbose=True
)

result = crew.kickoff(inputs={"topic": "AI Trends in 2024"})

Markdown(result.raw)
```

The diagram illustrates the CrewAI workflow. A 'Topic' is input into the 'crewai' system. This system interacts with two agents: 'Agent 1' (Research Analyst) and 'Agent 2' (Content Writer). Agent 1 performs 'Task 1: Web search & Analysis', which involves using a 'Tool: Web Search'. Agent 2 performs 'Task 2: Writing & editing'. The outputs of these tasks are combined to produce a 'News Article'.

Unlocking the Future: AI Trends to Watch in 2024

Artificial Intelligence (AI) is no longer the stuff of science fiction. Several key trends are set to redefine the landscape of AI, making the pressing need for strong governance frameworks, let's explore them.

The Rise of Generative AI: A Creative Revolution

Generative AI is leading the AI revolution with its ability to create content, from text to images. This technology is democratizing creativity, allowing anyone to leverage algorithms that can mimic human creativity, business (https://www.coursera.org/articles/ai-trends).

Multimodal AI: Enhancing Human-Like Interactions

As AI systems evolve, they are increasingly integrating multiple modalities, enhancing the capability of AI to understand and generate intuitive and effective in fields like customer service and virtual assistants (the-state-of-ai).

Tailored AI Solutions: Customization at Its Best



The code is available here:
<https://www.dailydoseofds.com/p/hands-on-building-a-multi-agent-news-generator/>